



Высшая
школа
бизнеса

Лекция 2. Эволюция методологий описания процессов.

Нотации для детализированного
моделирования процессов: EPC, BPMN2.0.

Ярослав Витальевич Горчаков,
доцент Департамента бизнес-информатики ВШБ НИУ ВШЭ

ygorchakov@hse.ru

dr.gorchakov@mail.ru



Высшая
школа
бизнеса

Источники

Долганова, О. И. Моделирование бизнес-процессов : учебник и практикум для вузов, Тема 7, с. 150

<https://urait.ru/bcode/583398/p.150> - о нотации BPMN

Каменнова, М. С. Моделирование бизнес-процессов : учебник и практикум для вузов. П. 4.2.2. Событийная цепочка процесса, с. 165 <https://urait.ru/bcode/568546/p.165> - о нотации EPC

НОТАЦИЯ BPMN- ОФИЦИАЛЬНАЯ СПЕЦИФИКАЦИЯ:

<https://bpmn20.ru/docs/docly-documentation/elements/oficzialnaya-dokumentacziya-bpmn/>

<https://elma365.com/ru/bpmn2/?ysclid=mu173xt8di828727232>

<https://www.omg.org/spec/BPMN/2.0.2/About-BPMN.rdf>

<https://bpmn20.ru/doc/sample-user-guide>

<https://optimacons.info/upload/kb-articles/pdf/BPMN-2-0-poster-RUS.pdf>

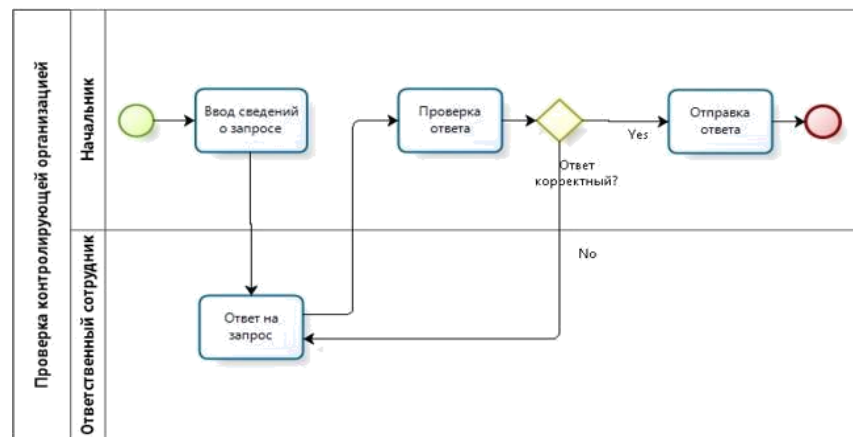
ПО ДЛЯ МОДЕЛИРОВАНИЯ BPMN

<https://bpmncat.ru>

<https://demo.bpmn.io/new>

МОДЕЛЬ И НОТАЦИЯ

Модель бизнес-процесса может быть представлена как простыми схемами, так и общеизвестными **нотациями**, часть из которых принята в профессиональной среде ИТ-специалистов и управленцев различного профиля



НОТАЦИЯ – способ моделирования предметной области деятельности, содержащий определённый набор графических символов (условных обозначений, «аналогов» реальных объектов) и связей между ними, в соответствии с методологически заданными правилами использования этих символов.

НОТАЦИЯ ≠ МОДЕЛЬ

Распространенные нотации:

EPIC, BPMN2.0., VAD, IDEF0, DFD, VSM, Flowchart (блок-схема процесса) и др.

Нотация может представлять собой самостоятельную методологию или быть частью методологии.



МЕТОДОЛОГИИ МОДЕЛИРОВАНИЯ ОРГАНИЗАЦИИ КАК СИСТЕМЫ

(Громов А.И., Чеботарев В.Г.)

Методология - теоретическая научная основа

(учение, концепция, общий принцип).

Часто содержит различные *методики, способы* ит.д.

КОМПОНЕНТЫ МЕТОДОЛОГИИ МОДЕЛИРОВАНИЯ ПРЕДМЕТНЫХ ОБЛАСТЕЙ ОРГАНИЗАЦИИ:

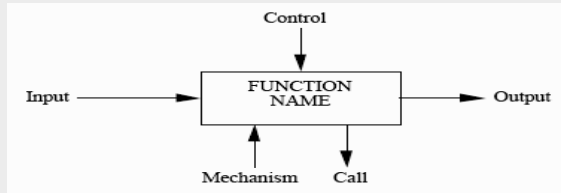
1. *Науки, направления (философия, психология, системный анализ, науковедение, этика, эстетика)*
2. *Характеристики деятельности (особенности, принципы, условия, нормы деятельности)*
3. *Логическая структура деятельности (предмет, субъект, объект, формы, средства, методы, результат деятельности)*
4. *Аспекты деятельности, связанные с временем (фазы, стадии, этапы)*

**Для разных методологий наиболее заметны отличия
во втором и в третьем компонентах**

Эволюция методологий моделирования

(Громов А.И., Чеботарев В.Г.)

Методологии структурного подхода



SADT, IDEF, DFD ...

описание системных требований и последовательности действий для реализации поставленных перед системой задач.

Методологии объектно-ориентированного подхода



UML –
стандартизированный
язык моделирования

Создание информационных систем. Описание спецификаций и реализация

Субъектно-ориентированные



С 2010-х гг.- описание взаимодействия субъектов между собой, на основе чего «от частного к общему» формируются требования для реализации процесса на основе модели

Методологии,
ориентированные на
бизнес-процессы

ОРИЕНТИРОВАННЫЕ НА WORKFLOW



• с 2000-х по настоящее время

• ARIS (VAD, EPC)



объект (процесс, функция).
описание системных требований,
спецификаций и реализации с
детализацией

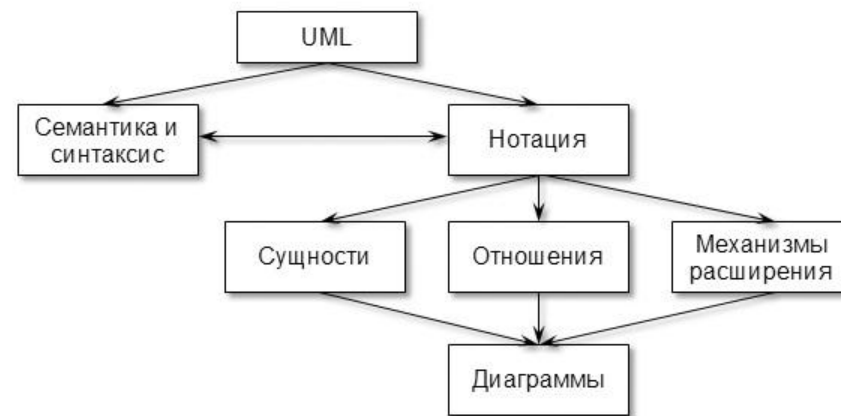
детализация процессов,
не описывает другие
предметные области

Множество моделей, принцип
«от общего к частному»



UML (Unified Modeling Language) - унифицированный язык моделирования

- Подходит для конструирования программных систем (больше для программистов). В языке UML есть 12 типов диаграмм:
 - 4 типа диаграмм представляют **статическую структуру** приложения;
 - 5 типов представляют **поведенческие аспекты** системы;
 - 3 представляют **физические аспекты** функционирования системы (диаграммы реализации).
- При помощи диаграмм разного типа можно изобразить в графическом виде проектируемую систему
- Однократная автоматизация – трансформация модели в исполняемый код





SADT, IDEF



Дуглас Росс

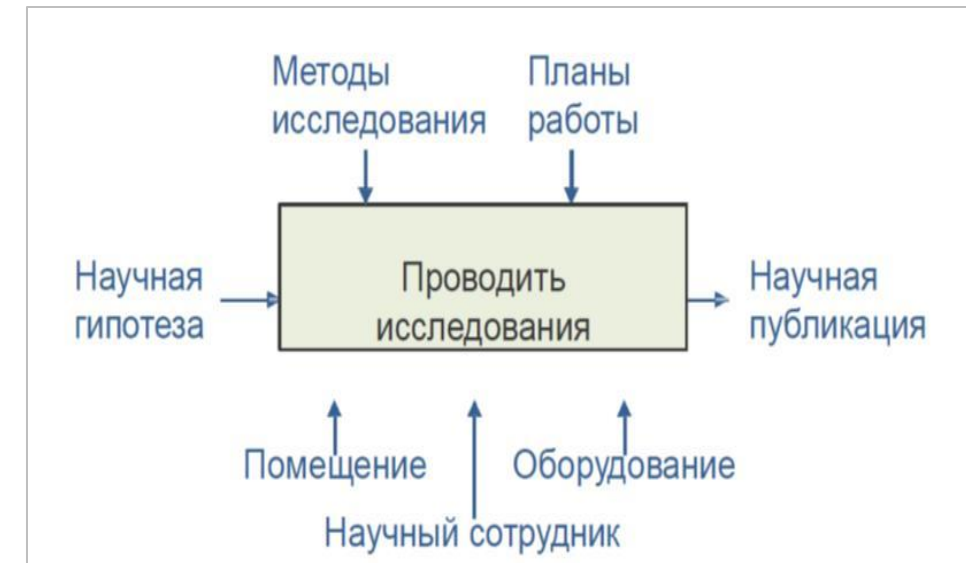
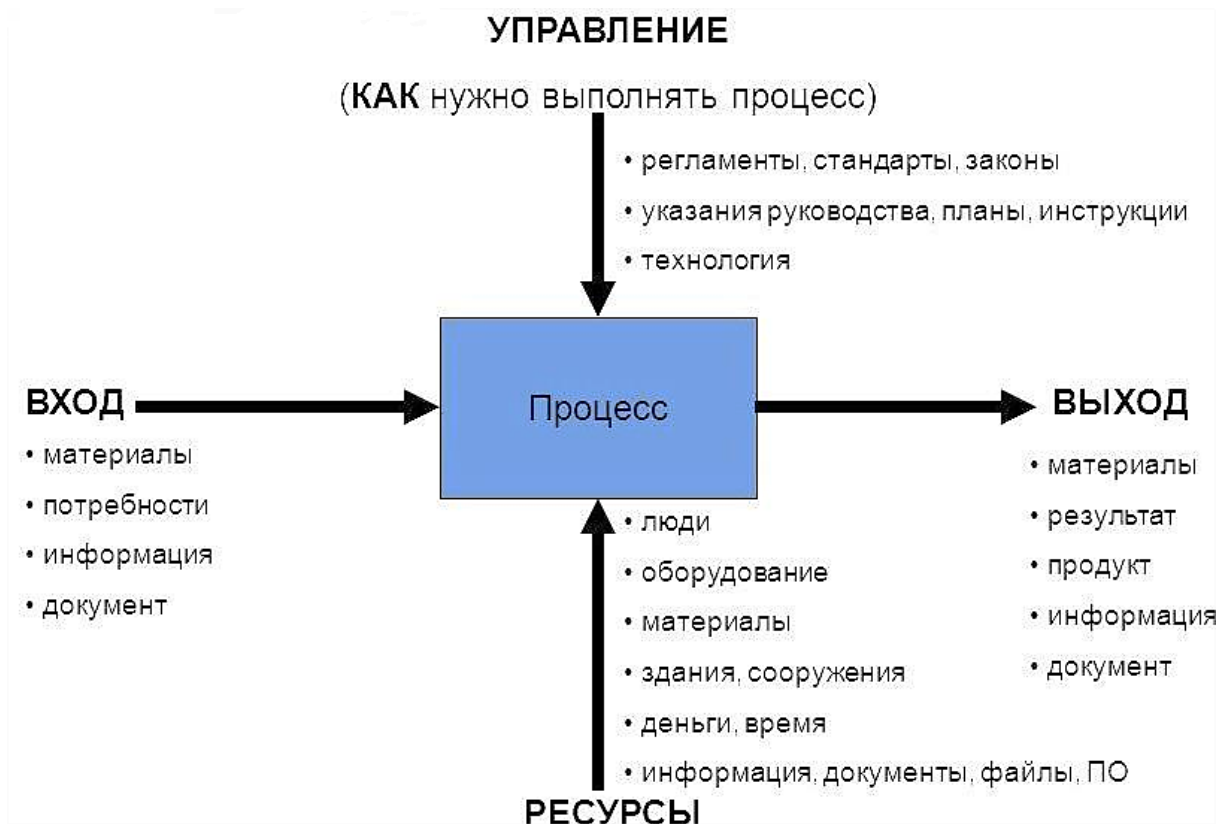
<https://www.computerhistory.org>

- В 1969 г. Дуглас Росс разработал **SADT** - методологию структурного анализа (Structured Analysis and Design Technique)
- **ICAM** (Integrated Computer Aided Manufacturing) – программа компьютеризации производства, развивалась в США в 1970-ых годах
- **IDEF** (ICAM Definition) - комплекс методологий для анализа и проектирования искусственных систем средней сложности

Часть SADT была принята в качестве стандарта и получила название **IDEF0**

IDEFO

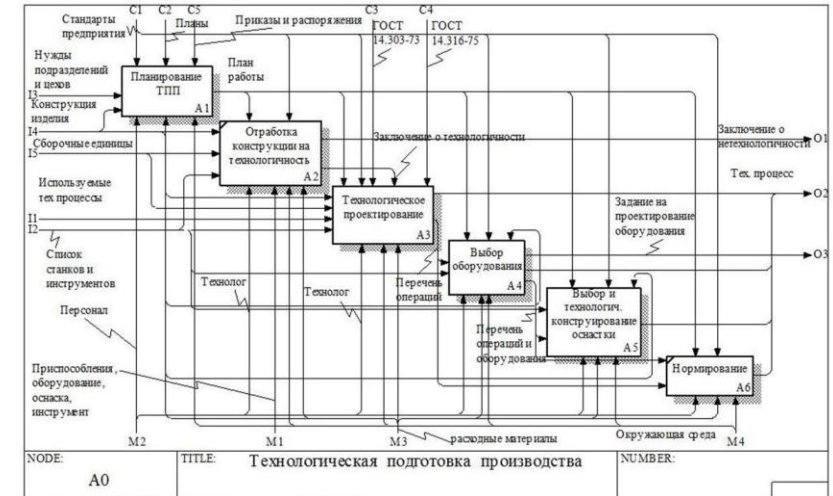
- Нотация внесла большой вклад в понимание функционального взаимодействия
- Реализовала понятие декомпозиции, разных уровней моделирования



Плюсы IDEF0



- Широко распространена, длительный опыт использования в различных областях
- Существует множество книг и методических материалов
- Есть много программных продуктов, поддерживающих моделирование в нотации



Недостатки IDEF0

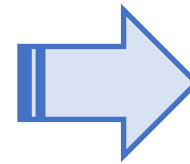
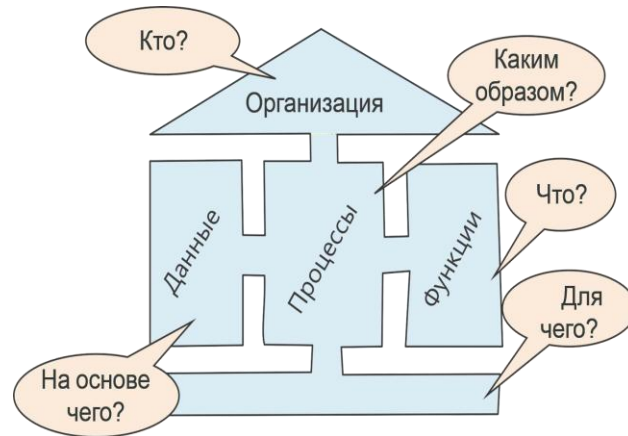
- Диаграммы IDEF0 часто оказываются слишком сложными для восприятия руководителями и топ-менеджерами
- Модель IDEF0 не содержит информации о времени и последовательности выполнения функций



В ПО **BUSINESS STUDIO** используется для моделирования процессов верхнего уровня

МЕТОДОЛОГИЯ и ПО ARIS

ARIS = ARchitecture of Integrated Information Systems



**Российский аналог
методологии и ПО**

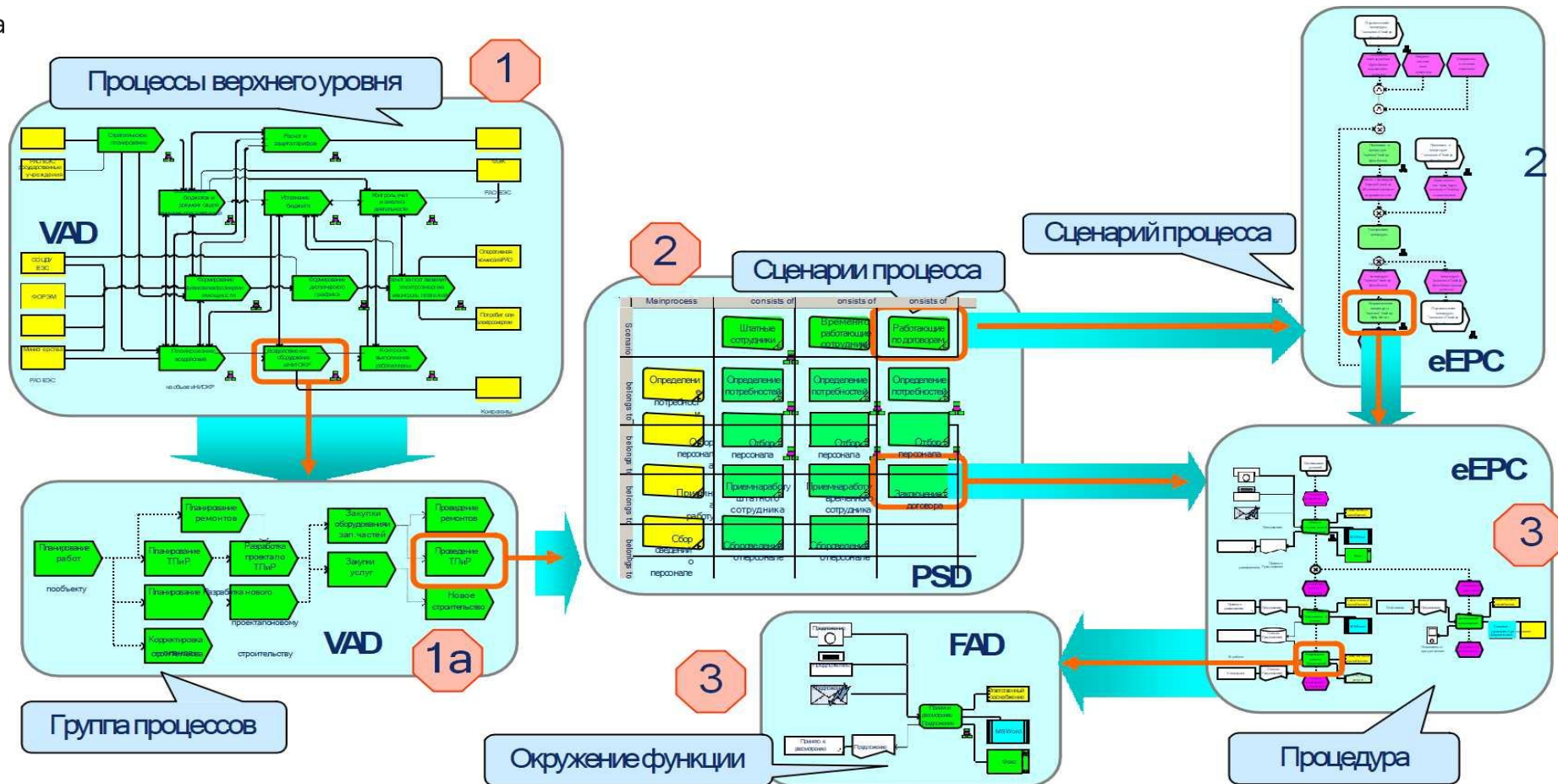
Методология ARIS - структурированное описание деятельности организации с различных, взаимодополняющих точек зрения и представление её в виде взаимосвязанных графических моделей, удобных для понимания и анализа

Объединила несколько методик и нотаций, подходов:

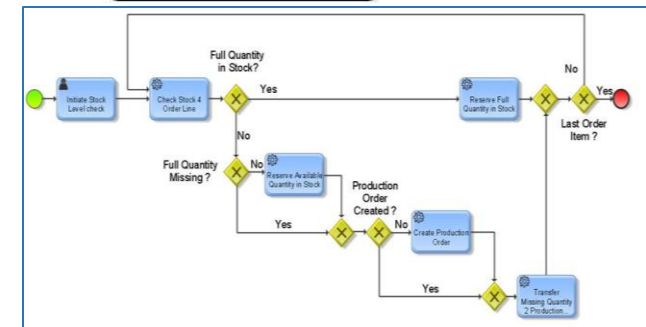
событийная цепочка процесса (eEPC), модель «сущность-связь» (ER- Model), язык UML, методика объектного моделирования OMT, методика построения сбалансированной системы показателей BSC (Balanced Scorecard).

В последних версиях инструментария ARIS также поддерживаются диаграммы в нотации BPMN 2.0., изначально не присущие самой методологии ARIS

УРОВНИ И НОТАЦИИ ПРОЦЕССОВ по методологии ARIS



+ BPMN 2.0.

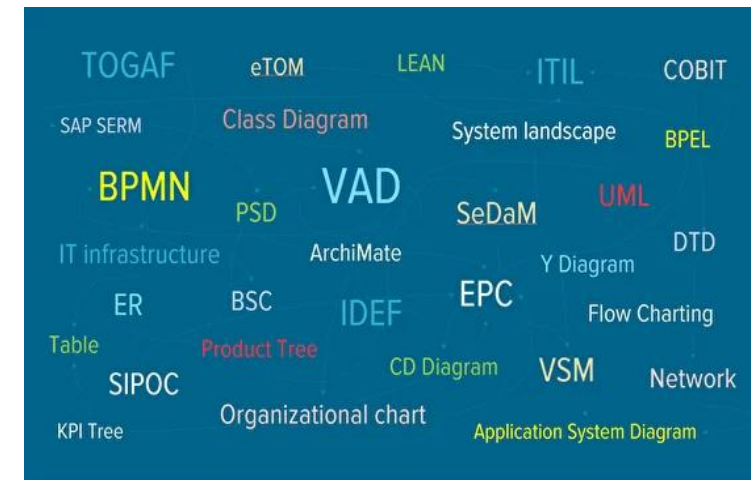
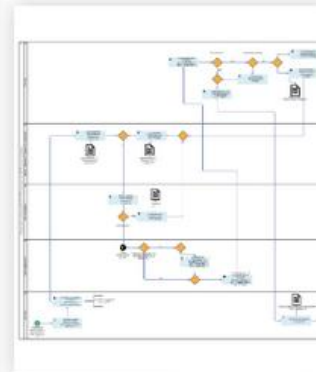
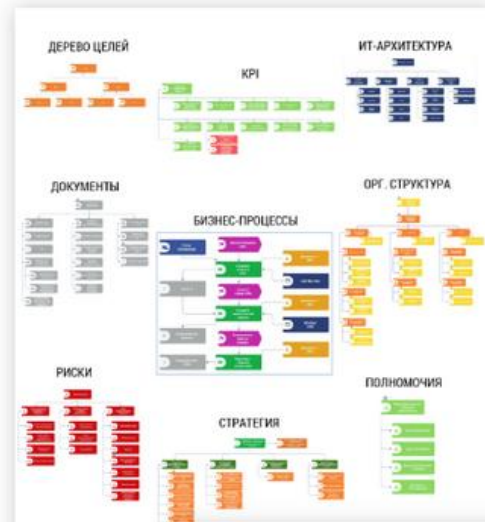


Инструментарий SILA Union - создание «цифрового двойника»

позволяет комплексно смоделировать деятельность организации:

- Отрисовать бизнес-процессы для унификации деятельности, а также последующей автоматизации и регламентации
- Смоделировать другие плоскости бизнеса (ИТ-архитектура, орг. структура, стратегия и пр.) и связать их с бизнес-процессами, построить зависимости и оценить степень влияния

<https://silaunion.ru/solutions/bp-modeling>





Высшая
школа
бизнеса

НОТАЦИЯ ЕРС*

EVENT-DRIVEN PROCESS CHAIN - СОБЫТИЙНАЯ ЦЕПОЧКА ПРОЦЕССОВ

* eEPC – extended, расширенная версия



Высшая
школа
бизнеса

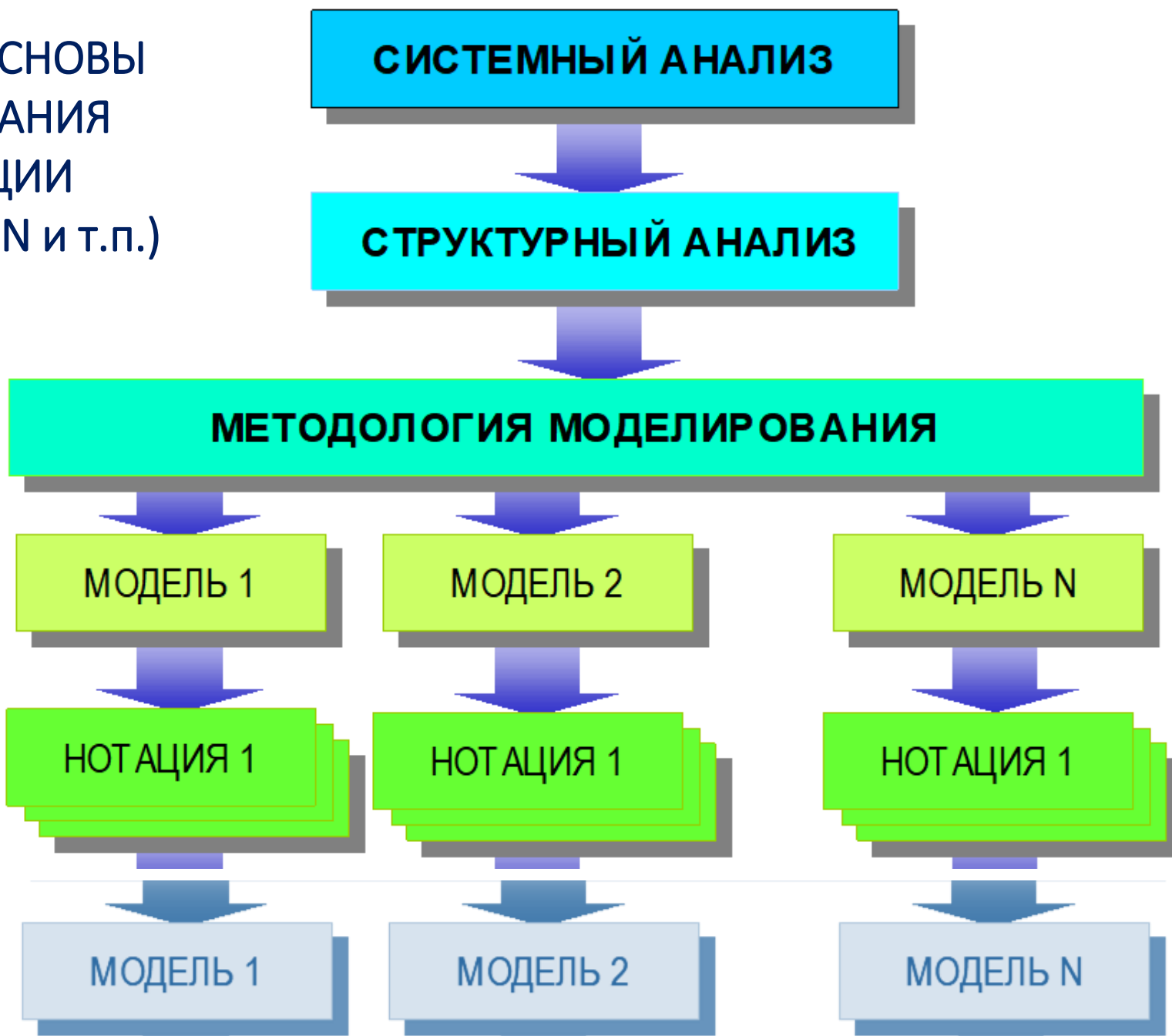
СИСТЕМНЫЕ ОСНОВЫ МОДЕЛИРОВАНИЯ ОРГАНИЗАЦИИ (ARIS, SILA UNION и т.п.)

НАУЧНАЯ ОСНОВА, ОБЩИЕ
ПРИНЦИПЫ И ПОДХОДЫ

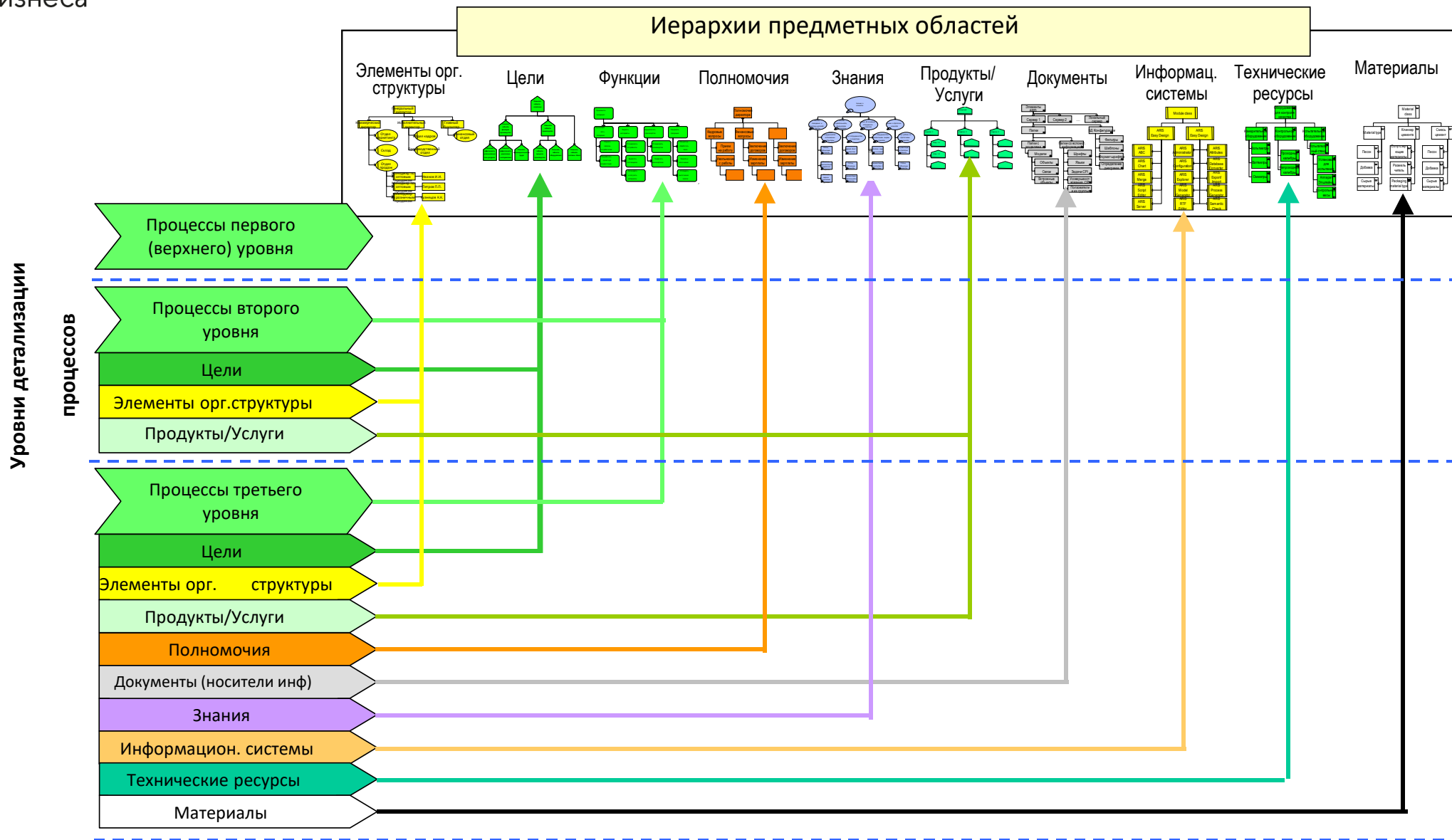
ПРЕДМЕТНЫЕ ОБЛАСТИ
МОДЕЛИРОВАНИЯ

С ПОМОЩЬЮ ЧЕГО, КАКИМ
СПОСОБОМ МОДЕЛИРУЕМ

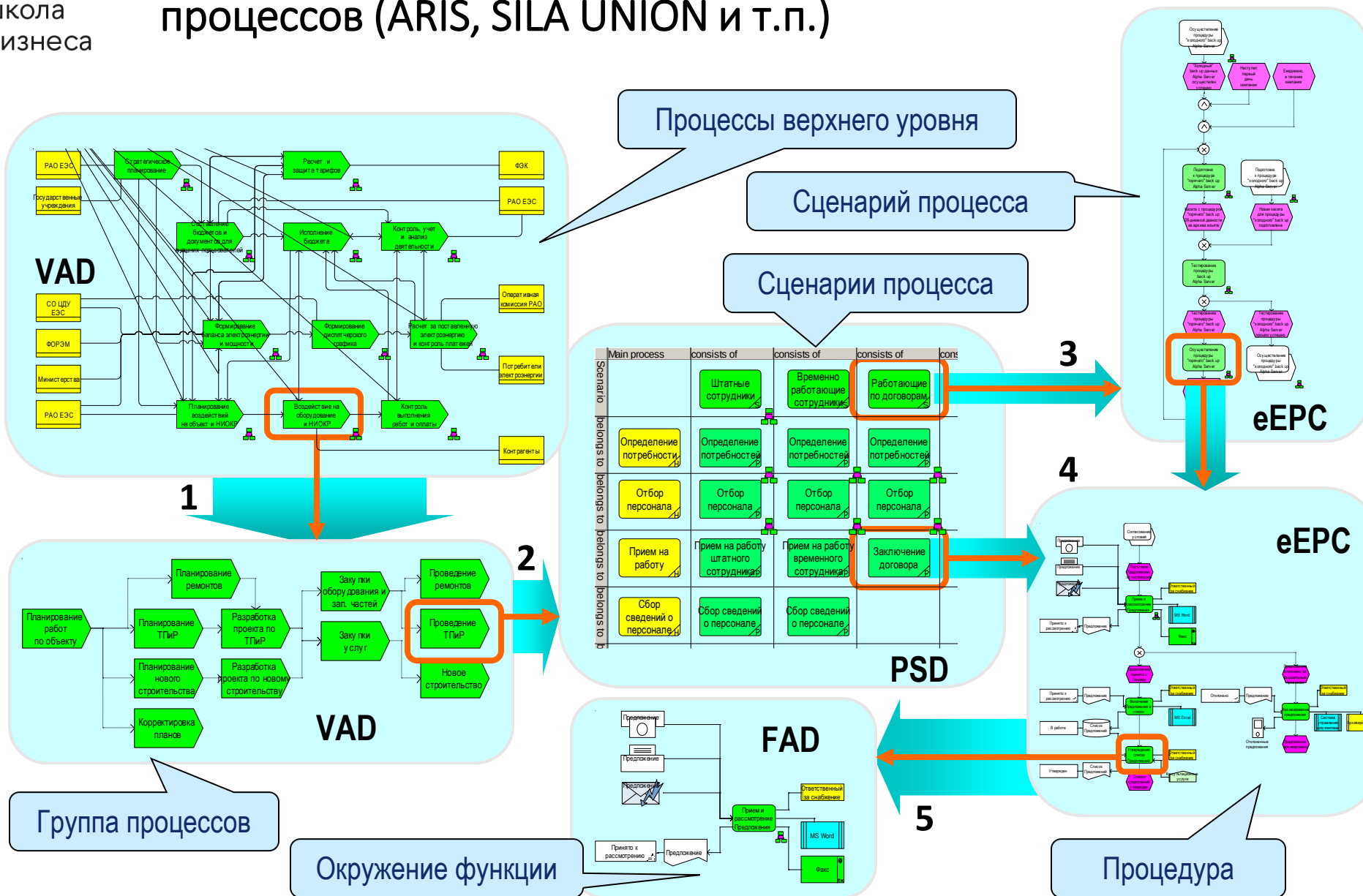
ГОТОВЫЕ МОДЕЛИ



Иерархическое описание деятельности организации по методологии ARIS (пример)



Пример методики иерархического описания процессов (ARIS, SILA UNION и т.п.)





ОБЪЕКТЫ СОБЫТИЙНОЙ ЦЕПОЧКИ ПРОЦЕССОВ (ЕРС): ФУНКЦИИ

Оформление
заказа

клиента

Функции, действия – Function, Activity

Согласовать
договор

Функция (Function) – элемент процесса, его этап (шаг), конкретное выполняемое кем-то (людьми) или чем-то (технологией, системой, оборудованием) на данном уровне рассмотрения (описания)

Изучение
рынка

Имя функции – глагол (что сделать) + хотя бы одно существительное (с чем сделать)

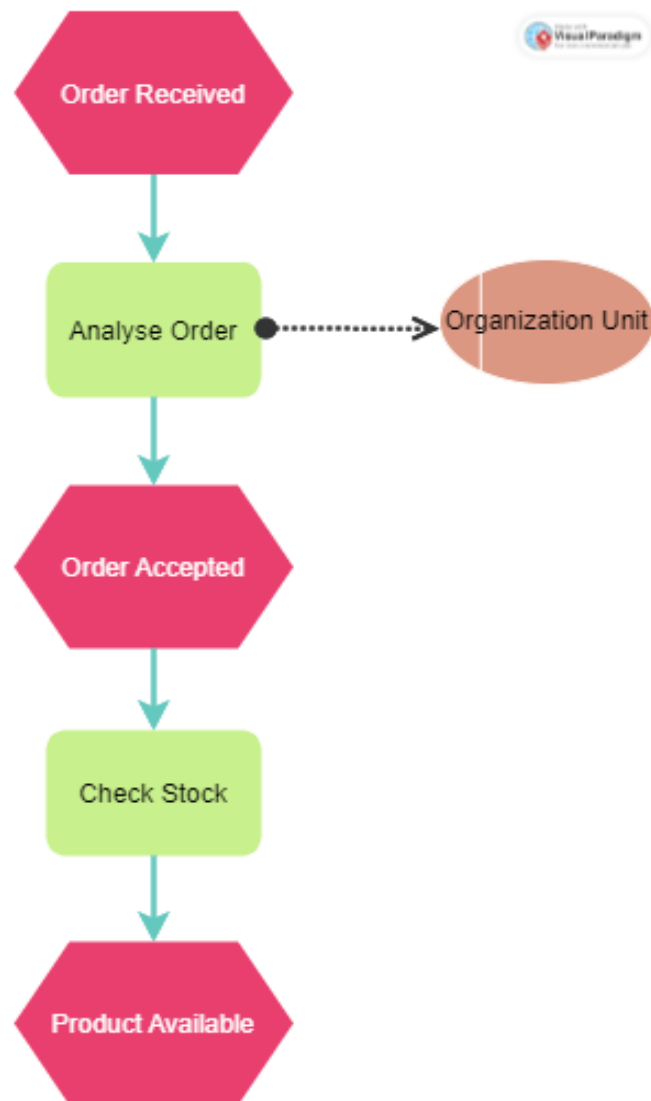
другой вариант – отглагольное существительное + существительное

Сборка
автомобиля



В рамках одной модели должен использоваться только один вариант наименования функций

Объекты (элементы) событийной цепочки процессов (EPC): Событие



- Каждое действие (функция) должно инициироваться (включаться) событием и завершаться событием
- В каждую функцию не может входить более одной связи (стрелки), «запускающей» выполнение функции, и выходить более одной стрелки, описывающей завершение выполнения функции
- С функцией связана бизнес-роль, должность или организационная единица (подразделение, отдел)

События и их свойства

Заявка от
клиента
поступила

Документ
составлен
неверно

Решение о
приеме
сотрудника
принято

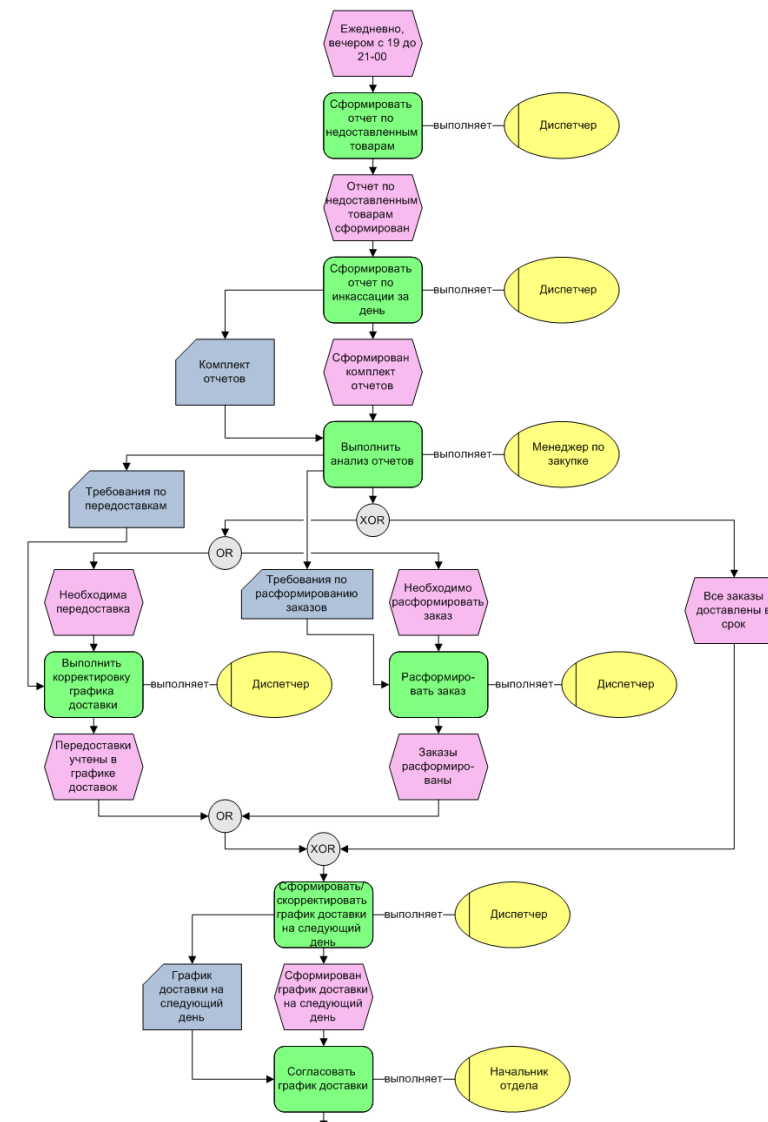
Любой процесс начинается и заканчивается событием (или интерфейсом в другой процесс)

Имя события – именная группа + глагол совершенного вида

Событие - необходимое условие начала и окончания выполнения функции

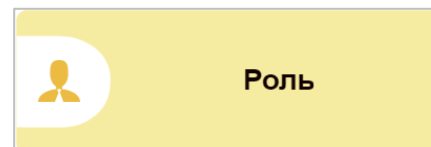
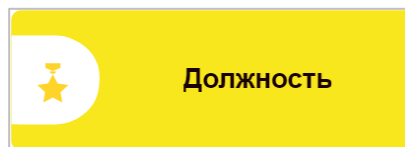
Событие отражает результат выполнения функции

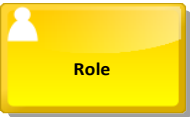
Объект, описывающий событие – это констатация факта изменения состояния системы





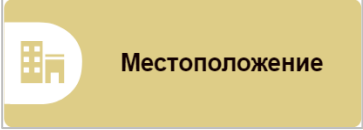
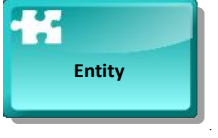
Часто используемые элементы нотации ЕРС

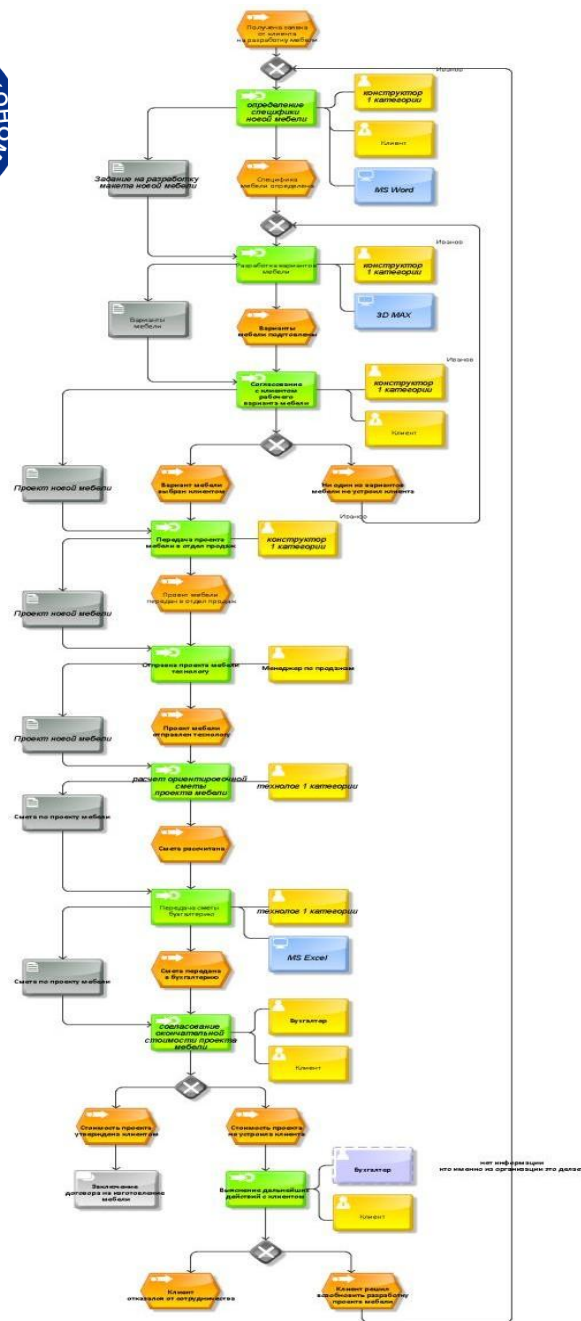


Должность (Role)		Объект соответствует должности. Также может применяться к бизнес-роли Наименование объекта соответствует наименованию штатной единицы в организационной структуре. Все названия объектов пишутся с заглавной буквы Пример: Экономист
Интерфейс процесса - Функция (Process interface -Function)	 	Объект показывает ссылку на смежный процесс. Имя соответствует названию модели, которая описывает смежную процедуру из модели сценариев процесса. Все названия объектов пишутся с заглавной буквы. Объект используется для ссылок на процедуры, не входящие в описываемый сценарий данной (как название предшествующего или последующего процесса) - указывается либо перед первым событием, либо после последнего события в процессе.
Документ (document)	 	Объект, отражающий реальные носители информации, например, бумажный документ. Также используется при неопределенном типе носителя (как гипотеза, напр., файл). Может быть входом или выходом функции. Пример: «Заказ», «Опросник», «Анкета», «Характеристика», «Сопроводительный документ», «Договор», «Паспорт продукта» и др.
IT-система (IT system)		Объект соответствует конкретной информационной системе, отражает реальную прикладную систему, программу. Пример: MS Office, Excel, Autocad, 1C и др.



Дополнительные элементы диаграммы EPC для детализации описания процесса

Данные (база данных)		Данные, файлы и т.п., создающиеся или используемые в процессе. Входы-выходы функций (шагов). Также используется для отображения названий используемых баз данных в процессе. Примеры: «Экранная форма», «База ARIS», «Электронный заказ клиента», «База поставщиков», «База договоров» и др.
Риск		Потенциальные угрозы, ограничивающие эффективность процесса. Возможная опасность того, что процесс (или его этап, функция) не достигнет цели или риск влияния чего-либо (напр., функции) каким-либо негативным образом на состояние системы. Напр., «Риск потери клиента»
Продукт, Товар		Результат человеческой деятельности или технической функции. Продукт также может быть услугой. Инициатором создания продукта является требование от организационной единицы и/или клиента процесса, также объект может отображать результат, соответствующий цели процесса
Расположение (Location)	 	Обозначает фактическое место, где совершается функция, процесс (здание, цех, кабинет, номер комнаты и др.)
Сущность (также может означать Цель или задачу)		Реальные или абстрактные сущности (предметы, явления), которые представляют интерес для текущих задач организации: конкретные - клиенты, товары, звонки и абстрактные - агент отвечает за клиента, клиент записан и т.д. Объект является индивидуально, специфически идентифицируемым объектом реальности. В процессах используется также в случаях, когда сложно определить специфику объекта другого типа, т.е. вместо других объектов (преодоление сложности системы, упрощение). В процессах обозначают также то, о чем нужно хранить информацию в базе данных, в процессах обработки данных, в моделях данных.



Правила размещения элементов EPC

- диаграмма изображается в направлении сверху-вниз на стандартном листе A4 или A3 в виде цепочки выполнения процесса;
- первым и последним объектами цепочки являются события;
- в цепочке процесса каждая функция начинается и заканчивается событием;
- связи между событиями и функциями изображаются в виде стрелок в направлении выполнения процесса;
- справа от функций располагаются прикрепленные к ним должности и при необходимости информационные системы, с помощью которых выполняются функции;
- связи между должностями (информационными системами) и функциями изображаются в виде сплошных линий;
- слева от функции располагаются входные и выходные документы (продукты/услуги), входные (используемые в ходе выполнения) – выше функции, выходные (создаваемые на выходе) – ниже функции;
- связи между документами и функциями изображаются стрелками (для входных – от документа (продукта/услуги) к функции, для выходных – от функции к документу (продукту/услуге)).



Модель и экземпляр бизнес-процесса

- **Экземпляр процесса** – фактически реализуемый вариант; цепочка действий, связанных с обслуживанием конкретного потребителя или производством конкретной единицы продукции
- Экземпляр всегда имеет привязку к времени.
- Например, обслуживание клиента А по понедельникам, вторникам – экземпляр 1, по средам – экземпляр 2 и т.п.
- Важно собирать статистику по реализованным экземплярам процессов за единицу времени.
- Экземпляр часто является вариантом сценария процесса



Логические операторы – используются для ветвления процесса



Оператор И - Правило
(AND rule)

Логический оператор «И», используется в случае, если в результате наступления какого-либо события начинается параллельное выполнение двух и более функций, либо в случае, если для начала выполнения функции требуется наступление двух и более событий



Оператор исключающее ИЛИ - Правило
(XOR rule)

Логический оператор «Исключающее ИЛИ», используется в случае, если для начала выполнения функции требуется наступление одного из двух и более событий либо в результате выполнения функции наступает одно из двух и более событий



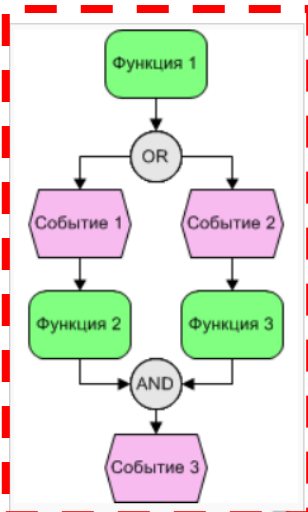
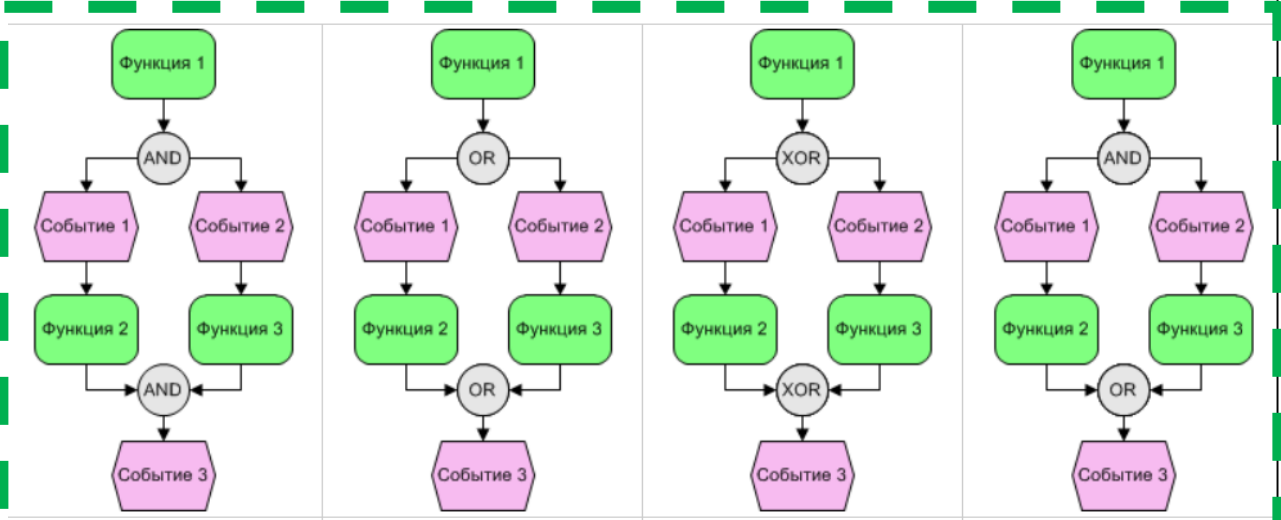
Оператор ИЛИ - Правило
(OR rule)

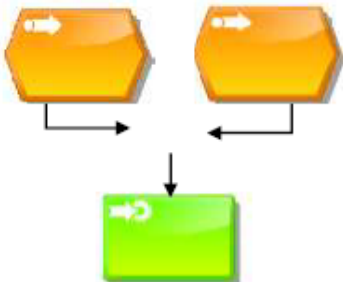
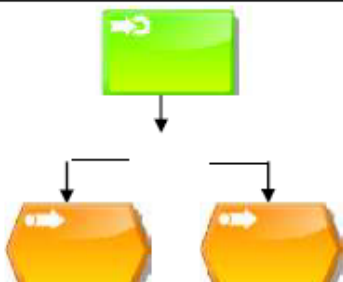
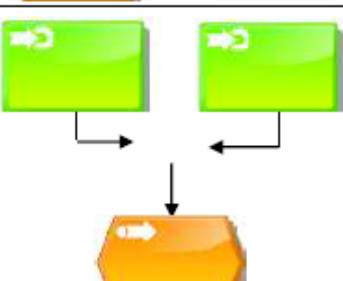
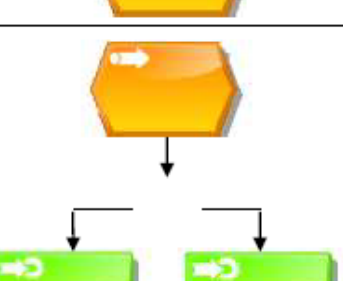
Логический оператор «ИЛИ», используется в случае, если для начала выполнения функции требуется наступление хотя бы одного из двух и более событий либо в результате выполнения функции наступает хотя бы одно из двух и более событий



Высшая
школа
бизнеса

Правила ветвления и использования логических операторов

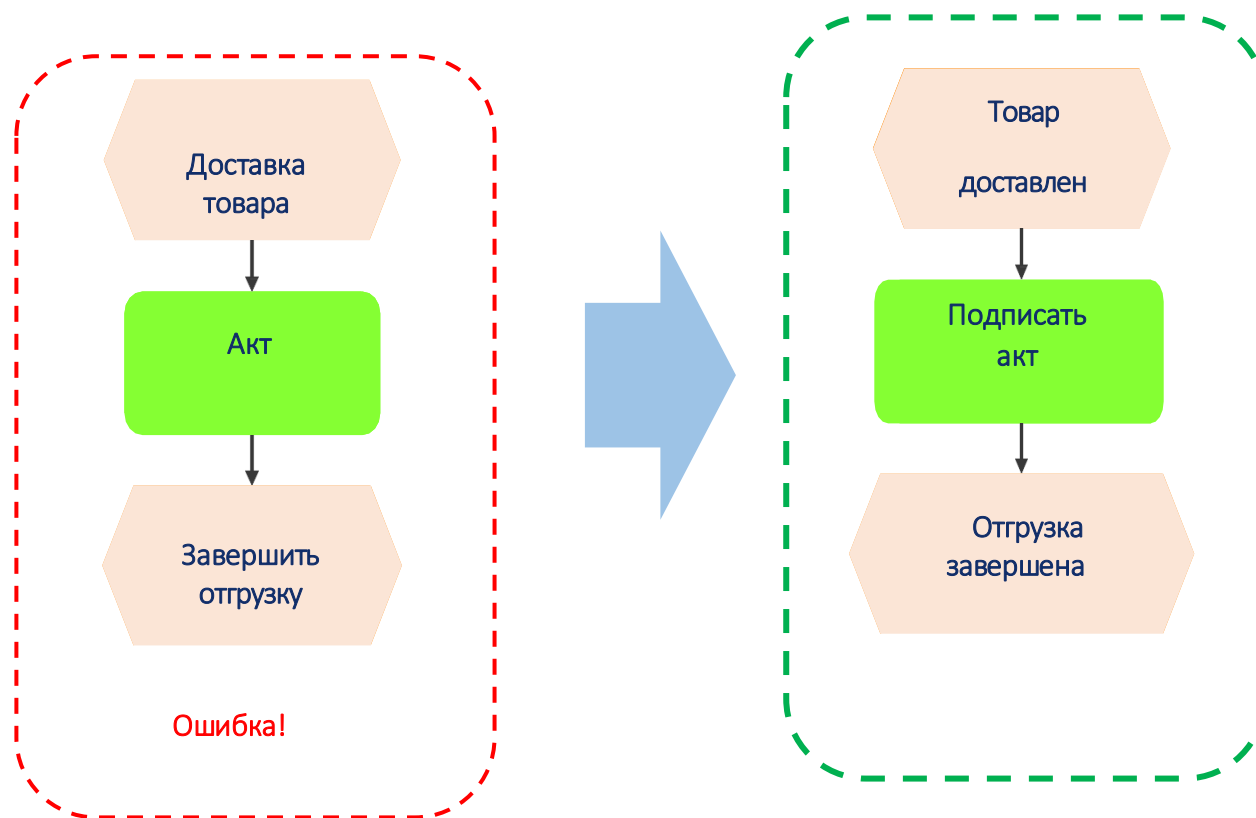


Типы ветвлений	 («И»)	 (исключающее «или»)	 («или»)
	Функция выполняется, если наступили все события	Функция начинается выполняться тогда, когда наступает только одно из событий	Функция начинает выполняться, если хотя бы одно из событий наступает
	После выполнения функции наступают все события	После выполнения функции наступает ровно одно из событий	После выполнения функций наступает хотя бы одно из событий
	Событие наступает, когда выполнены обе функции	Событие наступает после выполнения ровно одной функции	Событие наступает после выполнения хотя бы одной функции
	При наступлении событий обе функции выполняются	Не разрешено, поскольку событие не может принимать решения (только в функции принимают решения)	



ОШИБКИ В НАИМЕНОВАНИЯХ

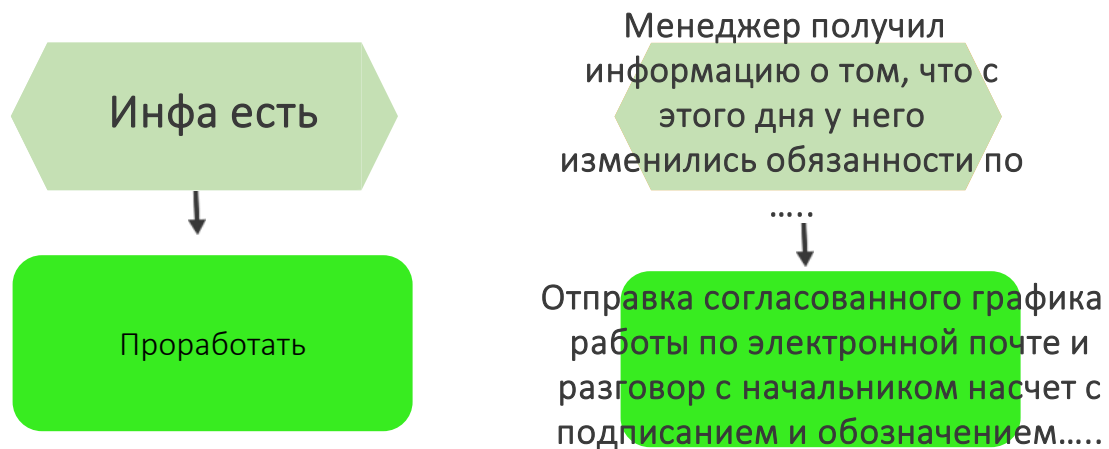
Наименования (атрибут Name) всегда должны соответствовать типу сущности элемента. Нельзя называть событие как действие и наоборот, названия должны быть полными для понимания сути действия, события и др.





Высшая
школа
бизнеса

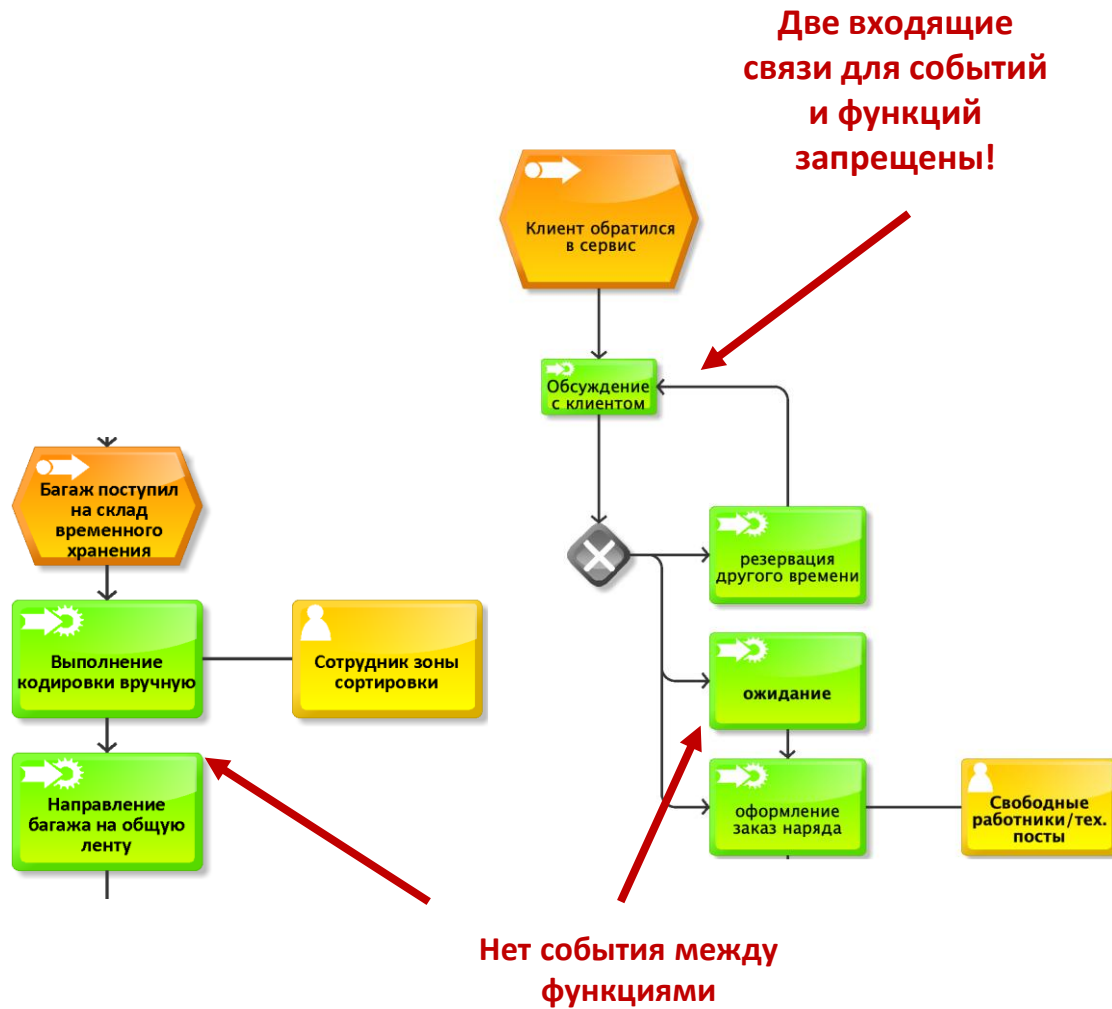
Ошибка недостаточности или избыточности наименования элементов



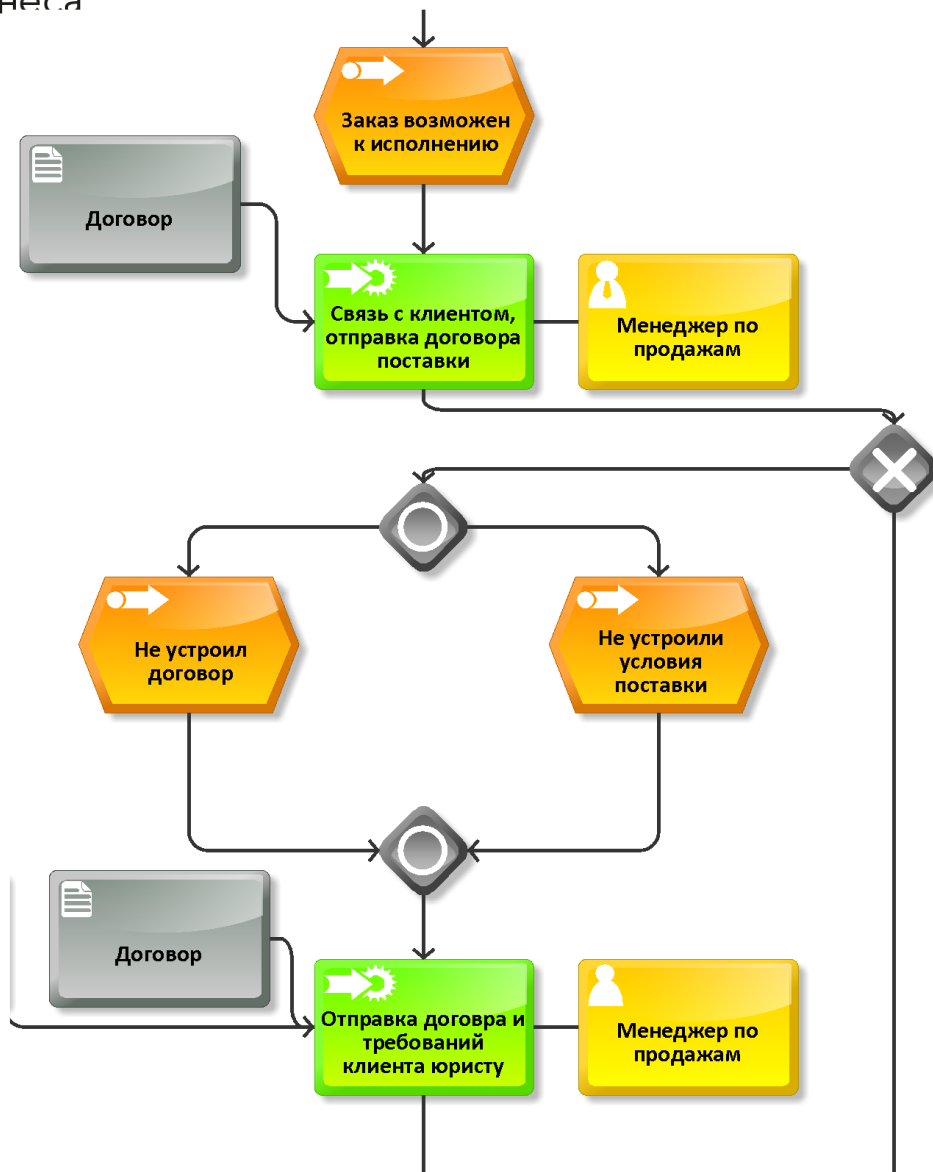


Высшая
школа
бизнеса

Ошибки методологические: отсутствие необходимых элементов и неверное использование связей



Правило исключения «избыточных» ветвлений:



Даже если по описанию наступают два события, напр., по правилу «Или», но при этом:

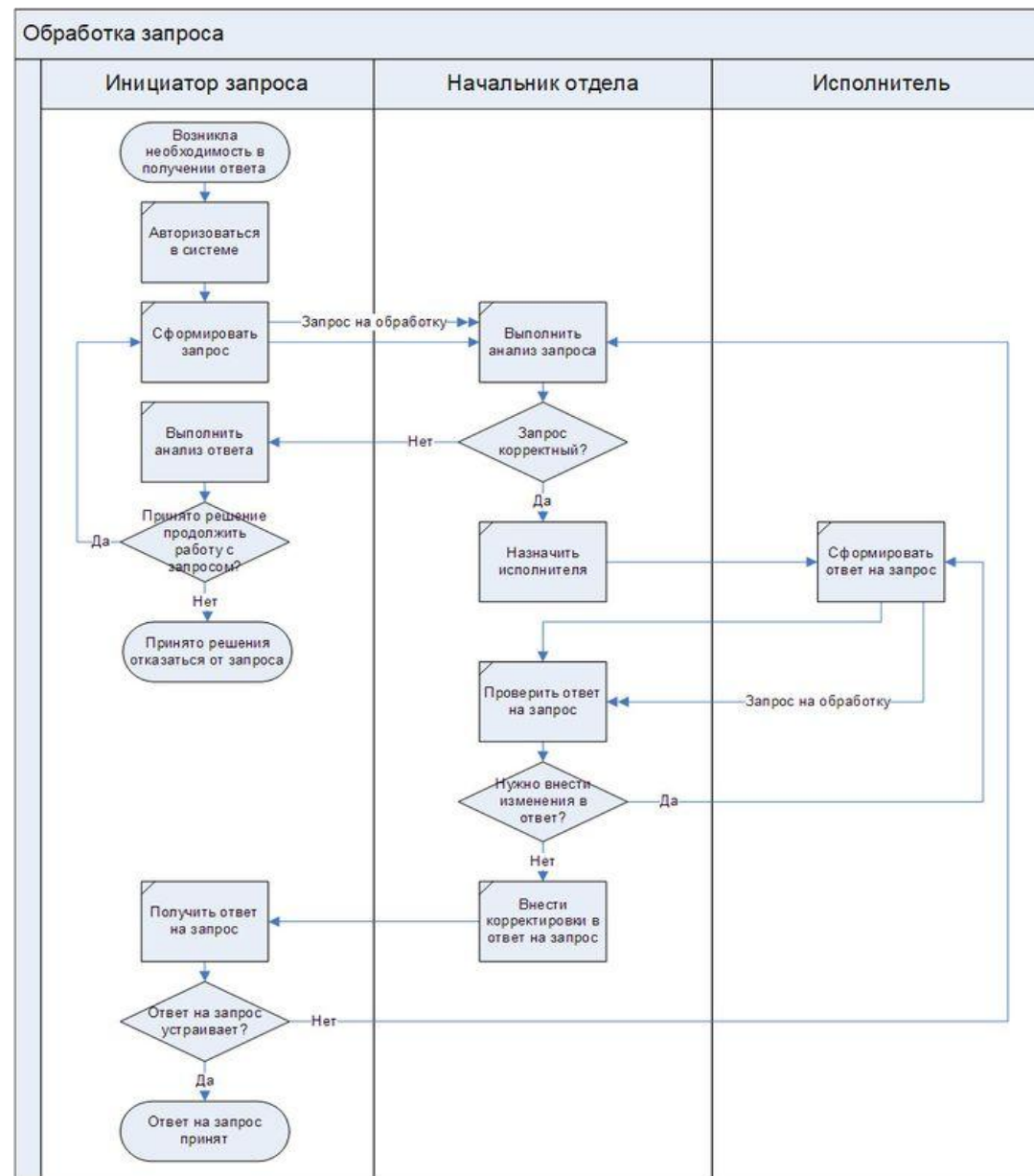
- a) События ни на что не влияют – не создают соответствующих функций
- b) События – результат функций одних и тех же участников событий
- c) События «похожи» как результат одной функции и по смыслу являются одним и тем же фактом
- d) Нет дополнительных выходов и т.п., то СОБЫТИЯ НУЖНО ОБЪЕДИНИТЬ В ОДНО!

**В примере – одно событие
«Не устроили условия поставки в
договоре»**



Похожая нотация - Flowchart

- **Стрелки** показывают направление движения процесса
- Элемент **Решение** - ромб, обозначающий вопрос, на который необходимо ответить, например, да / нет или истина / ложь
- Элемент **Процесс** - это прямоугольник, представляющий процесс, действие или операцию.
- Овал представляет начальную или конечную точки потоковой диаграммы





Высшая
школа
бизнеса

Выводы по нотации EPC

Преимущества:

- Простота восприятия, наглядность. Широкая распространенность, много примеров
- Возможность чётко отследить передвижение входных и выходных потоков, видны все ресурсы, документы, информационные системы, базы данных и т.д. в процессе
- Удобно контролировать процесс - каждая функция приводит систему в новое состояние, следовательно, после выполнения каждой функции систему можно проверить на фактический переход в нужное состояние.
- Простота в использовании и при изучении: набор ключевых элементов и правил достаточно невелик.

Недостатки:

- Каждая функция порождает событие, что ведёт к нагромождению диаграммы и увеличению её по вертикали.
- Сложно отследить объём работ у конкретного сотрудника/департамента, поскольку исполнители указываются у каждой функции, для определения объёма работ у каждого исполнителя придётся неоднократно просмотреть всю диаграмму.
- Отсутствие типов событий – события отличаются только названием, могут быть ошибки описания из-за некорректных названий элементов модели



Высшая
школа
бизнеса

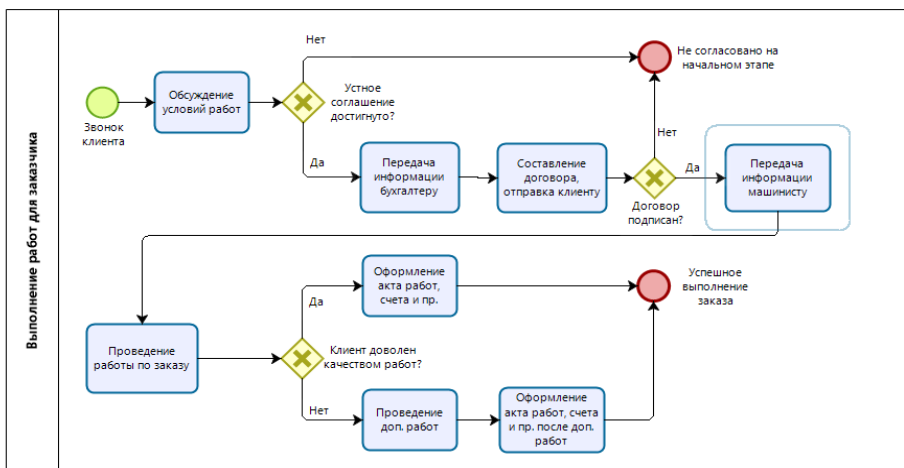
Нотация BPMN 2.0.



BPMN (Business Process Model And Notation)

(нотация и модель бизнес-процессов) – это открытая система условных обозначений для описания бизнес-процессов, состоит из более чем 100 визуальных элементов и соответствующих правил

Основной целью разработки BPMN было получение нотации, легко понимаемой всеми пользователями: от рядовых сотрудников, бизнес-аналитиков и менеджеров нижнего звена до ИТ-специалистов, отвечающих за реализацию этих процессов в ИС и руководителей организации

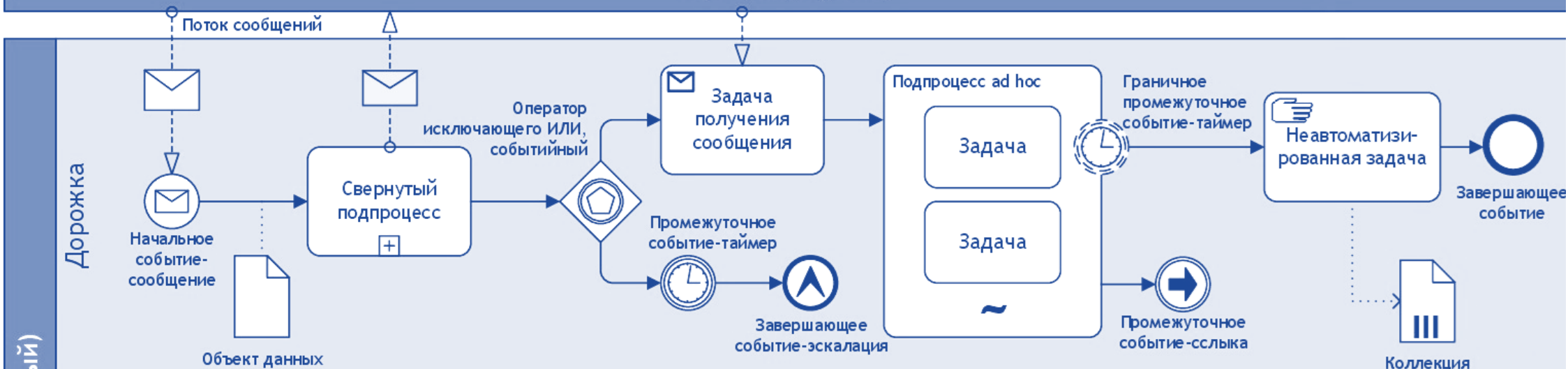


BPM (BUSINESS PROCESS MANAGEMENT) – КОНЦЕПЦИЯ УПРАВЛЕНИЯ БИЗНЕС-ПРОЦЕССАМИ

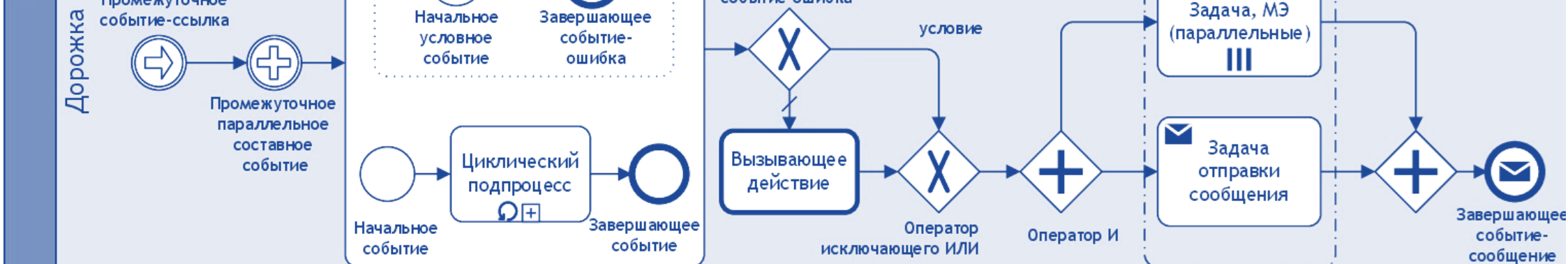
BPMS:

- a) Business Process Modeling System) – инструменты для создания и исполнения созданных моделей: Bizagi, Questetra, TIBCO, Comundo, ELMA и др.
- b) Business Process Management System – система управления бизнес-процессами
- c) Business Process Model Suite – программное обеспечение для автоматизации бизнес-процессов

Пул (свернутый)



Пул (Развернутый)





История создания

BPMPN — спецификация, разработанная в 2001-2004 годах специально созданной некоммерческой организацией Business Process Management Initiative (BPMI). Она создавалась для стандартизации бизнес-процессов.

После слияния в 2005 году с некоммерческим объединением Object Management Group (OMG), занимающимся разработкой и продвижением объектно-ориентированных технологий и стандартов, было выпущено две версии нотации:

- 1.2 (2009 г.) - нотация моделирования, пиктограммы и правила их использования
- и 2.0 (2011 г.) с уточнениями 2.0.1 и 2.0.2. (2014 г) - нотация моделирования с семантикой элементов, структура данных модели процесса, формат файла
- С 2013 г. является международным стандартом ISO 19510
- Вендоры: <http://bpm-directory.omg.org/vendor/list.htm>





BPMN как стандарт

- Международный Стандарт (ISO / IEC 19510) представляет собой объединение лучших практик в сообществе бизнес-моделирования как спецификацию нотации и семантику диаграмм «сотрудничества», «процесса» и «хореографии».
- Цель -стандартизировать модель бизнес-процесса и использующихся объектов на фоне множества различных других нотаций и методов. При этом BPMN должна служить простым средством передачи информации о процессе другим бизнес-пользователям, разработчикам процесса, клиентам и поставщикам
- Первая Спецификация OMG, которая стала стандартом ISO, была UML® в 2005
- Ссылка на документы:
- <https://www.omg.org/spec/BPMN/2.0.1/>

Уровни использования BPMN

•УРОВЕНЬ 1

Описательное моделирование (descriptive modeling) — простое моделирование, абсолютно понятное всем бизнес-пользователям. Используется минимум базовых графических элементов (объектов)

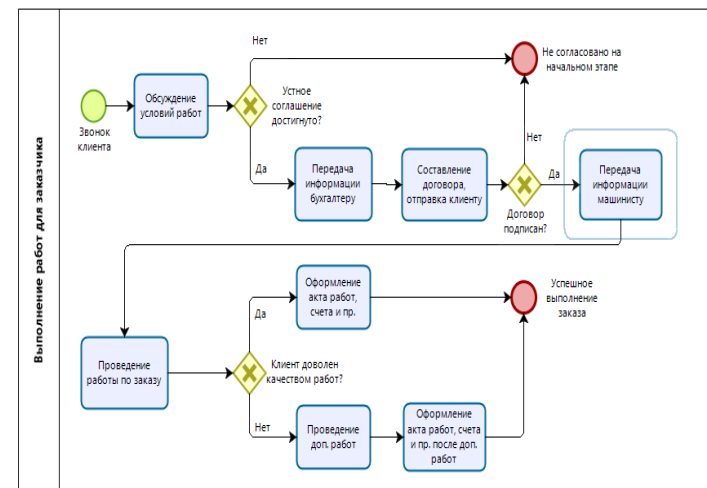
•УРОВЕНЬ 2

Аналитическое моделирование (analytical modeling) — более полное описание, включающее анализ, метрики. Требуется более глубокое знание BPMN, описывает процесс с максимально необходимой детализацией

Используется для описания моделей As Is и To Be для дальнейшего анализа, не требующее исполнения в BPMS

•УРОВЕНЬ 3

Исполняемое моделирование (executable modeling) — моделирование процесса, пригодного для исполнения в BPMS. Добавляются специфические атрибуты, параметры и т. п. Ориентировано на разработчиков — полная цифровизация процесса с автоматизацией и поддержкой в актуальном состоянии, мониторинг





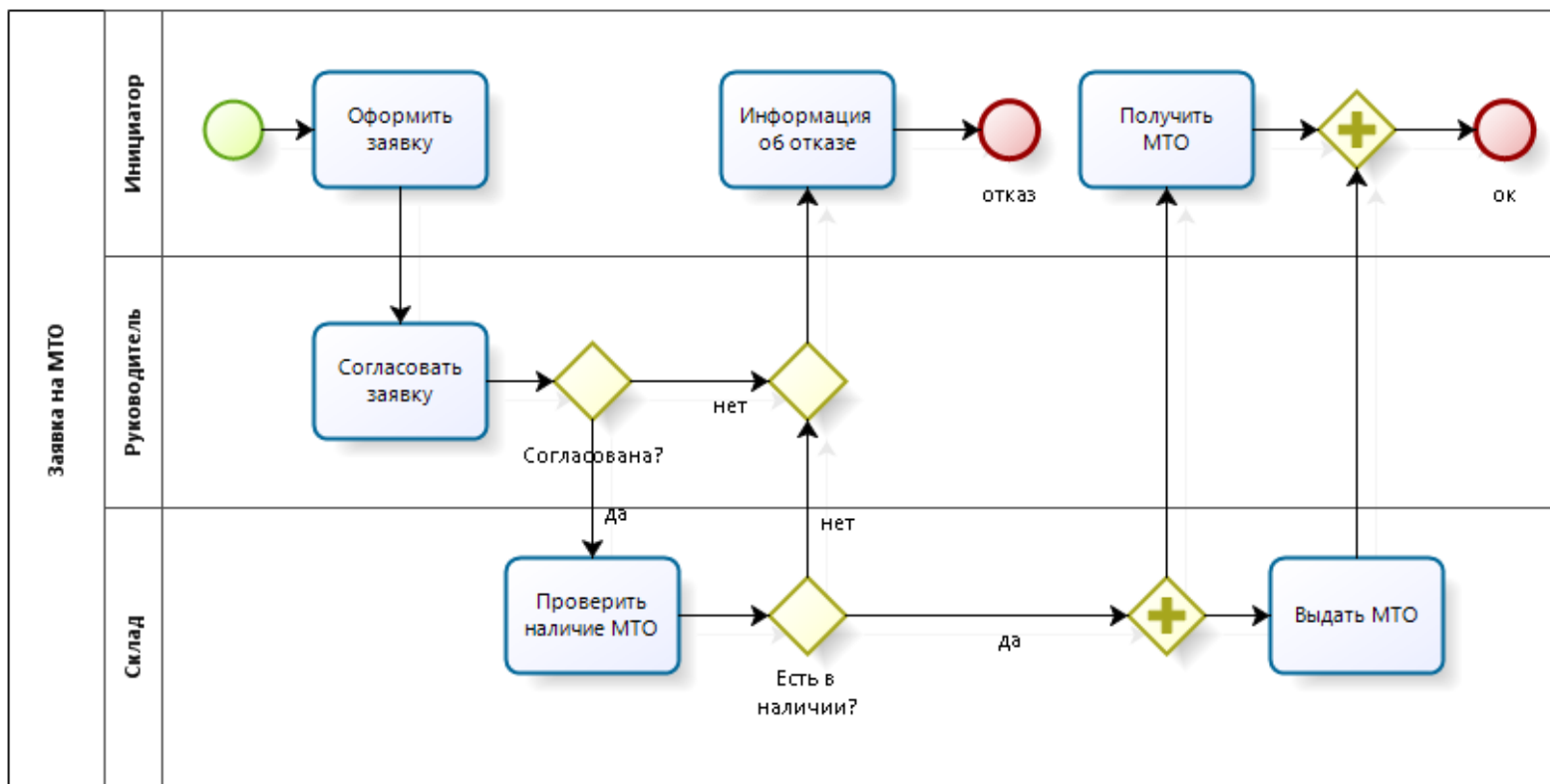
Высшая
школа
бизнеса

Основные понятия модели процесса

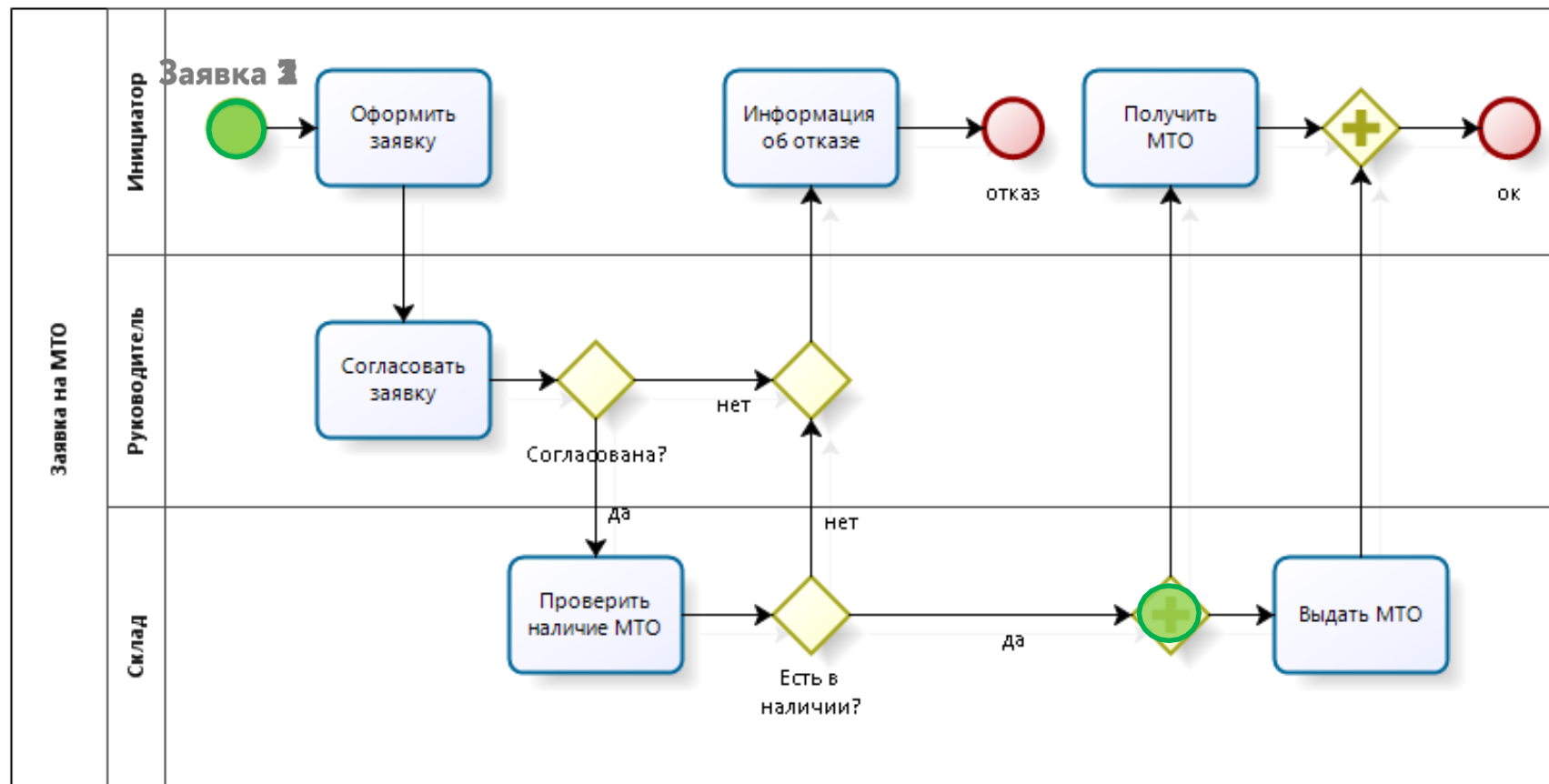


- Схема (шаблон) процесса
- Process Instance (экземпляр процесса)
- «Токен»
- Process Model (модель процесса)
- Process Model Attribute (атрибут модели процесса)

- Схематичное описание, которое предусматривает все возможные сценарии процесса («рецепт на все случаи жизни»)
- Прямоугольная область, содержащая схему процесса, называется «пул» (pool)



- По каждому шаблону может быть запущено неограниченное количество экземпляров процесса
- Каждому экземпляру процесса соответствует свой маршрут

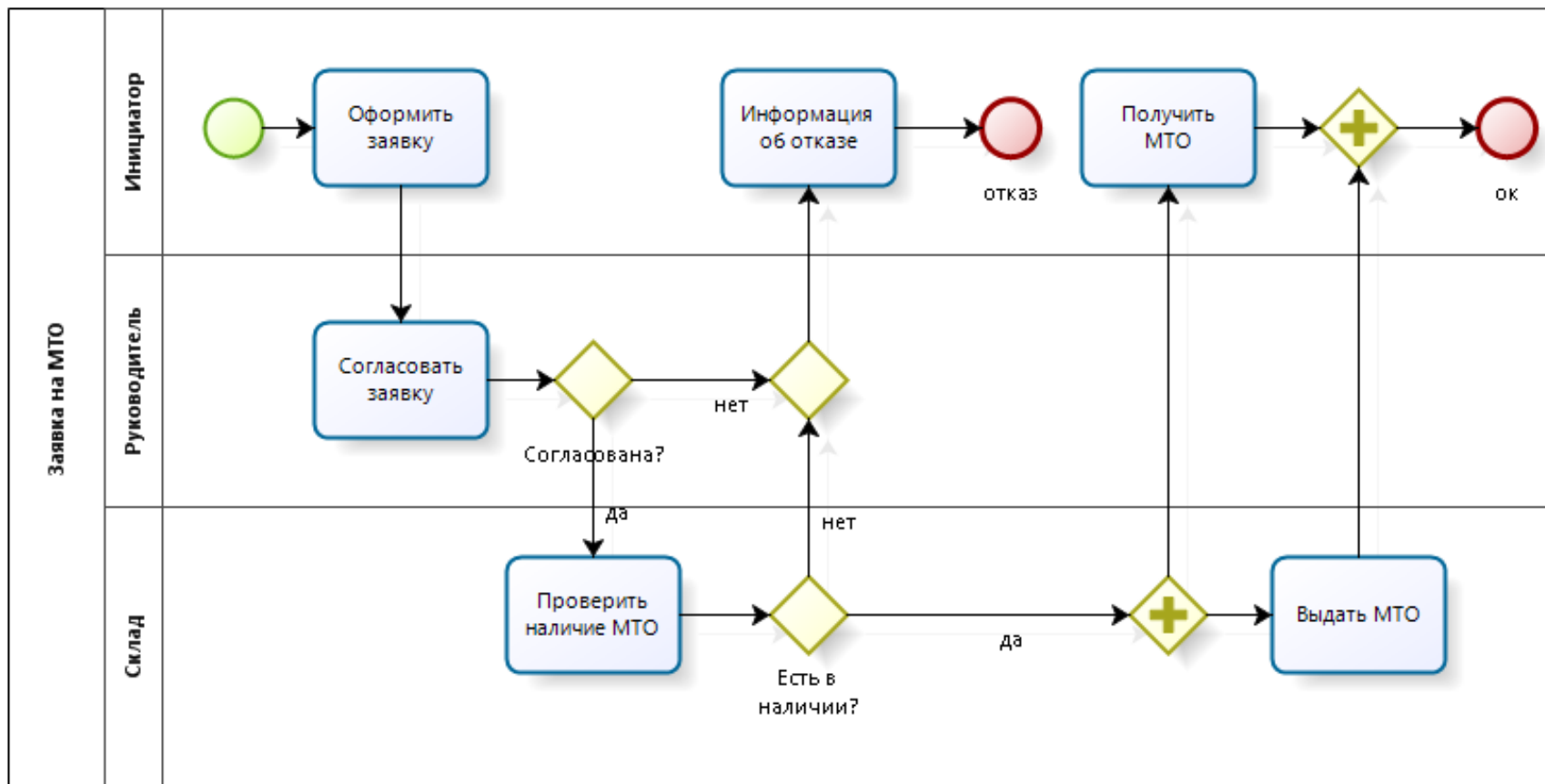


«Токен»

Токен – это бегущий по потоку маркер, показывающий куда движется поток.

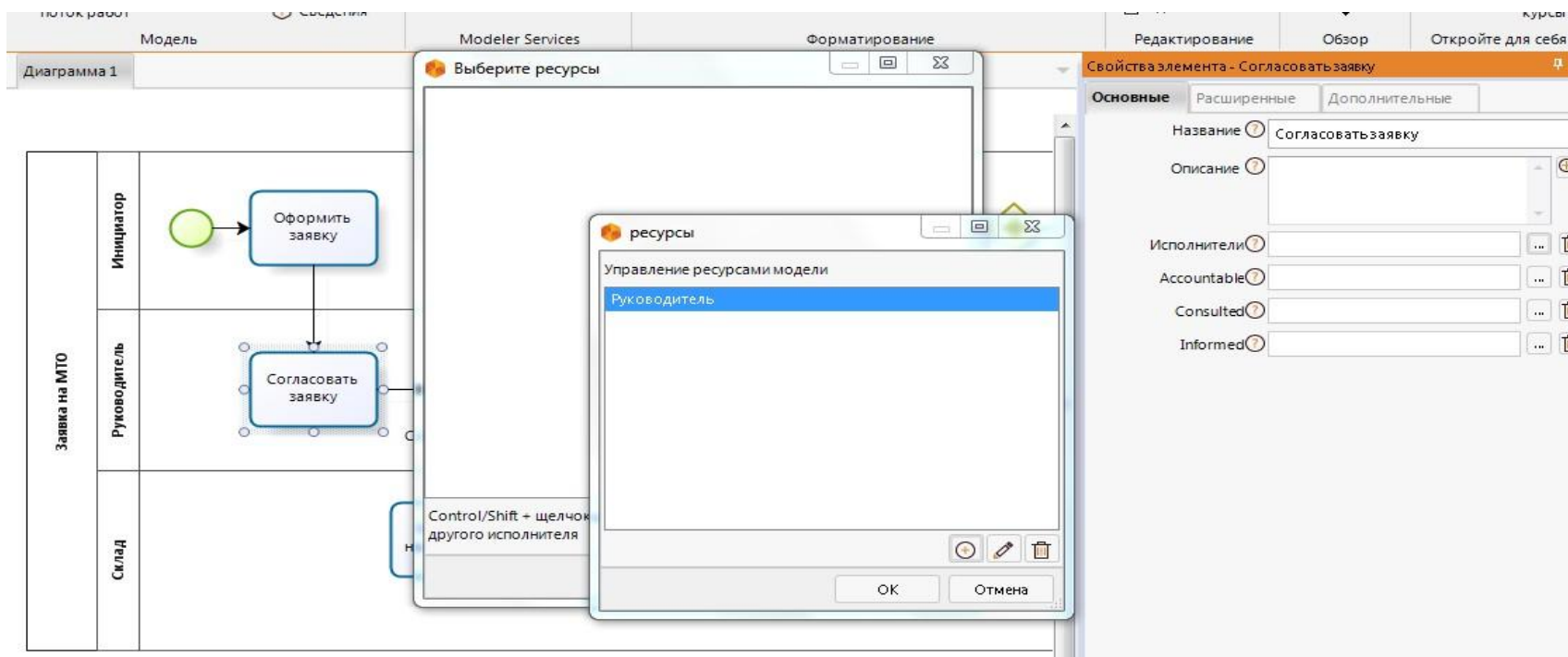
Токен очень хорошо знает BPMN. Поэтому, если затрудняетесь в правильности схемы – спросите у токена, куда он пойдет

Токен **обязательно должен добежать до конца процесса**



Модель, атрибуты модели процесса

- Process Model (модель процесса) – это диаграмма процесса плюс структура процесса – действия, исполнители, порядок взаимодействия
- Process Model Attribute (атрибут модели процесса) – дополнительные свойства модели
 - например, имя задачи, тип задачи, тип подпроцесса и другие свойства, не всегда отображаемые непосредственно на диаграмме





- Process Attributes (атрибут процесса) - бизнес-данные, контекстное содержание процесса. Часто называют пользовательскими данными
 - например, процесс согласования договора содержит данные договора (дату, номер, контрагента и т.д.), а также информацию о результатах согласования (согласующие лица, резолюции, комментарии и т.д.)
- Атрибуты процесса не отображаются непосредственно на схеме процесса (за исключением отдельных случаев, когда это необходимо для понимания процесса)
- Но (!) без понимания контекста процесса нельзя корректно описать процесс



Высшая
школа
бизнеса

Типы процессных диаграмм



Межпроцессное взаимодействие

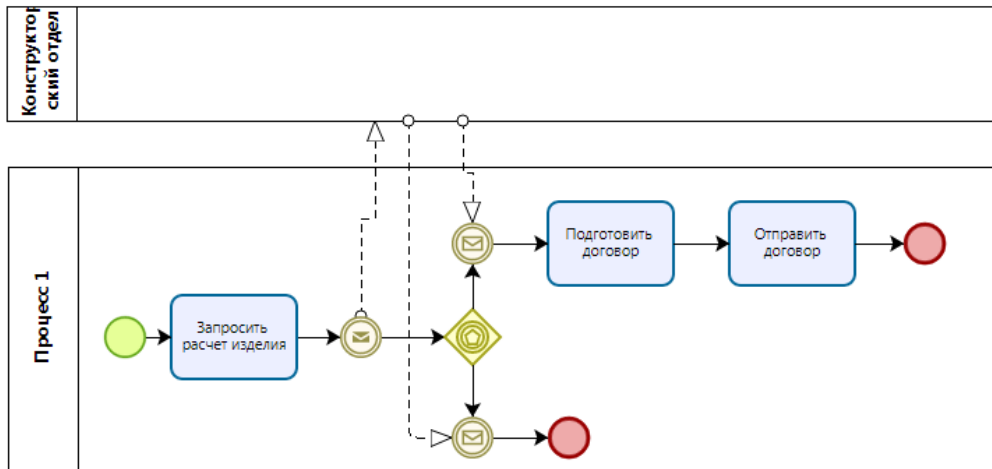
- От выполнения действий или событий, происходящих в процессе, могут зависеть другие процессы
 - Например, процесс выполнения заказа ожидает от бухгалтерии подтверждения оплаты
- «Общение» процессов между собой называется «межпроцессным взаимодействием»
- Механизмы межпроцессного взаимодействия:
 - Обмен сообщениями (точка-точка)
 - Сигналы (широковещательная рассылка)
 - Через данные (как внутренние, так и внешние)



BPMN диаграммы

- **Orchestration = оркестровка**
 - Все, что происходит внутри одного потока работ
- **Collaboration = межпроцессное взаимодействие**
 - Обмен сообщениями между несколькими потоками работ
 - в BPMN 1.x назывался хореографией
- **Choreography = хореография**
 - Последовательность актов межпроцессного взаимодействия
 - в BPMN 2.0
- **Conversation = переговоры**
 - Логическая группировка сообщений
 - в BPMN 2.0

Orchestration = оркестровка



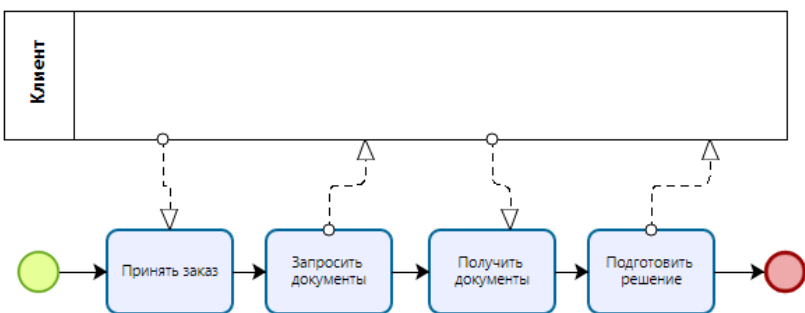
Публичный
процесс

Приватный
процесс
исполняемый

- Отдельный процесс, все, что происходит внутри одного пула либо без пула

- **приватный процесс** : один развернутый пул или без пула

- **публичный процесс**: внешний по отношению к организации процесс или участник, процессы которого находятся вне нашей зоны ответственности

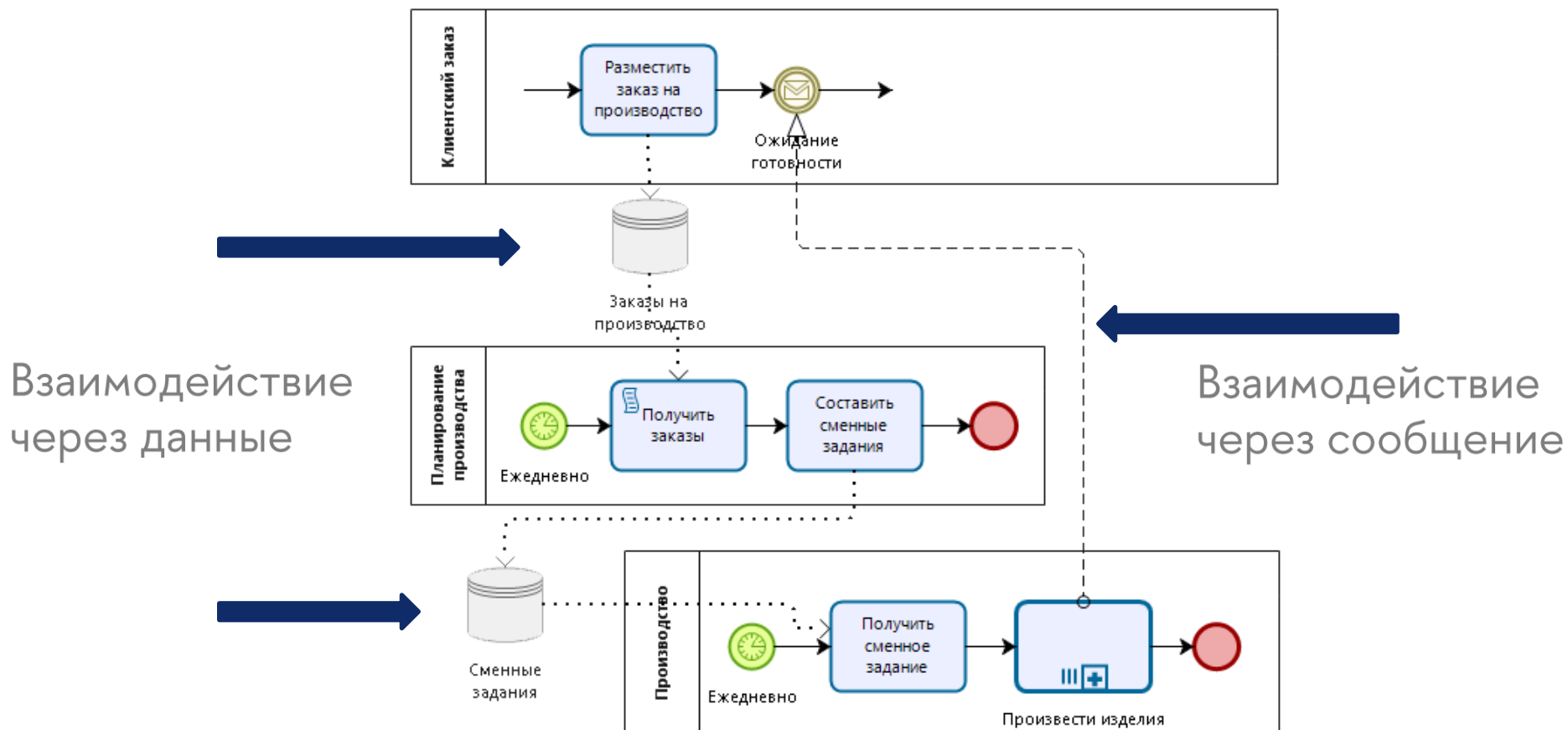


Публичный
процесс

Приватный
процесс не-
исполняемый

Такую диаграмму также называют аналитической

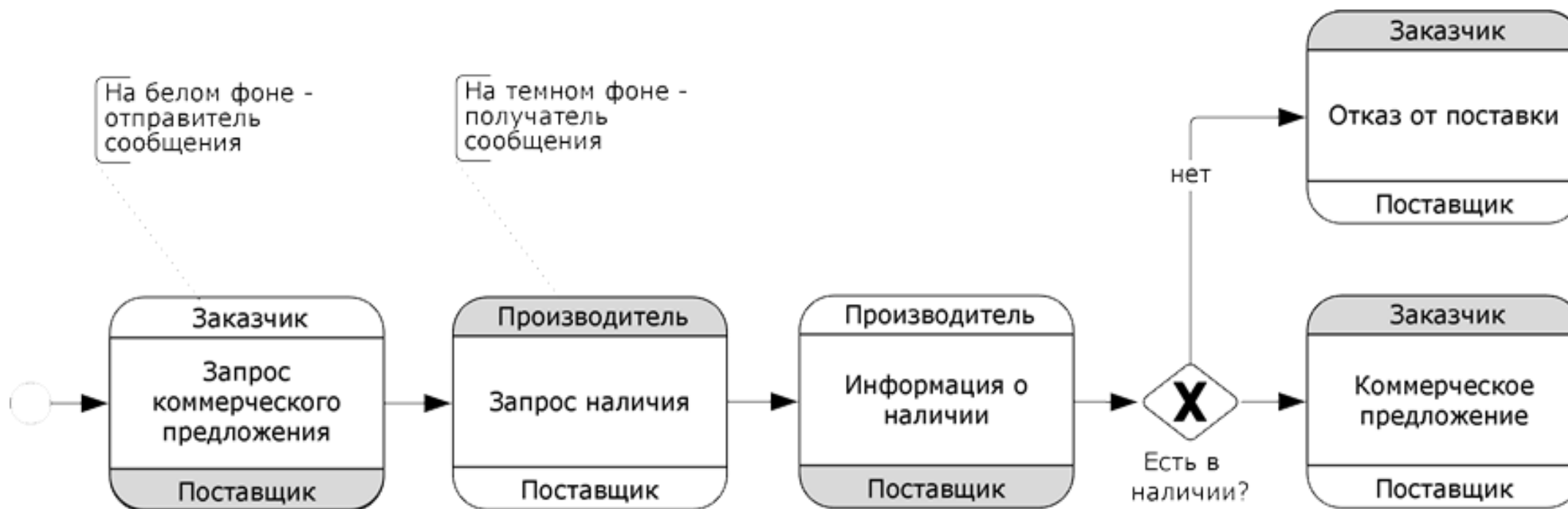
Collaboration = межпроцессное взаимодействие



Choreography = хореография

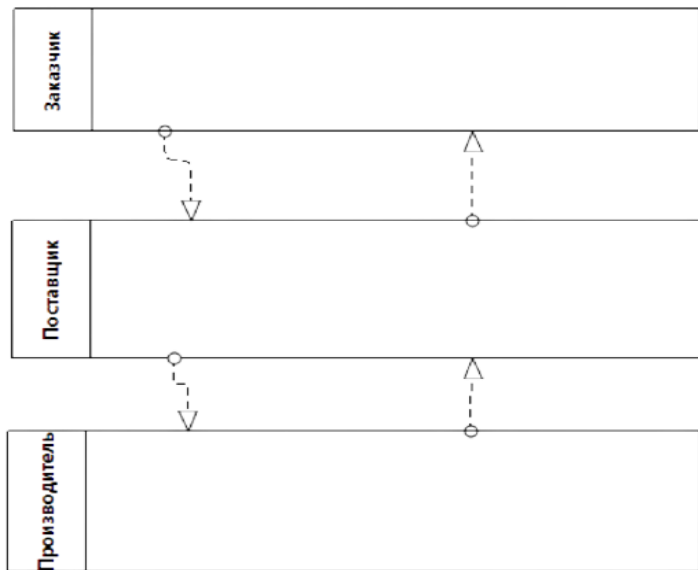
Данный тип диаграммы
присутствует
не во всем программном
обеспечении (ПО) BPMN

- Последовательность актов взаимодействия двух- или многостороннего

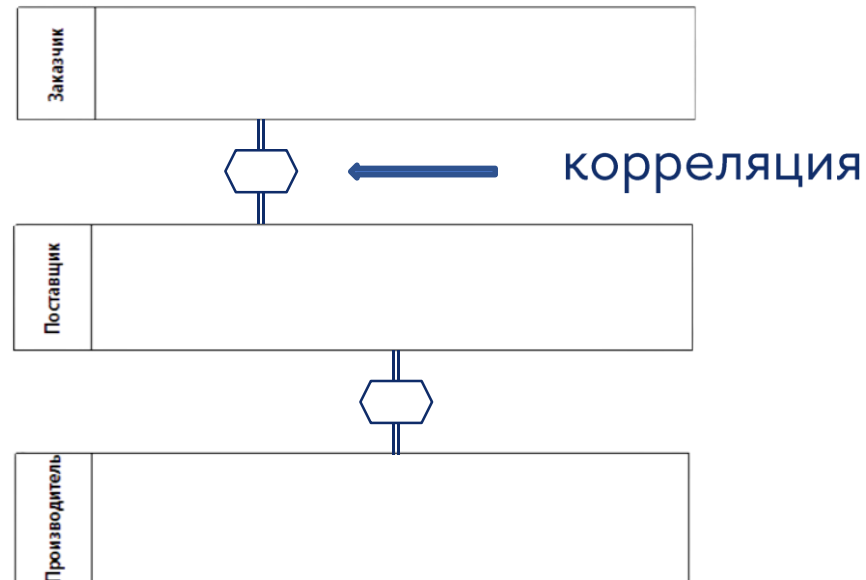


Conversation = переговоры

- Последовательность запросов-ответов
 - группировка обмена сообщениями
 - связаны друг с другом т.н. корреляцией



Развернутый поток сообщений



Группировка сообщений

Данный тип диаграммы присутствует
не во всем ПО нотации BPMN



Уровни моделирования

- Карта процессов верхнего уровня (архитектура процессов)
 - Показывает взаимосвязь процессов в границах проектируемого конечного процесса
 - Может быть описана при помощи IDEF0
 - Но (!) можно моделировать и в BPMN
- Регламентация процессов
 - Описывает действия исполнителей, порядок взаимодействия, нормативные сроки и т.д.
 - Может применяться как должностные инструкции для отдельных исполнителей
- Описание процессов для их последующей автоматизации
 - Содержит действия и события, выполняемые без участия человека
 - Исключает неверную трактовку логики процесса ИТ-специалистами
- Процессы, исполняемые в BPMS



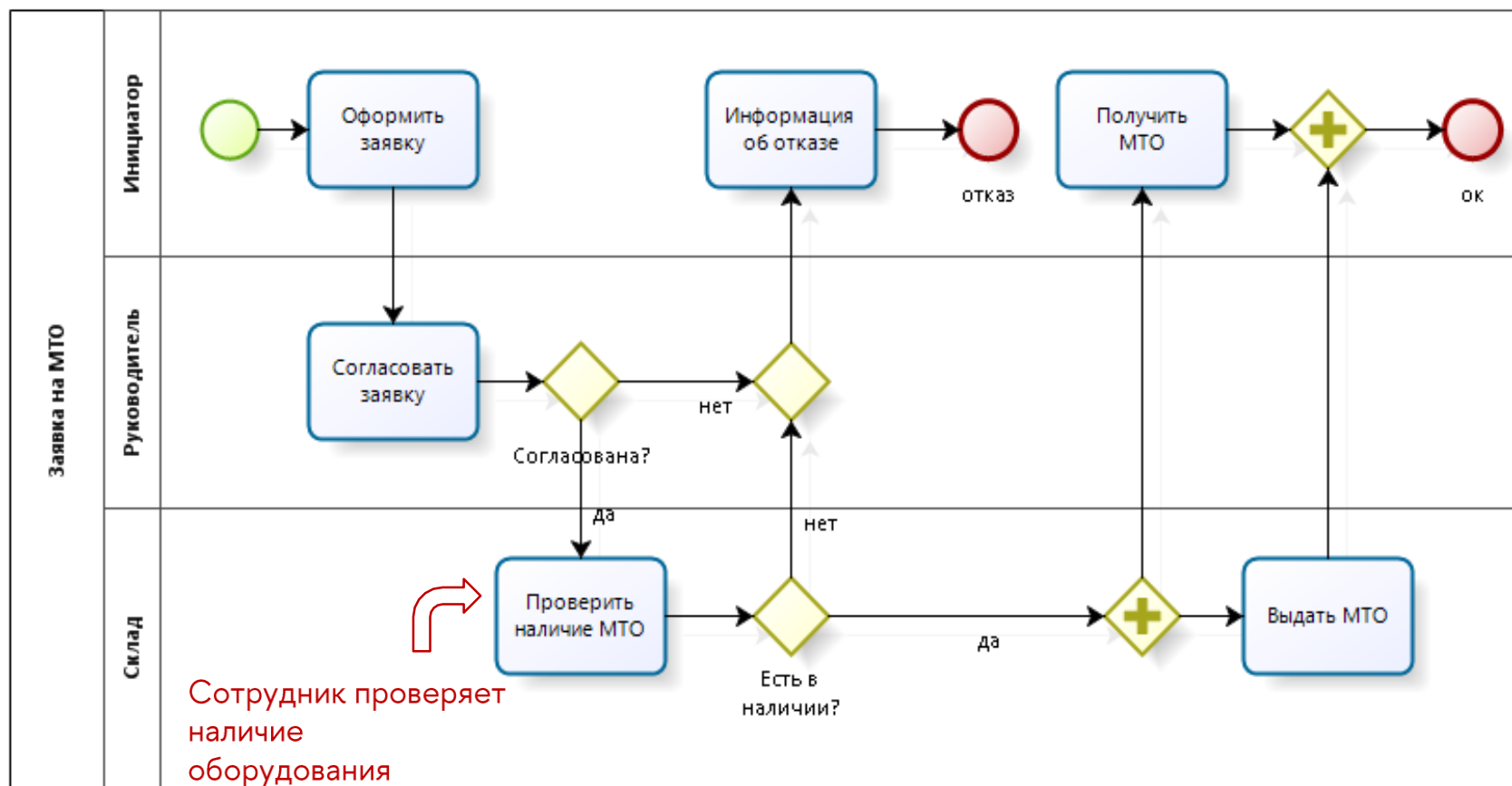
Для регламентации

- Описание отдельных процессов
 - Оркестровка
 - Workflow, без элементов межпроцессного взаимодействия
- Разработка регламентов и должностных инструкций
 - Используются только элементы нотации, моделирующие деятельность человека
 - Схема процесса может быть опубликована в виде текстового регламента
- Общий язык с «бизнесом»
 - Интуитивно понятные элементы нотации

Для автоматизации

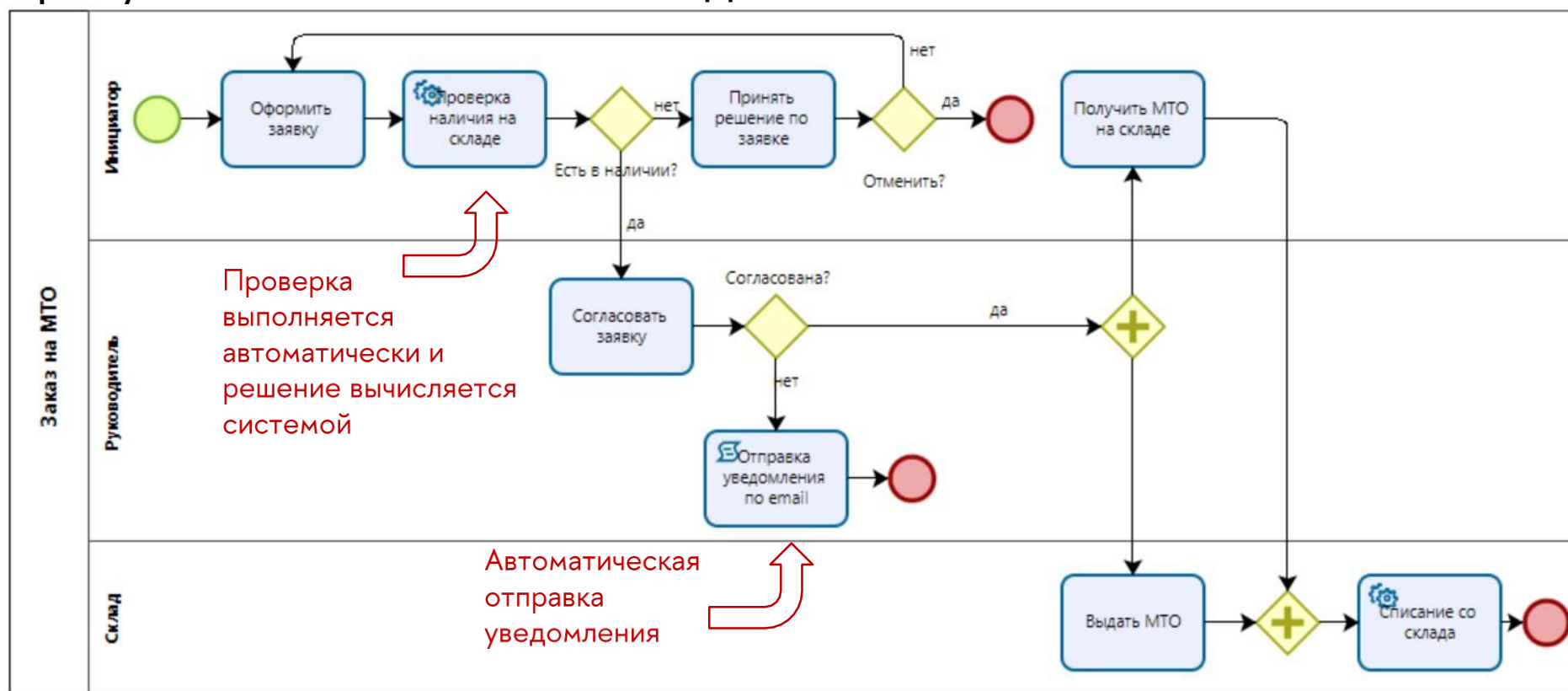
- Использование расширенной палитры
 - Включаются элементы, описывающие автоматические действия
 - Элементы реакции на события
- Межпроцессное взаимодействие
- Применяется для моделирования процессов в целях автоматизации либо написание ТЗ на автоматизацию
 - Поддерживаются специальными BPMN-движками

- Задачи, выполняемые человеком
- Развилки, действие которых основаны на решениях или ответах человека



Сотрудник проверяет
наличие
оборудования
самостоятельно
(ручная обработка)

- Задачи, выполняемые человеком или системой
- Развилки, действие которых основаны на решениях или ответах человека либо результат автоматического действия





На какие вопросы должна ответить
схема процесса?

- Что является целью процесса
 - объект (заявка, заказ, документ, требующий согласования и т.д.)
 - начальное и целевое состояние (выполненный заказ, согласованный документ и т.д.)
- Кто участвует в процессе
 - сотрудники организации
 - внешние участники (люди, организации)
- Какие задачи должны быть выполнены
 - последовательность исполнения
 - сроки
 - результат
- Какие риски следует учесть
 - исключения
 - отказы
 - альтернативные варианты исполнения
- Только ответив на эти вопросы можно приступить к моделированию

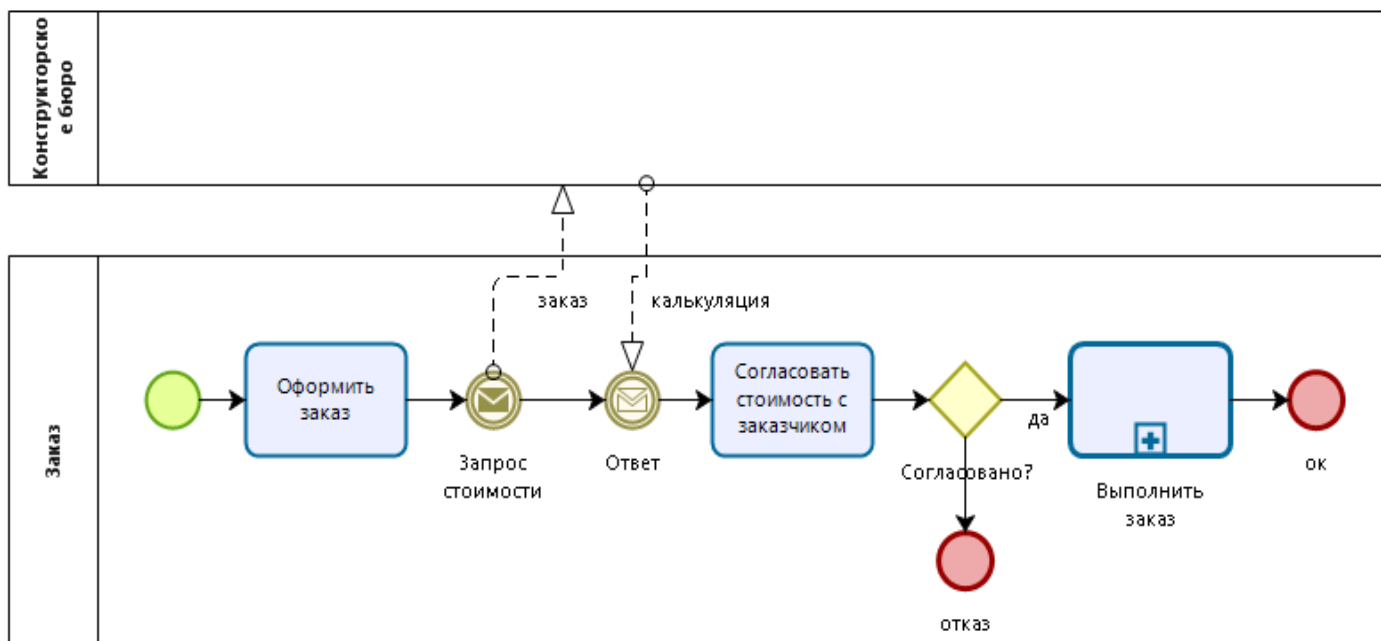


- Пулы и дорожки
- Данные
 - Объекты данных (Data Objects)
 - Хранилища данных (Data Stores)
- Артефакты
 - Группы
 - Аннотации
- Элементы потока
 - Действия (Activities)
 - Задачи (Tasks)
 - Подпроцессы (Sub-Processes)
 - Развилки (Gateways)
 - События (Events)
- Соединительные элементы
 - Потoki управления (Sequence Flows)
 - Потoki сообщений (Message Flows)
 - Ассоциации (Associations)

Pool = пул

- Один пул = один процесс
- Опционален (можно не использовать, если позволяет ПО / специфика процесса)
- Может содержать детализированный поток работ («white box», приватный процесс)
- Может быть пустым или содержащим только точки взаимодействия - внешняя сущность («black box» или публичный процесс)

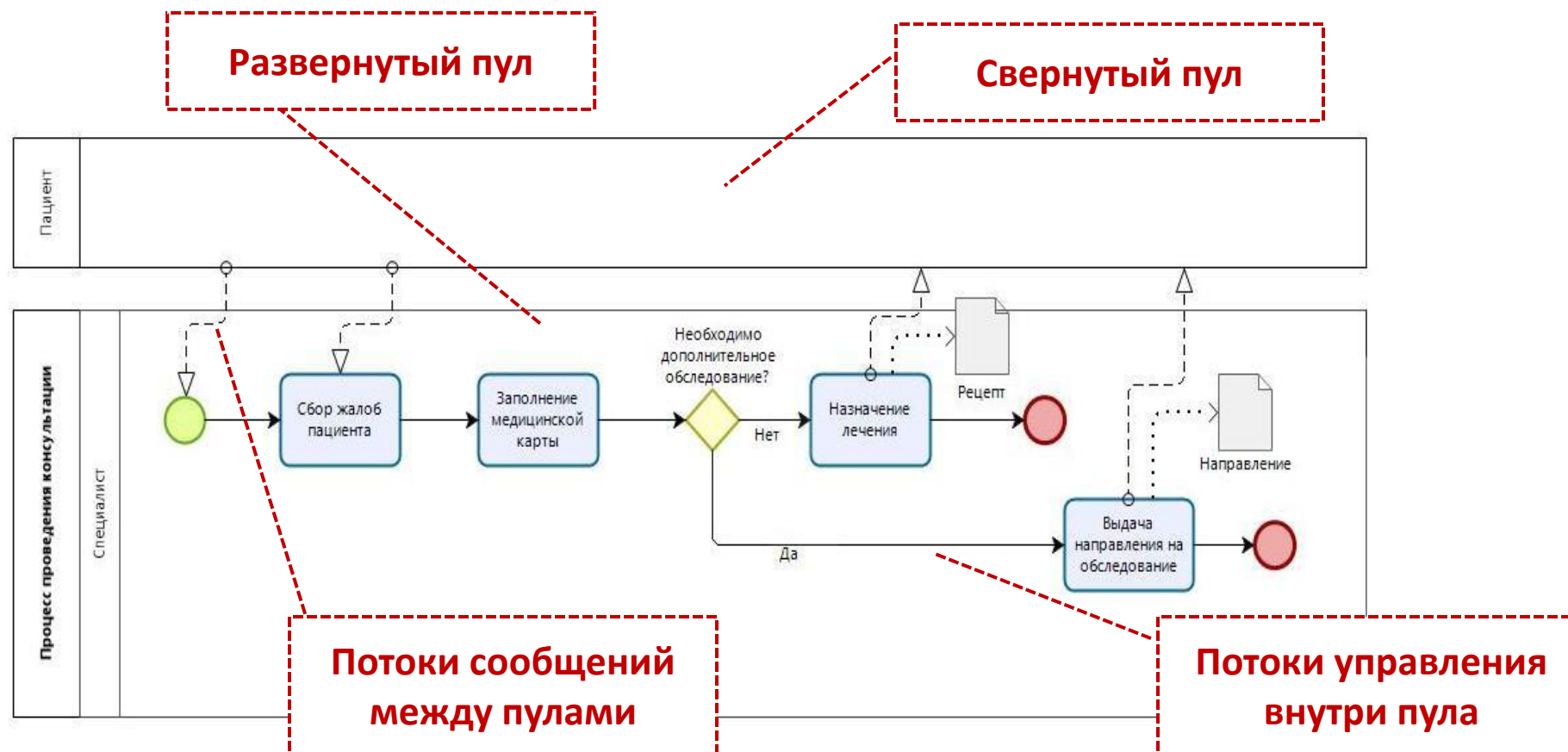
На одной диаграмме могут использоваться оба вида пулов для визуализации межпроцессного взаимодействия



← black box

← white box

ВИДЫ ПУЛОВ НА ДИАГРАММЕ ВЗАИМОДЕЙСТВИЯ (COLLABORATION)



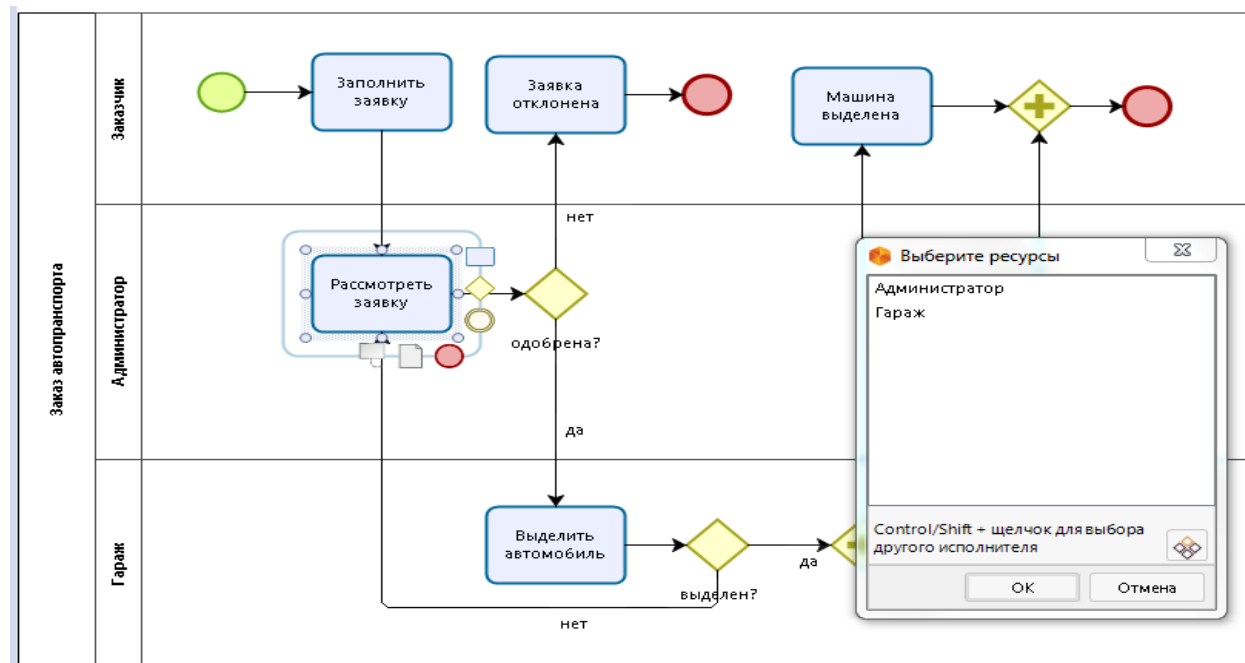
Пулы могут быть развёрнутыми («белый ящик») или свёрнутыми («черный ящик»).

Свёрнутые пулы изображают внешние сущности (процессы), действия внутри которых нам неизвестны, с ними можно только обмениваться сообщениями

Lane = дорожка

- Опциональна (можно не использовать, если нет цели показать участников или из-за специфики проекта/описания/процесса/ПО)
- **Отображает на диаграмме участников процесса**
 - Подразделение, роль, группа, пользователь
- Не регламентируется нотацией, изображается только для визуализации
- **Исполнители задач задаются на уровне задачи**
 - (участник, обозначенный на дорожке = исполнителю задачи)

Дорожки
Исполнители



Пулы и дорожки

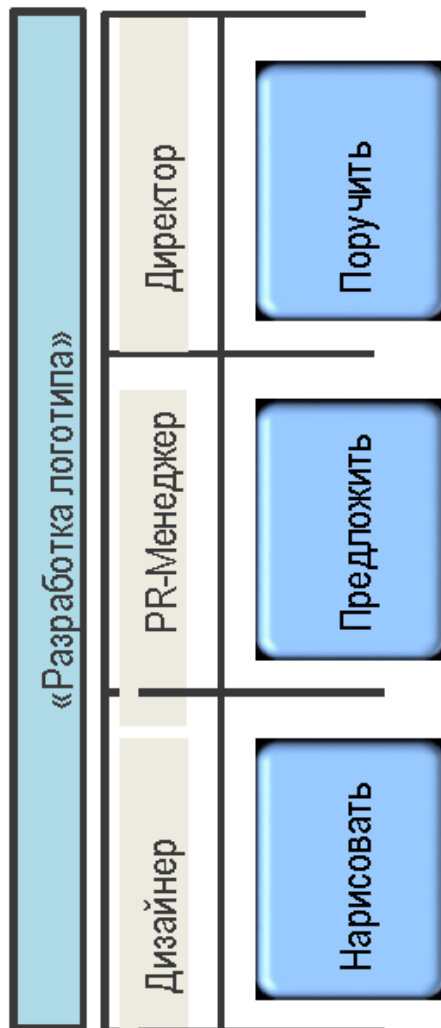
Каждый пул задаёт границы и участников данного бизнес-процесса. На одной диаграмме может быть несколько пулов (тогда это диаграмма взаимодействия). Встречается как горизонтальное, так и вертикальное расположение дорожек. Однако, **в настоящее время практически повсеместно принято горизонтальное.**



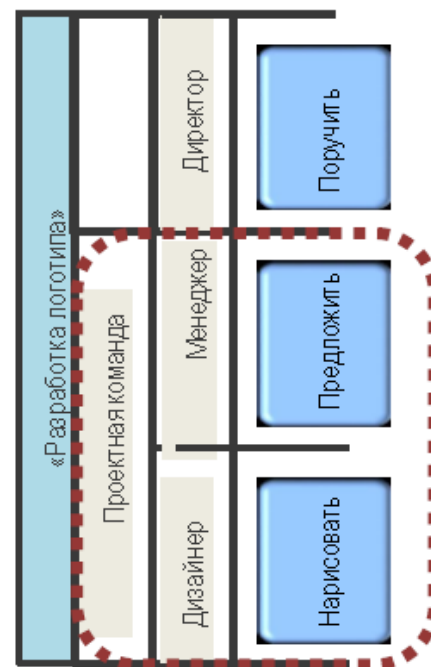
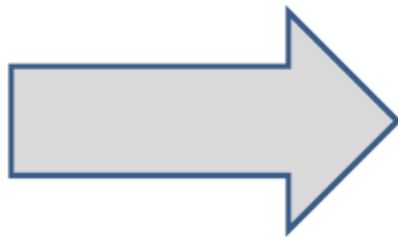
Каждая дорожка выполняет задачи или подпроцессы, указанные на данной дорожке. Бизнес-роль может выполнять подразделение организации, конкретный человек, должность, информационная система

В любом случае диаграмма процесса должна быть размещена на одном листе и быть читаемой, различимой, включая названия элементов

Пулы и дорожки



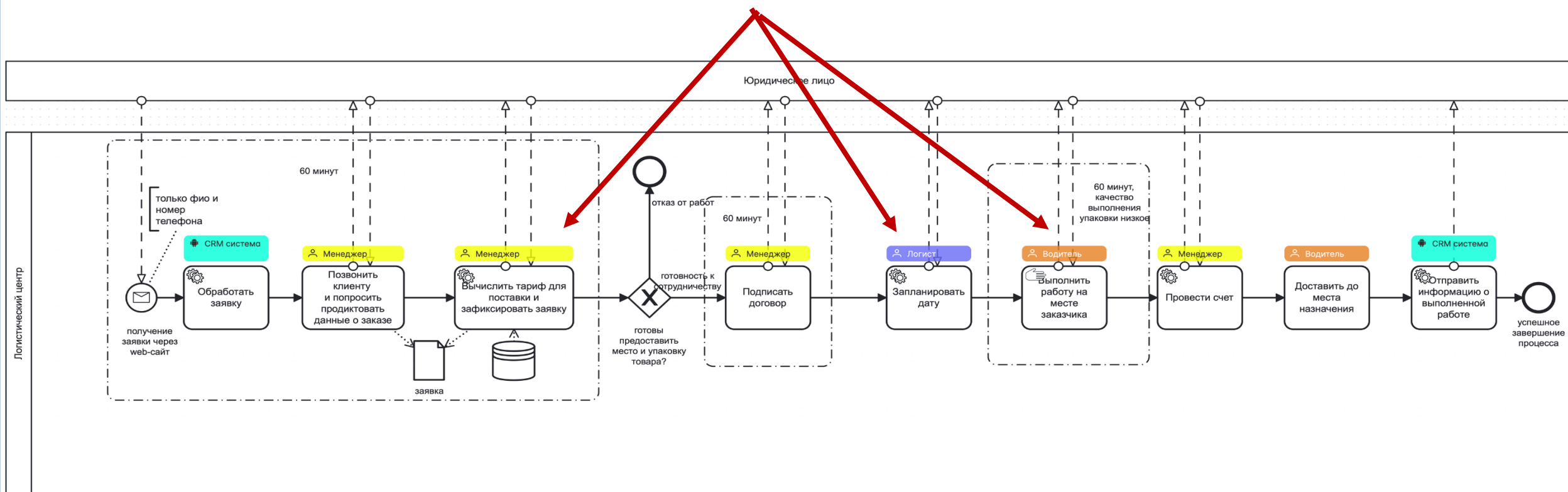
Имя пула - название бизнес-процесса, напр. «Разработка логотипа»



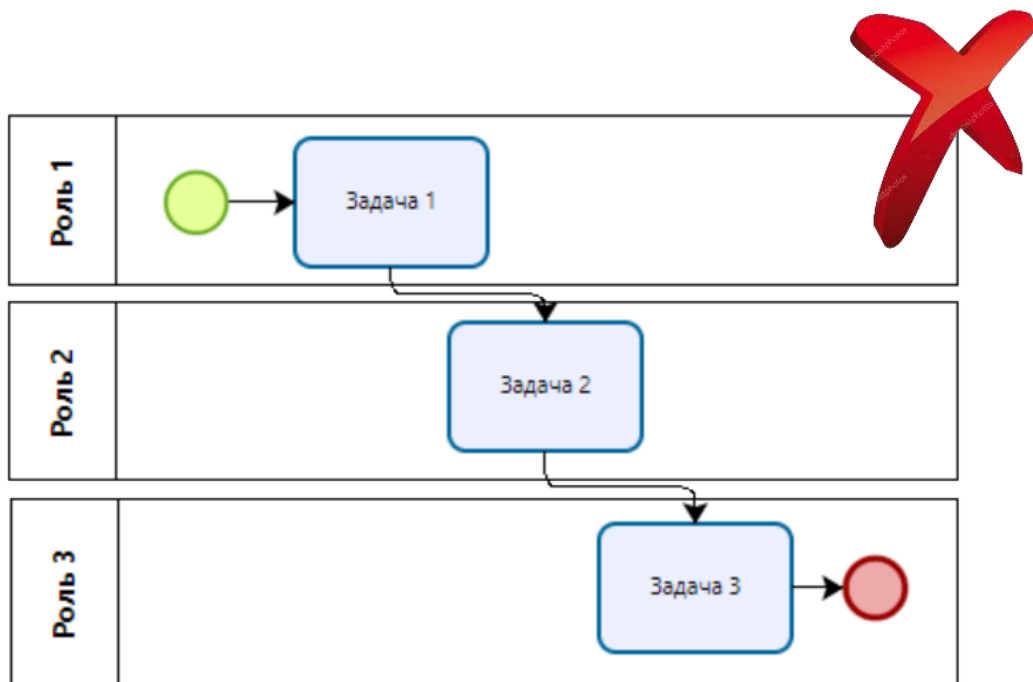
Дорожки в пуле можно декомпозировать, т.е. разделять одну бизнес-роль на несколько составных частей

Пример BPMN без дорожек – ПО “Storm”

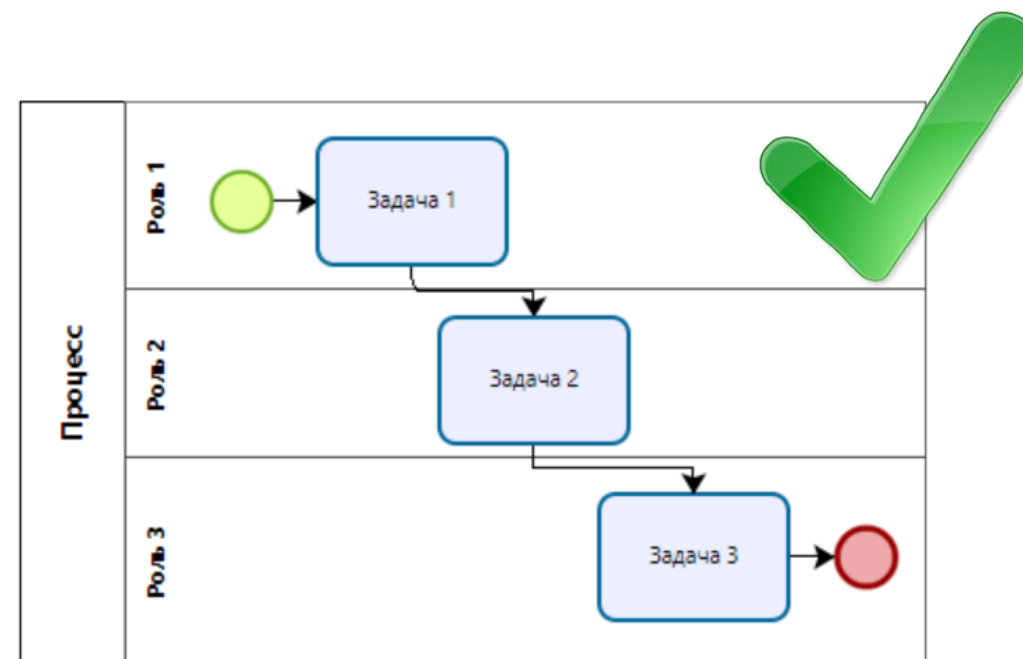
Принцип построения диаграммы получается похожим на нотацию EPC:
участники процесса/роли присоединяются к соответствующим задачам



Ошибки моделирования



- Пулы вместо дорожек



- Пул и дорожки внутри пула

- Используется для отображения на процессной диаграмме хранилища данных
 - СУБД
 - используется как элемент межпроцессного взаимодействия, когда надо отобразить на диаграмме передачу данных между процессами
 - НЕ используется для отображения того факта, что все данные сохраняются в БД
 - внешняя система
 - отображается передача данных во внешнюю систему



Объекты данных

Объект данных



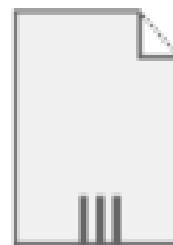
- Используется для отображения на процессной диаграмме объекта данных
 - документ

Объект данных (вход)



- Используется для отображения на процессной диаграмме входящих данных

Объект данных (коллекция)



- Используется для отображения на процессной диаграмме хранилища данных
 - список

Объект данных (выход)



- Используется для отображения на процессной диаграмме исходящих данных

Потоки данных

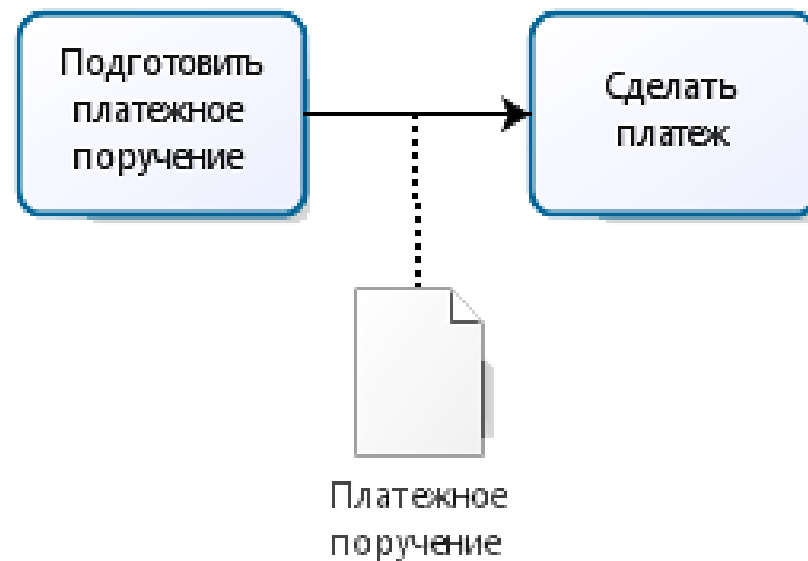
Входы-выходы активности

- Задача обрабатывает запрос
 - состояние «получен» меняется на состояние «проверен»



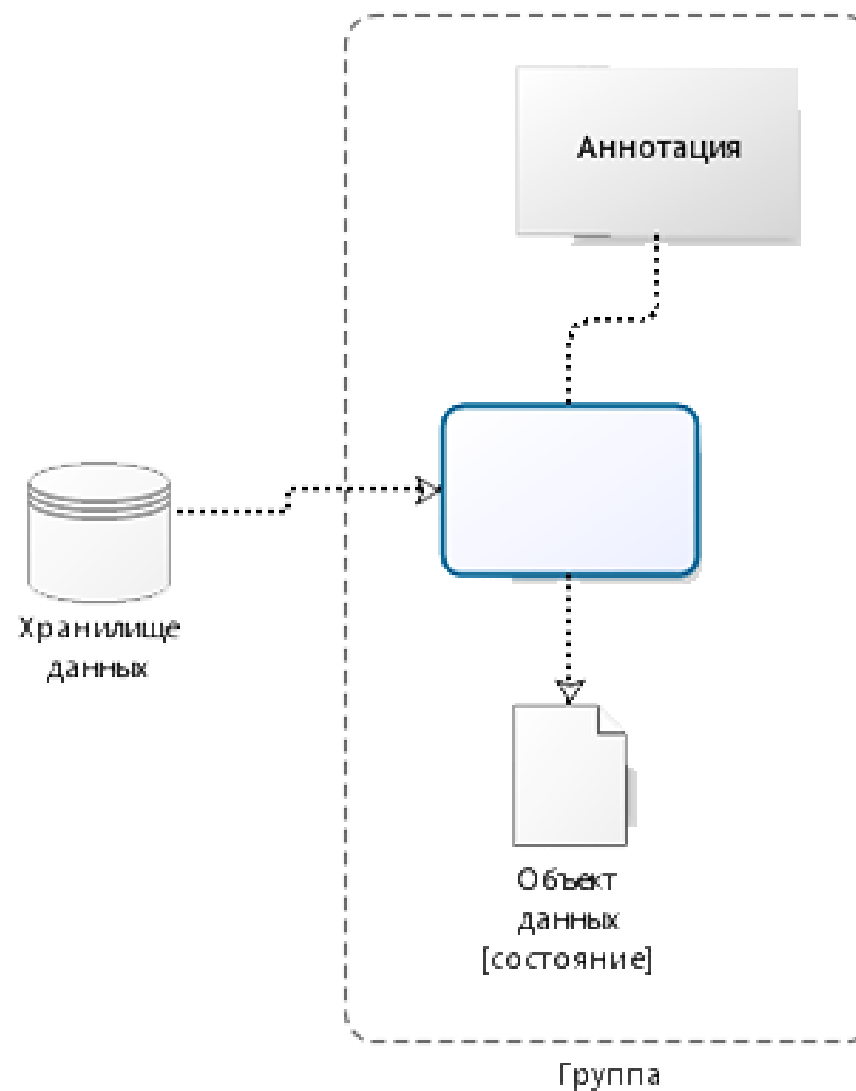
Атрибут потока управления

- Подготовленное Платежное поручение передается на следующую активность



Artifact = артефакт

- Annotation = аннотация (комментарии)
- Group = группа (визуально объединяет объекты на схеме)





Поток управления (Control Flow)



- **Последовательность операций (Sequence Flow)**

- определяет порядок выполнения действий внутри процесса
 - поток управления не может выходить за границы пула
 - поток управления не может выходить за границы подпроцесса



- **Условный поток (Conditional Flow)**

- последовательность, для которой заданы условия
 - используется, когда условие задано на выходе из задачи

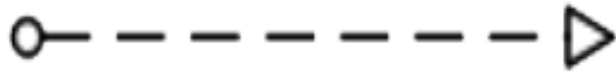


- **Поток по умолчанию (Default Flow)**

- поток, который будет выполнен в результате логического условия, когда все остальные результаты ложны



Поток сообщений (Message Flow)



- Передача сообщений между потоками работ (пулами)
 - потоки сообщений не могут соединять объекты внутри одного пула



Ассоциации (Associations)

.....>

- Связывает объекты данных и хранилища данных с объектами процесса

.....

- Связывает артефакты с объектами процесса
 - аннотации



Высшая
школа
бизнеса

Задачи



Задача (Task)

Атомарное действие процесса, не
разбиваемое на более мелкие операции



Пользовательская задача

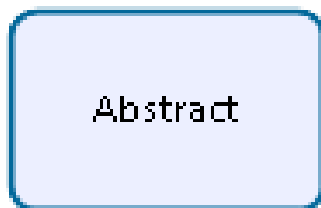
- выполняется человеком в процессной системе или вне ее

Автоматическая задача

- выполняется системой (процессной или внешней)

Пользовательские задачи

С точки зрения схемы процесса – «шаг процесса». С точки зрения пользователя или исполняемого процесса – «задача»



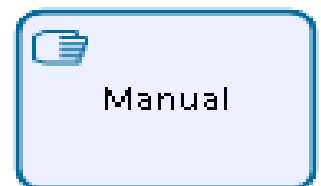
- **Abstract Task**

- Неопределенная
 - тип задачи не определен



- **User Task**

- Выполняется пользователем с использованием компьютера
 - н/п, ввести данные о заемщике



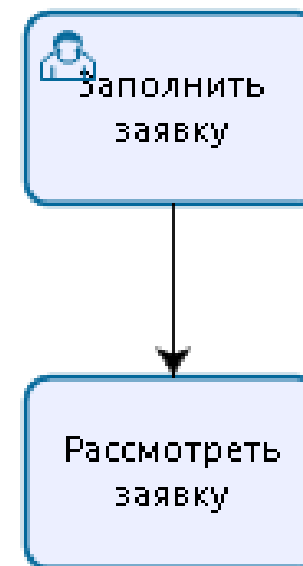
- **Manual Task**

- Работа вне системы. Выполнение задачи не контролируется автоматизированной системой
 - выкопать яму
 - отправить документы по почте



Как правильно ставить задачи

- Задача, предназначенная человеку - это задание, которое он должен выполнить
 - поэтому рекомендуется давать названия задачам в форме глагол + существительное - «Заполнить заявку»
 - Хотя иногда встречается и отглагольное существительное - «Заполнение заявки»



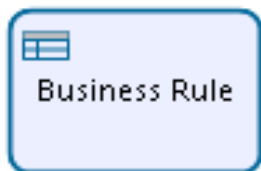


Автоматические задачи



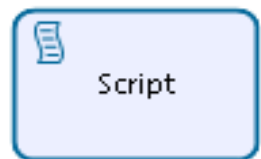
- Service Task

- Внешняя программа / сервис
 - вызов веб-сервиса
 - н/п, вычислить кредитный рейтинг, запросить остатки материалов на складе



- Business Rule Task

- Вычисление бизнес-правила
 - логическое выражение
 - н/п, принять решение о выдаче кредита на основании кредитного рейтинга



- Script Task

- Автоматическая операция системы
 - рассчитать сумму заказа
 - автоматическая отправка Email

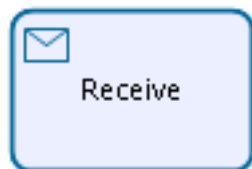


Задачи отправки/получения сообщений



- Send Task

- Отправка сообщения в другой процесс
 - автоматическая задача
 - аналог Send Message Event



- Receive Task

- Ожидание прихода сообщения из другого процесса
 - автоматическая задача
 - аналог Receive Message Event

Подпроцессы

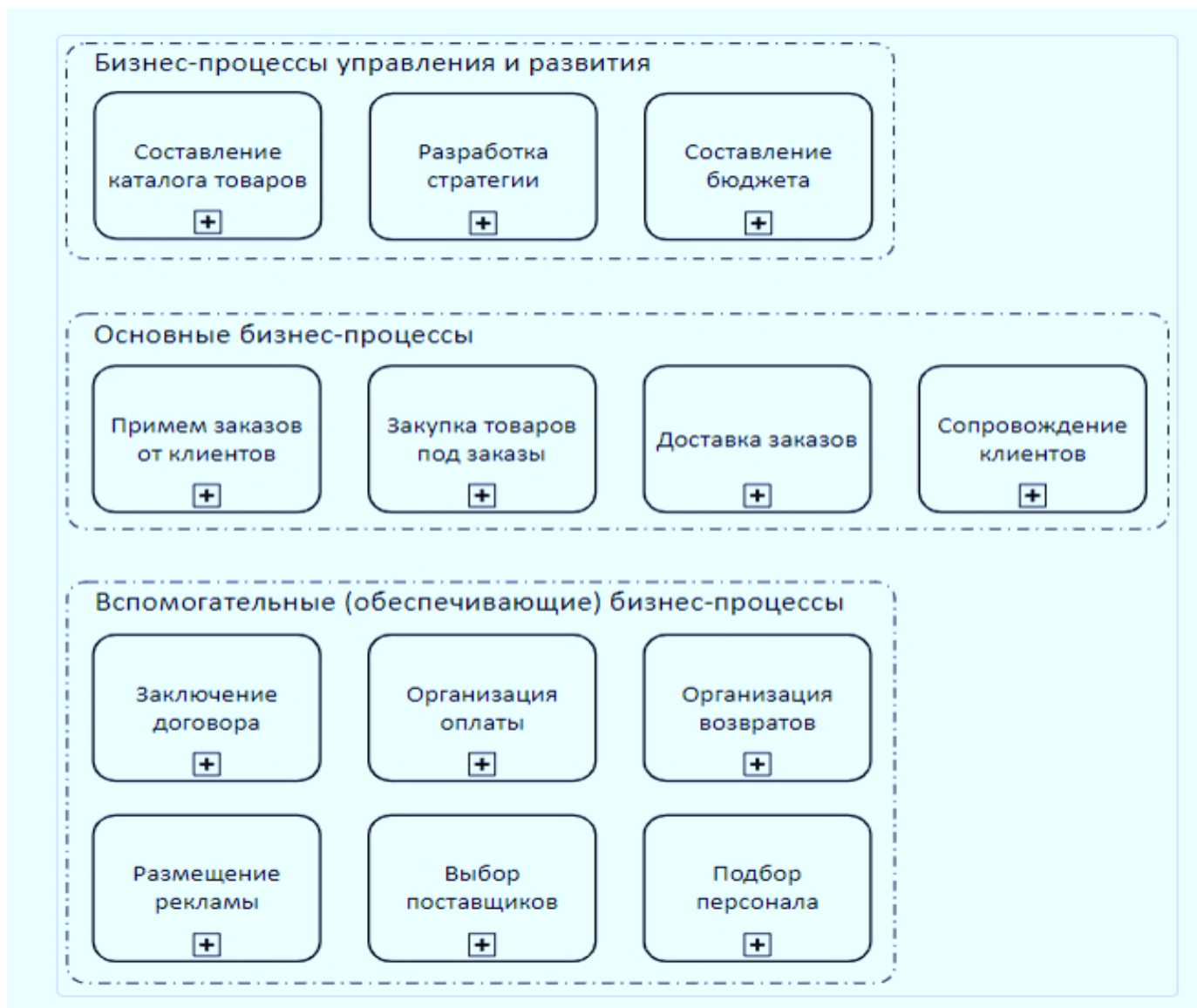
Действия-задачи на BPMN-диаграммах могут быть декомпозированы (детализированы). Эту специфику можно использовать для отображения обобщенных процессов более верхних уровней. Подпроцесс (subprocess) – сложные действия, состоящие из нескольких задач и логики их исполнения

Подпроцессы упрощают восприятие диаграммы. Подпроцесс может отображаться на диаграмме в развёрнутом или свёрнутом виде, в этом случае в нижней части подпроцесса отображается знак «+»





Использование «задач-подпроцессов» для моделирования процессов верхнего уровня





Высшая
школа
бизнеса

Развилки

Запомнить



Развилка НЕ принимает никаких решений!

Решение принимает пользователь, либо оно вычисляется автоматически до того, как токен дойдет до развилки

Развилка только определяет маршрут на основании данных процесса

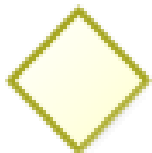


Все типы шлюзов-развилок

	Exclusive = «или/или» «Исключающее ИЛИ»
	Parallel = «параллельная»
	Inclusive = «и/или»
	Complex = комплексная
	Event-Based = по событиям
	Exclusive Event-Based = исключающая по событиям
	Parallel Event-Based = параллельная по событиям

Развилка (Gateway)

- Иногда используют название «Шлюз»
- Применяются **для контроля** потока (маршрутизации) процесса
- Изображаются ромбом с маркером или без маркера.
- Свойства:
 - Тип развилки (определяется маркером)
 - Схождение/расхождение (правила схождения/расхождения)



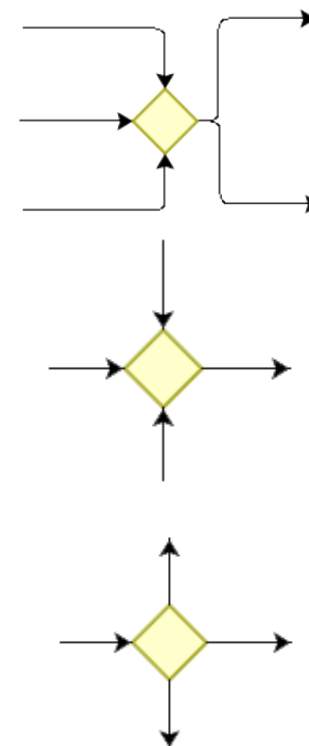
Развилка без маркера (пустой ромб) – всегда «ИЛИ / ИЛИ»



Развилка с маркером – определяет, уточняет тип развилки

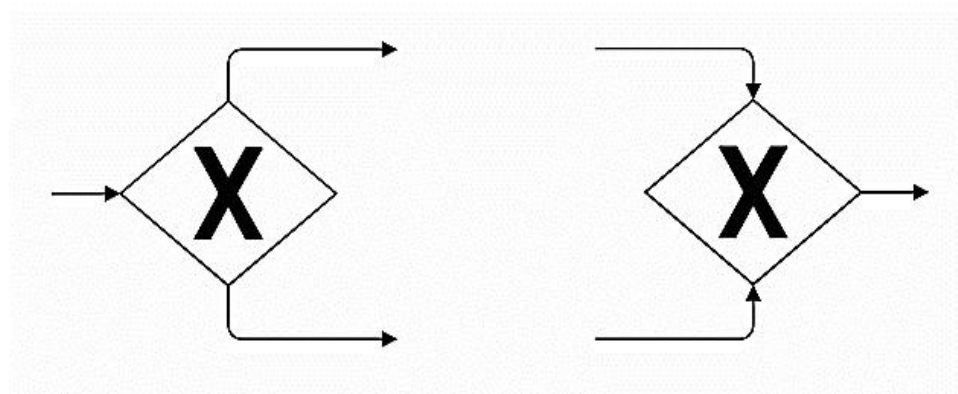
Атрибут Gateway Direction

- Unspecified = неопределенная
 - может иметь любое количество входящих и исходящих потоков
- Converging = сходящаяся
 - может иметь много входящих потоков, но только один исходящий
- Diverging = расходящаяся
 - может иметь много исходящих потоков, но только один входящий
- Mixed = смешанная
 - должна иметь несколько входящих и несколько исходящих потоков

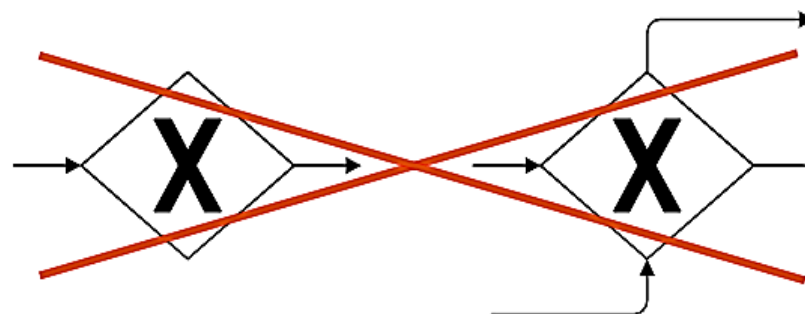


Ошибки при использовании шлюзов

Так можно



А так нельзя!

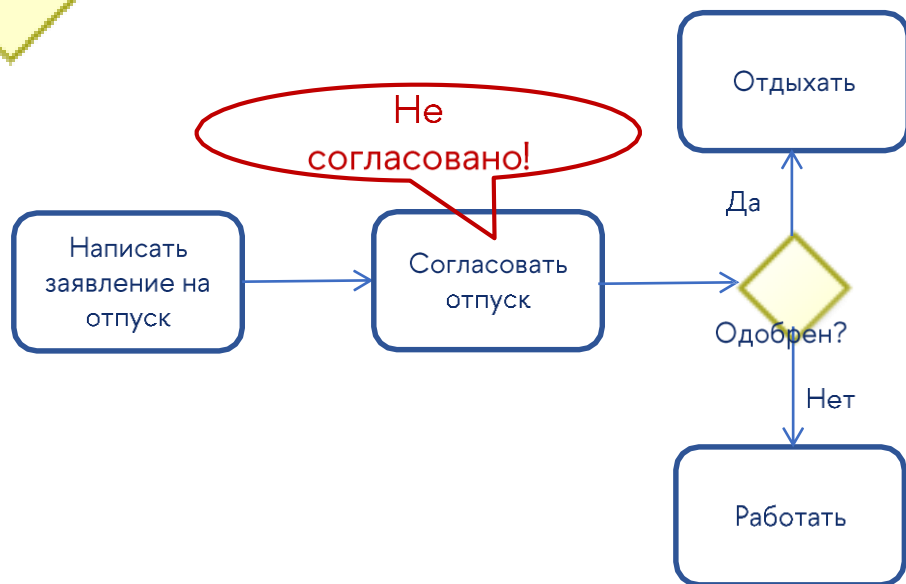
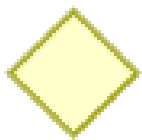


Развилка «или/или» (Exclusive (XOR) Gateway)

• Маркер



- может изображаться как с маркером, так и без.
- Оба изображения идентичны



• Расхождение

- процесс продолжается только по одному из исходящих потоков управления



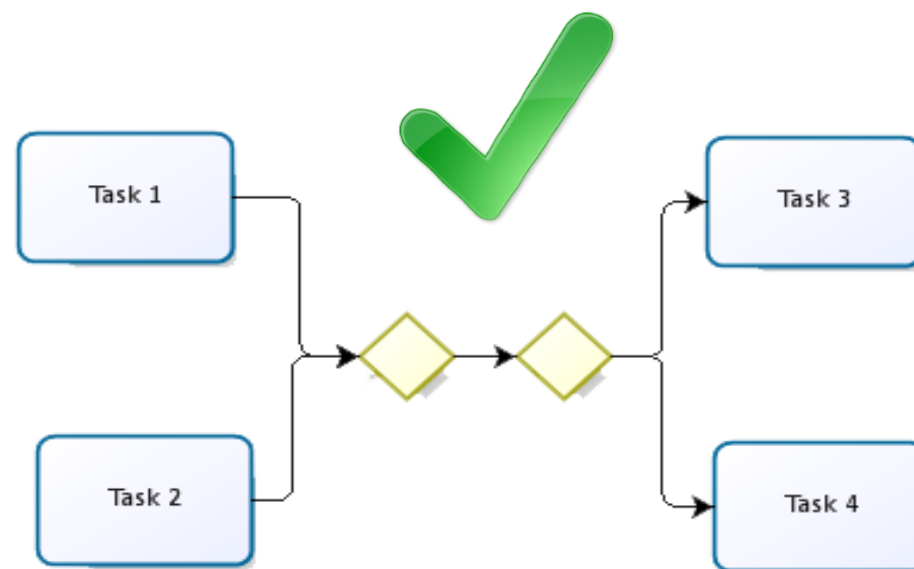
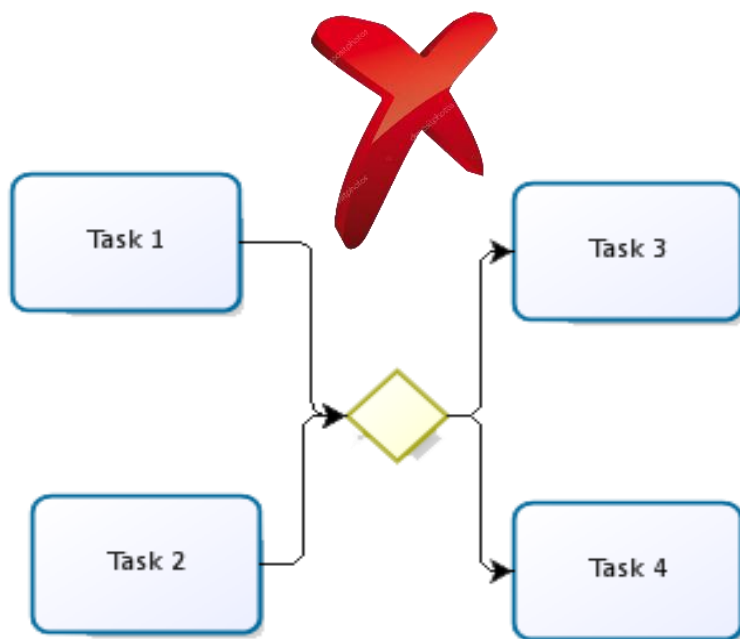
• Схождение

- не синхронизирует потоки. Каждый входящий поток будет выпущен из развилки



Развилка «или-или»: схождение и расхождение

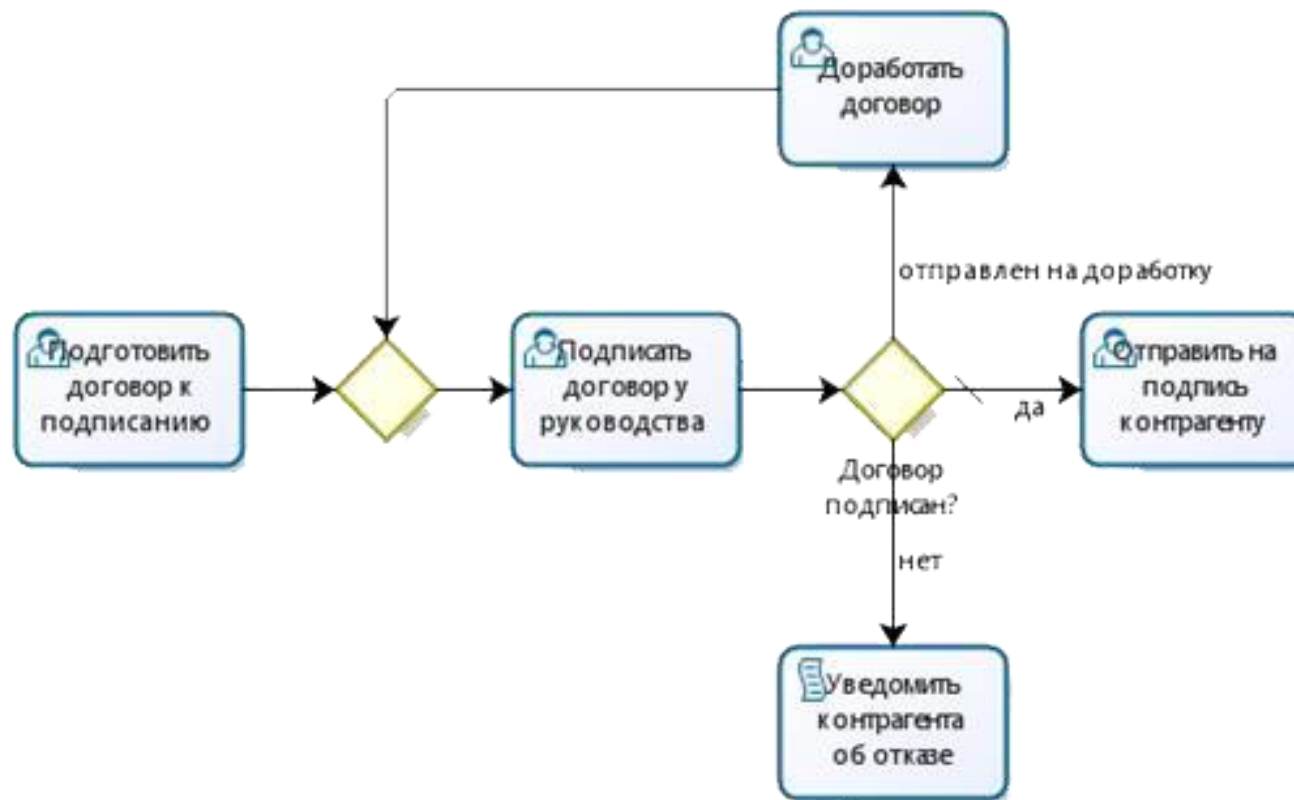
- Может быть только сходящейся или расходящейся *
- для схождения и расхождения используйте отдельные развилки



* Здесь и далее: рекомендовано разработчиками BPMN для описания процессов

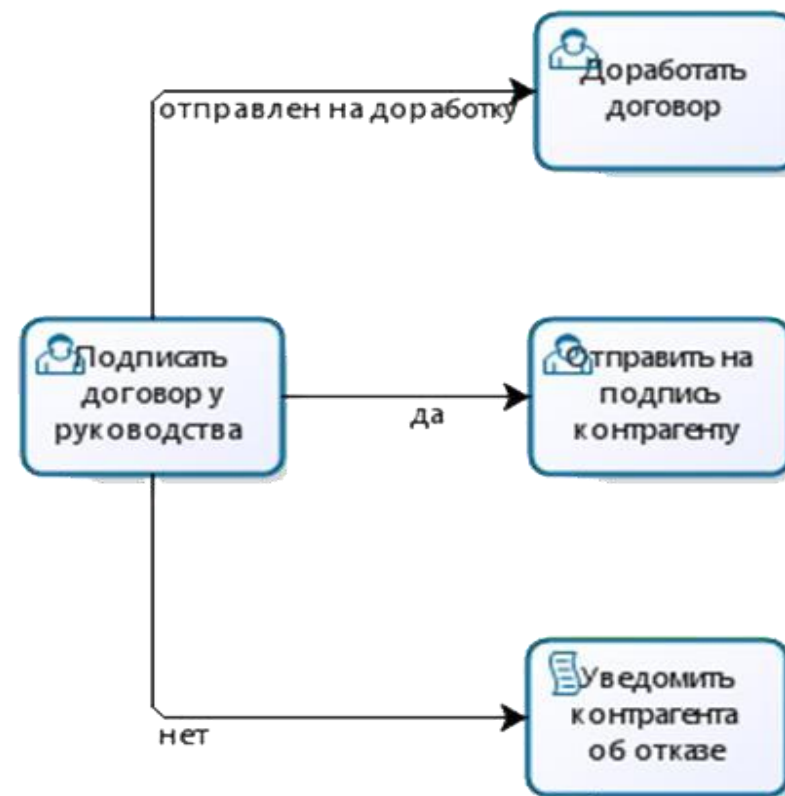
Развилка «или-или»: расхождение

- Варианты решений:
 - подписать
 - доработать
 - отказать
- Решение о подписании договора принимает человек
 - решение – это данные экземпляра процесса
- Развилка сравнивает значение данных (решение) с условиями на стрелках и выбирает соответствующий маршрут
 - если истинны несколько условий, результат непредсказуем



Неявное распараллеливание

- Часто встречающаяся ошибка: такая схема не является аналогом Exclusive Gateway.
- Это - неявное распараллеливание.
- Процесс пойдет по всем исходящим потокам.
- рекомендуется в таких случаях пользоваться параллельным шлюзом



Условный переход (Conditional Flow)

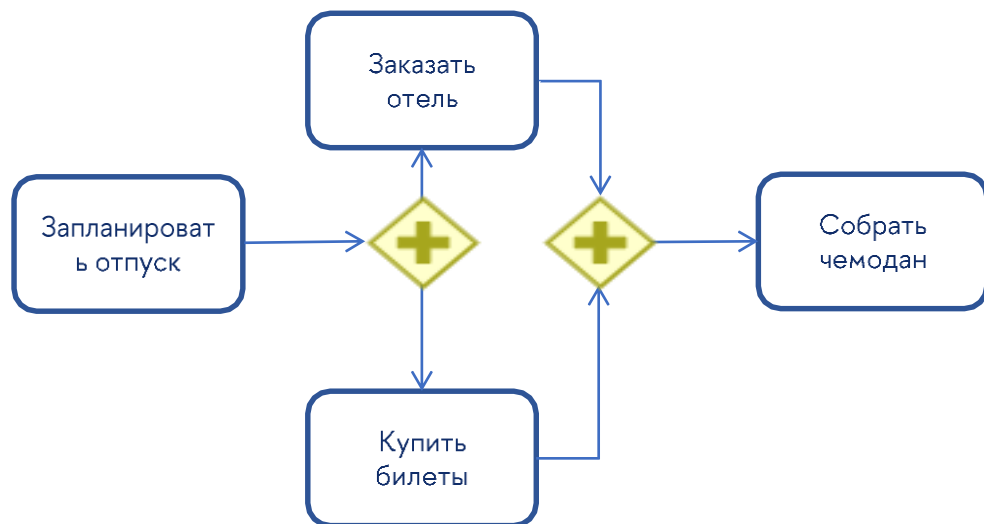
- Поведение аналогично Exclusive Gateway
 - формально такая конструкция является корректной, но рекомендуется использовать Exclusive Gateway



Параллельная развилка «и» (Parallel Gateway)



Маркер



- Расхождение

- Процесс продолжается по всем исходящим потокам управления

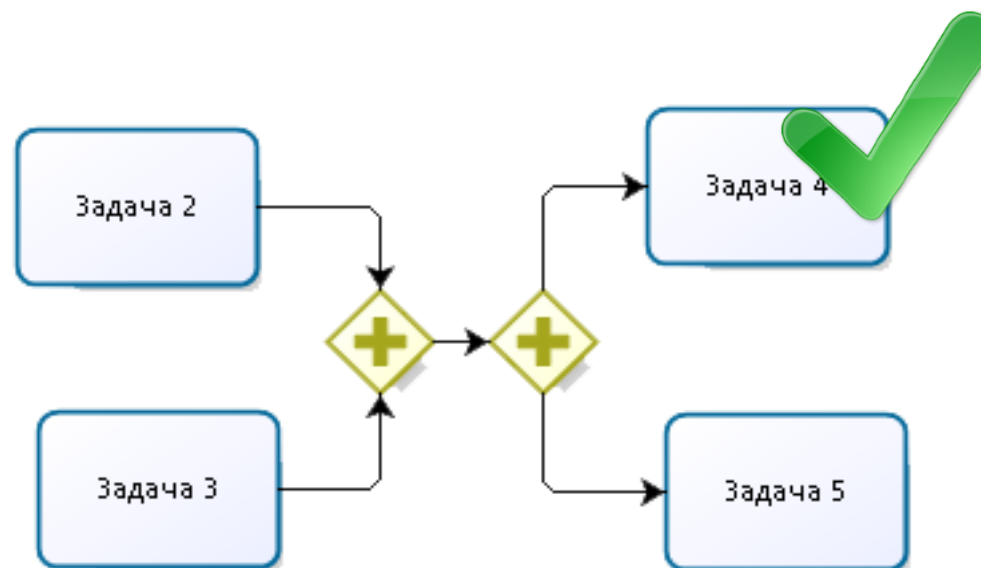
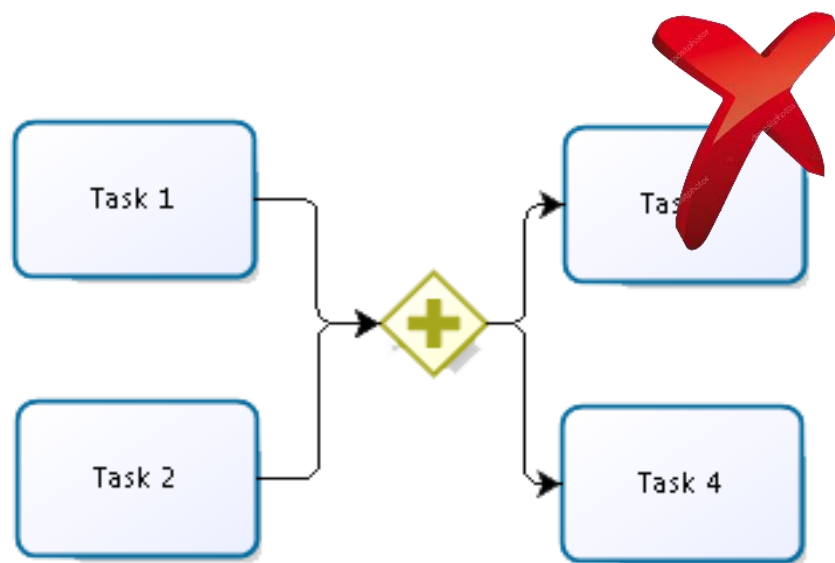


- Схождение

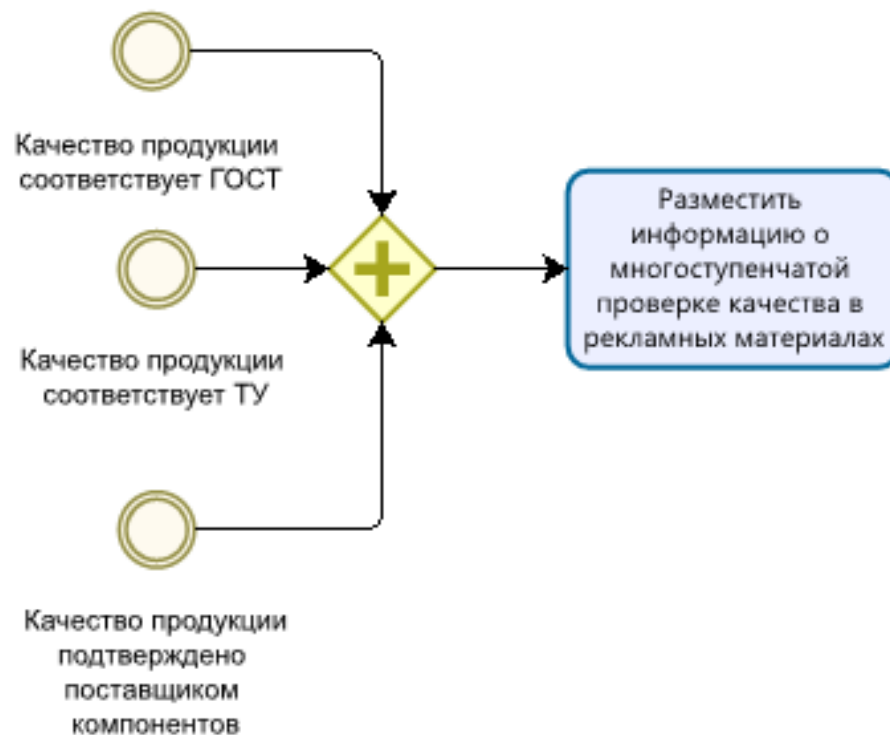
- Поток будет выпущен из развилки только тогда, когда будут получены все входящие токены



- Может быть только сходящейся или расходящейся
 - для схождения и расхождения используйте отдельные развилки

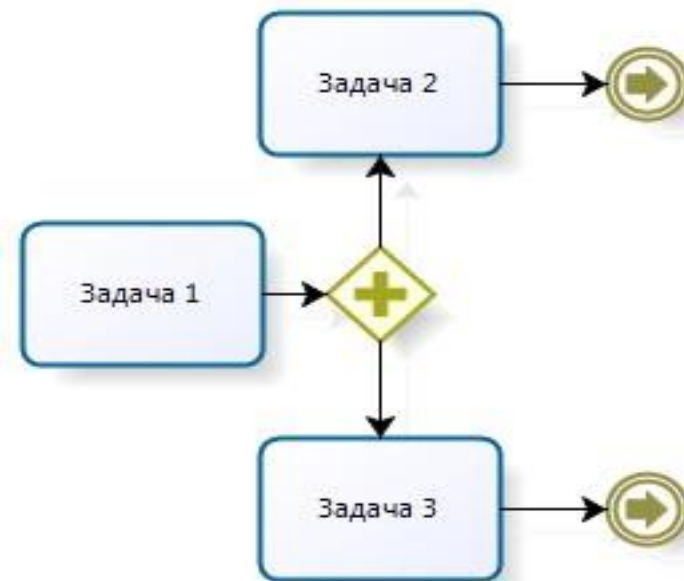


Пример использования шлюза «И»



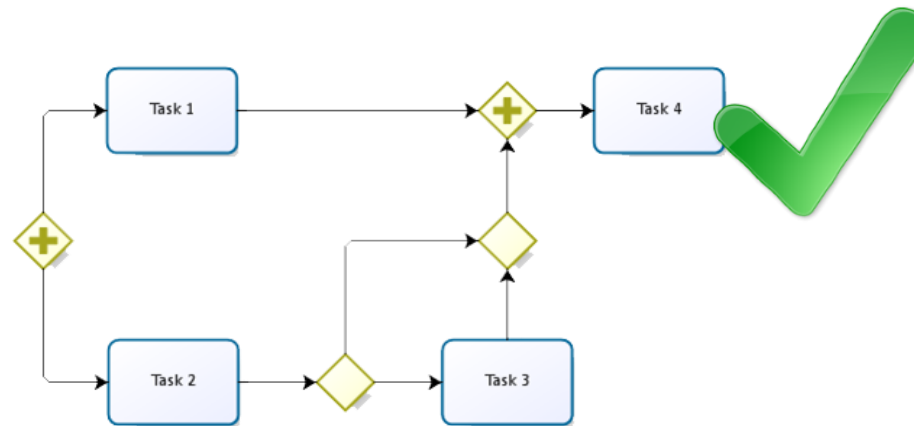
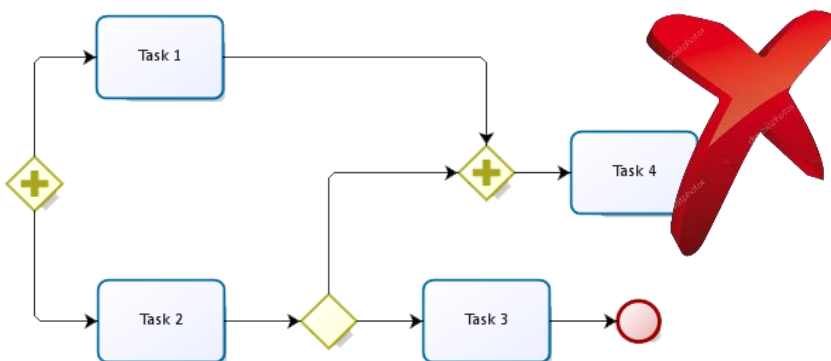
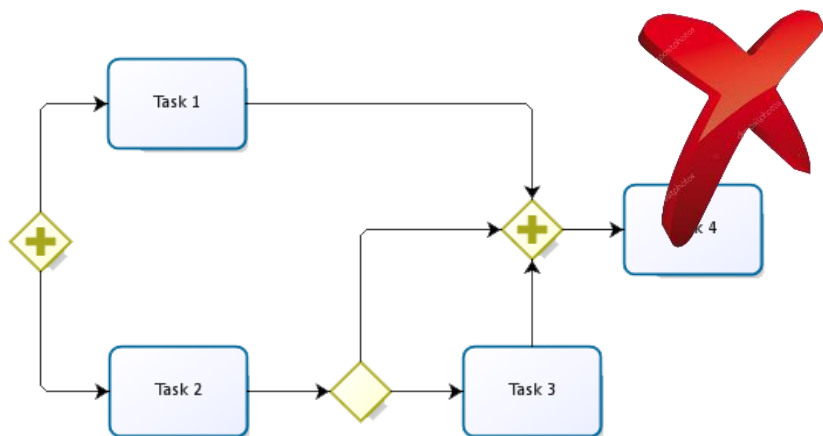
Только наступление всех трех событий инициирует задачу

- Параллельная развилка означает, что все исходящие потоки будут активны одновременно
 - активация одновременно нескольких задач или событий НЕ означает, что они будут выполнены одновременно
 - порядок выполнения параллельных задач не определяется



Параллельная развилка: схождение

- Процесс продолжится, когда придут токены из всех входящих маршрутов (все входящие потоки)



параллельное разветвление может не иметь
сходящейся развилки

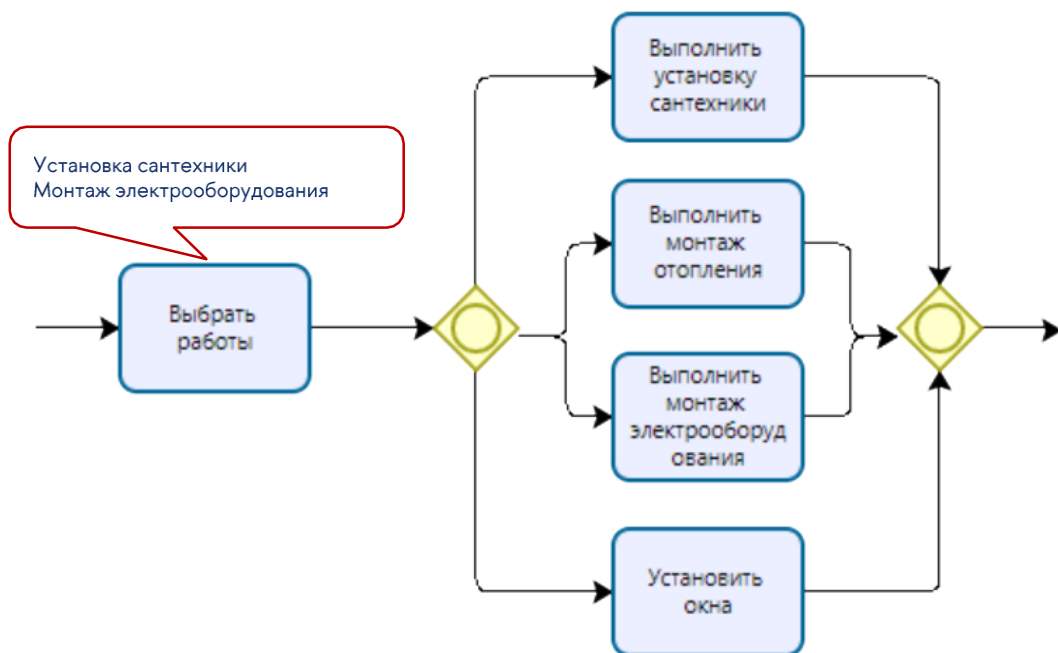


Развилка «и/или» (Inclusive (OR) Gateway)

Используется, когда одновременно могут быть истинными несколько условий



• Маркер



• Расхождение

- Процесс продолжается по всем потокам, для которых условие верно

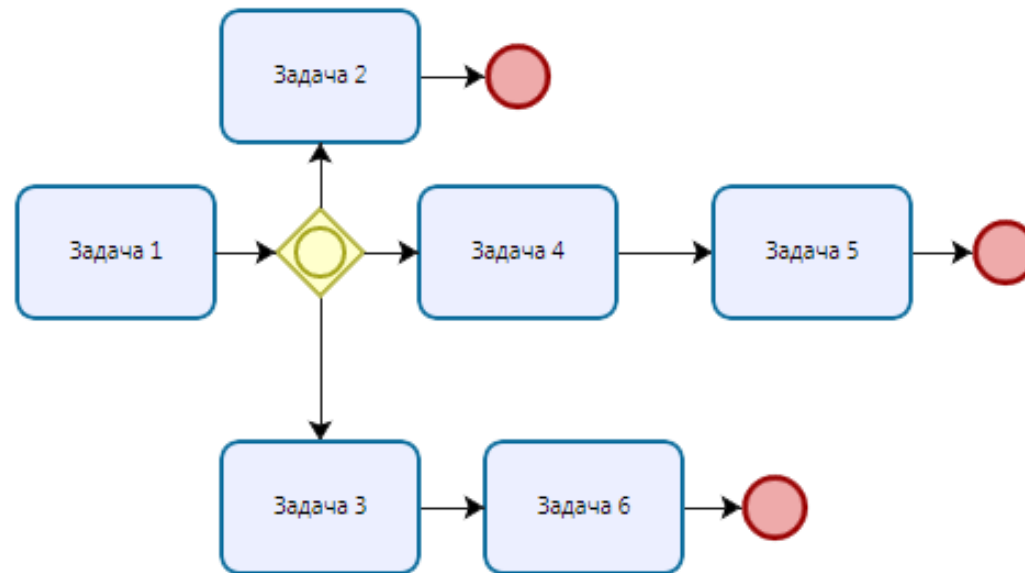
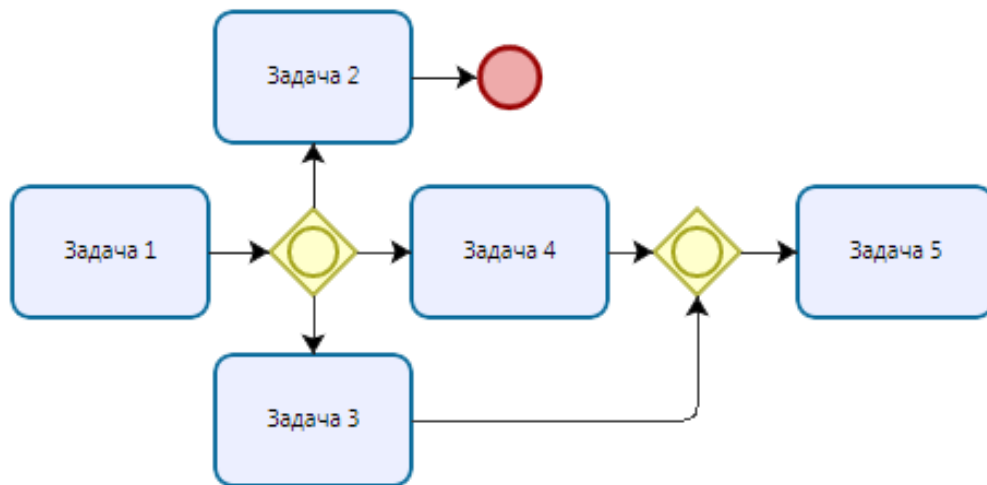
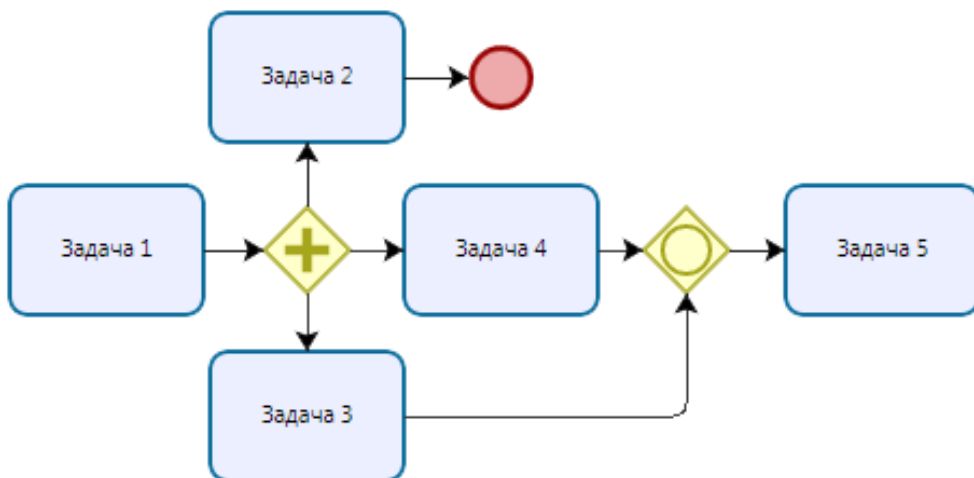


• Схождение

- Поток будет выпущен из развилки только тогда, когда будут получено столько токенов, сколько вышло из расходящейся развилки

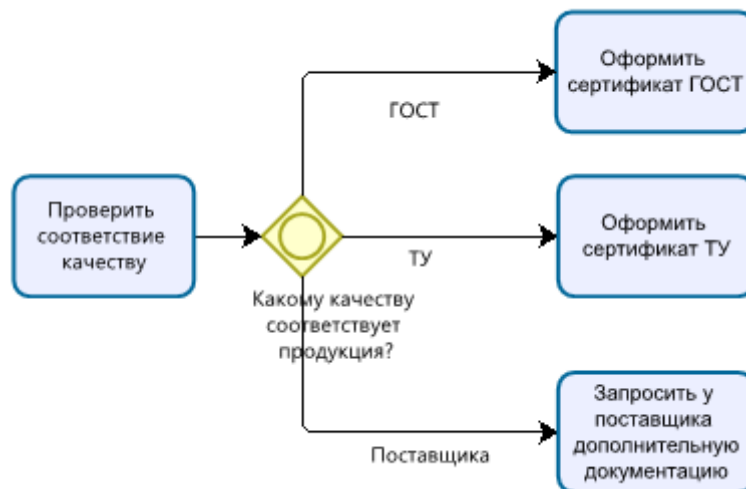
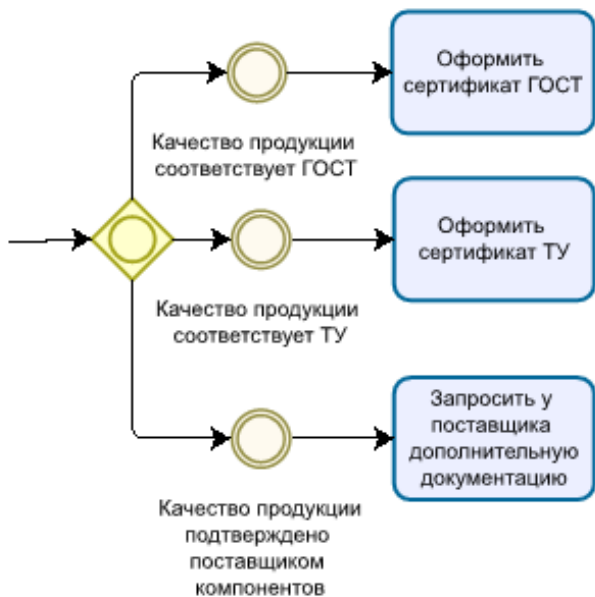


Развилка и/или: схождение

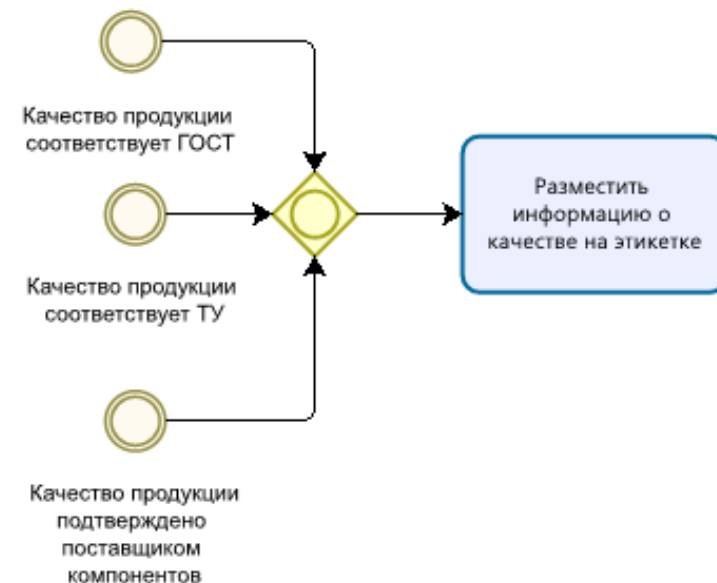


- может не иметь схождения. В этом случае потоки пойдут независимо

Пример использования шлюза «Или» (и/или)



Более лаконичный вариант
с вопросом на шлюзе

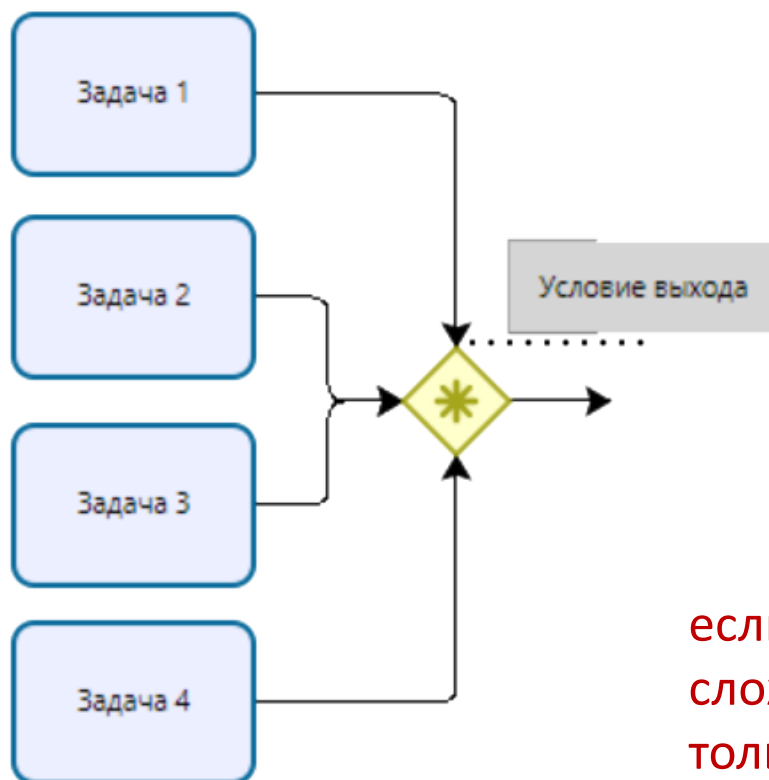


Разделение потока управления на одну или несколько альтернативных ветвей или объединение их в один маршрут

Комплексная развилка (Complex Gateway)



• Маркер



• Расхождение

- Процесс продолжается по всем потокам, для которых условие верно



• Схождение

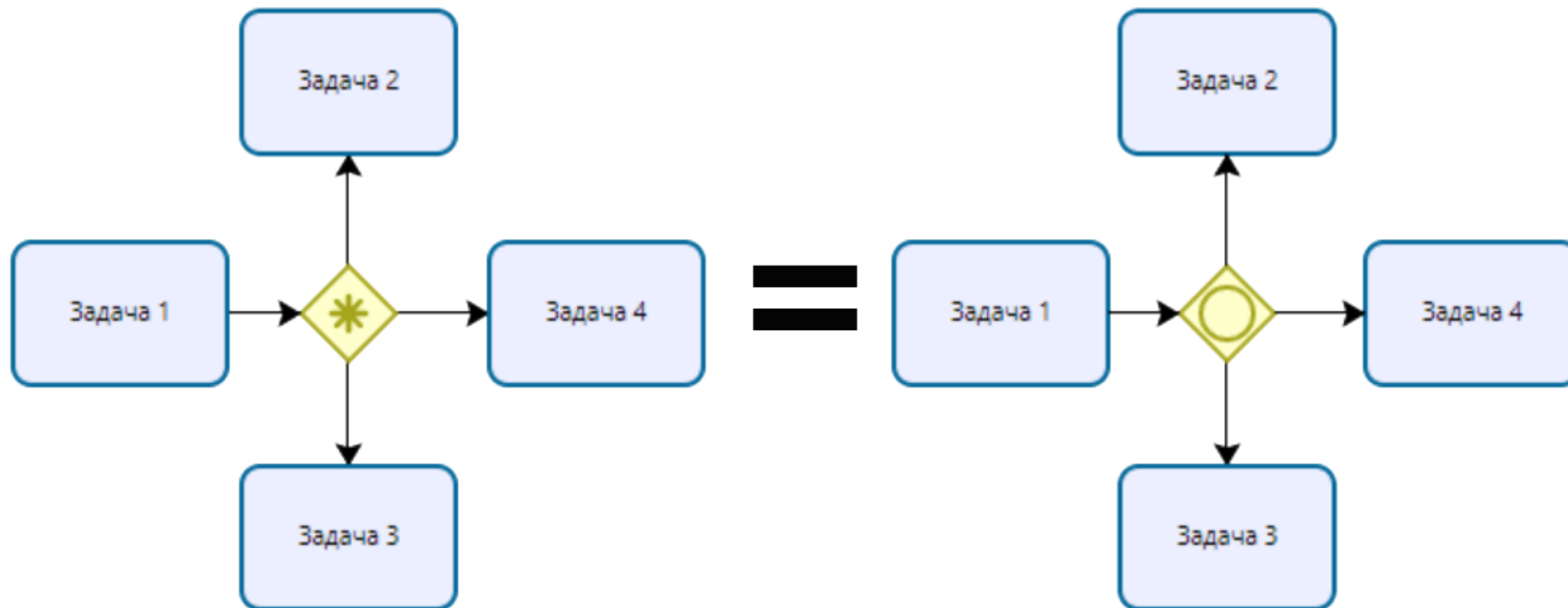
- Условие схождения определяется при проектировании
- Можно задать любое логическое условие



если в «Или» возможна любая из комбинаций, то в сложном операторе (комплексной развилке) возможны только определенные комбинации

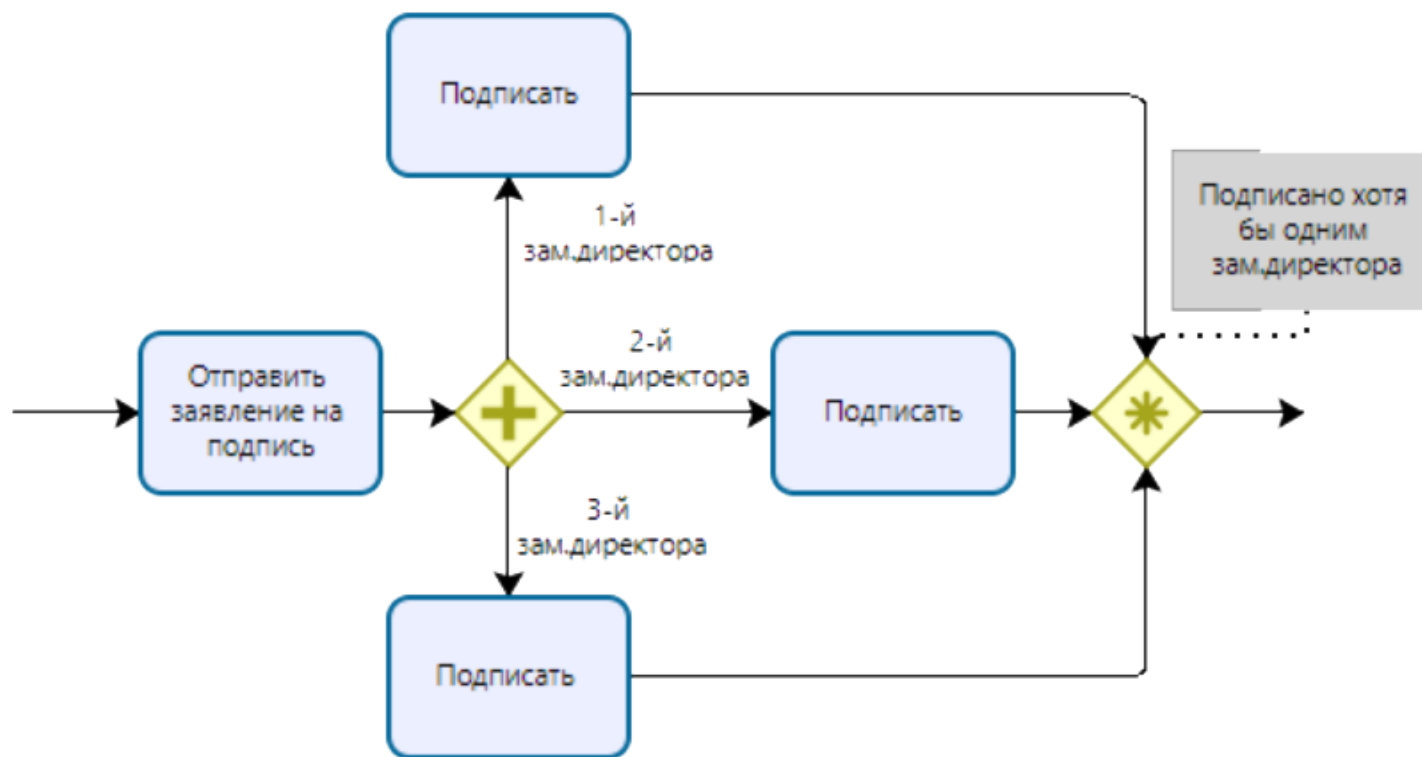
Комплексная развилка: расхождение

- Расходящаяся развилка имеет такое же поведение, как у развилки «и/или»
 - выполняются все потоки, условие для которых истинно



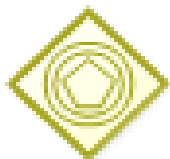
Комплексная развилка: схождение

- Условие схождения может быть произвольными
 - задается на этапе описания процесса
 - аннотируется на схеме



Развилка по событиям (Event-Based Gateway)

- Маркер



- Event-Based Gateway



- Exclusive Event-Based Gateway



- Parallel Event-Based Gateway

- Расхождение

- Ожидает первого из наступивших событий
- Процесс продолжится по пути, следующему за наступившим событием

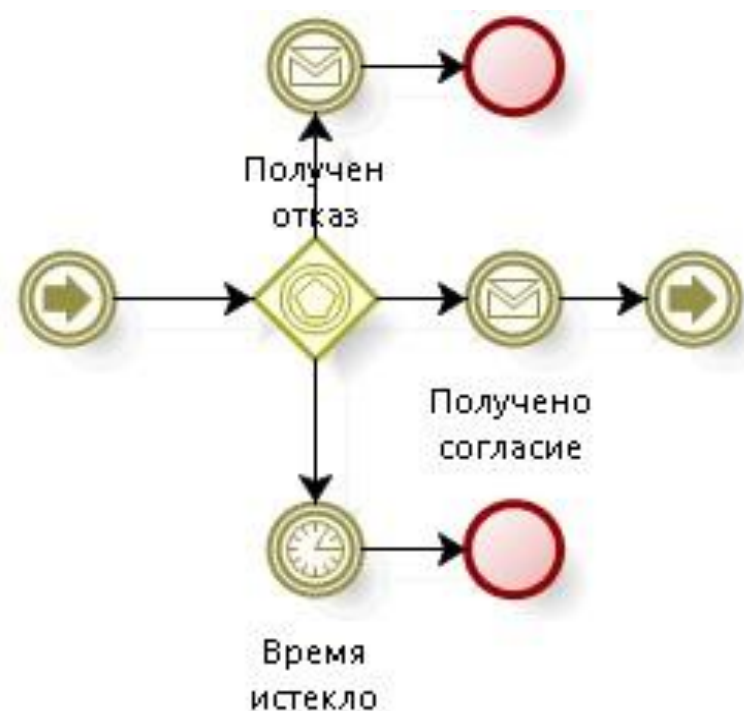


- Схождение

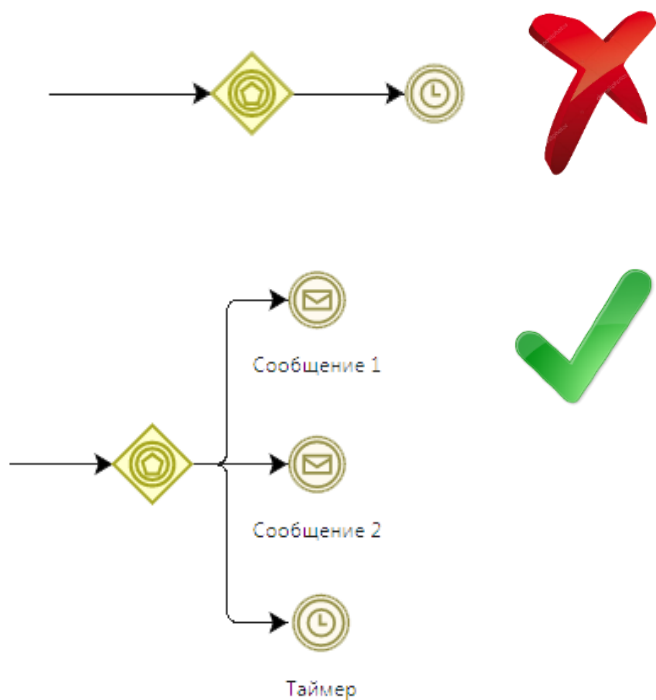


Развилка по событиям: расхождение

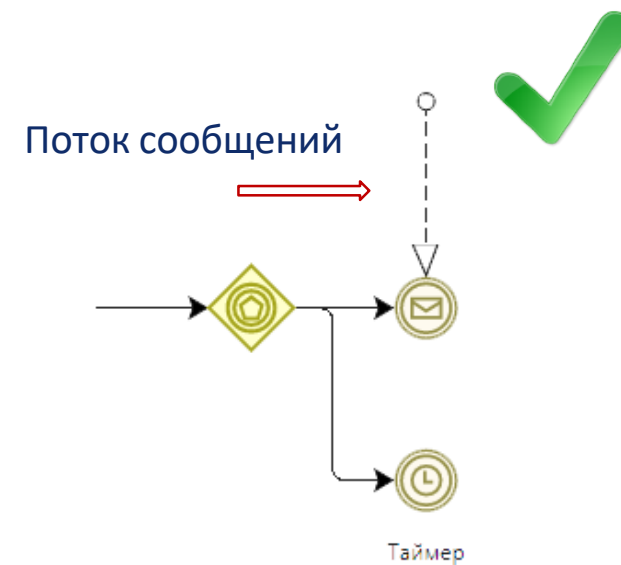
- На исходящих стрелках (потоках) нет условий
- Все исходящие потоки активируются одновременно
- Ожидает наступления первого из возможных событий
- При возникновении любого из ожидаемых событий остальные потоки завершаются (остальные события не будут приняты)
- Если ни одно из событий не наступит, то процесс застрянет



Правила для развилки по событиям

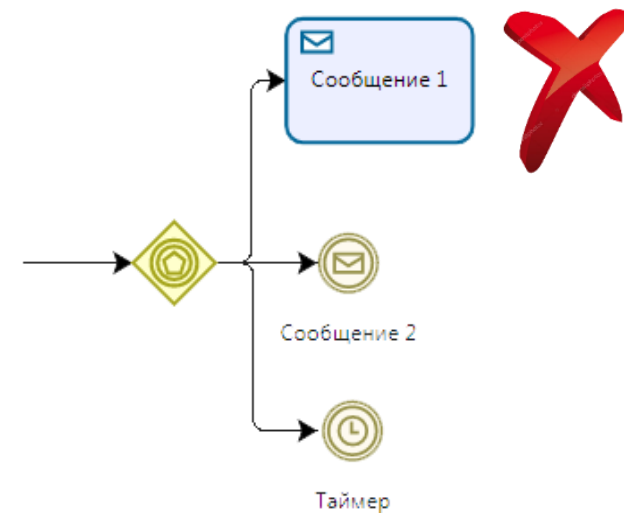
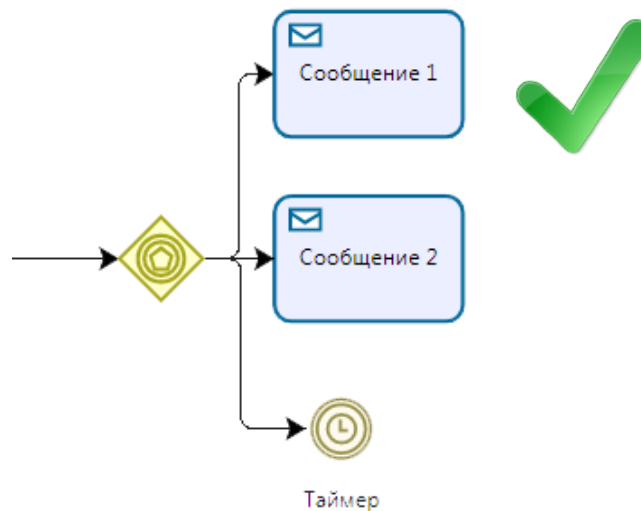
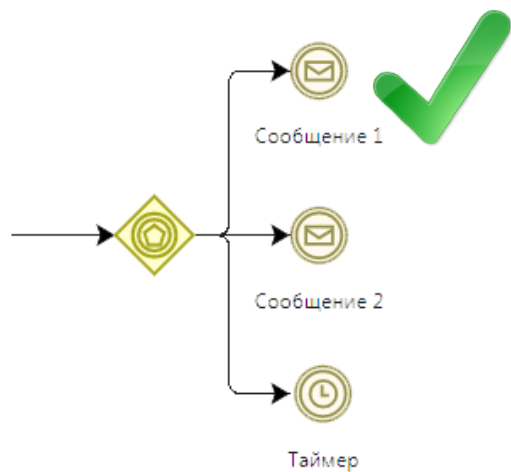


Имеет два и более исходящих
Потока - корректно



Элементы, выходящие из развилки, не
могут иметь других входящих потоков
управления, кроме потока из развилки

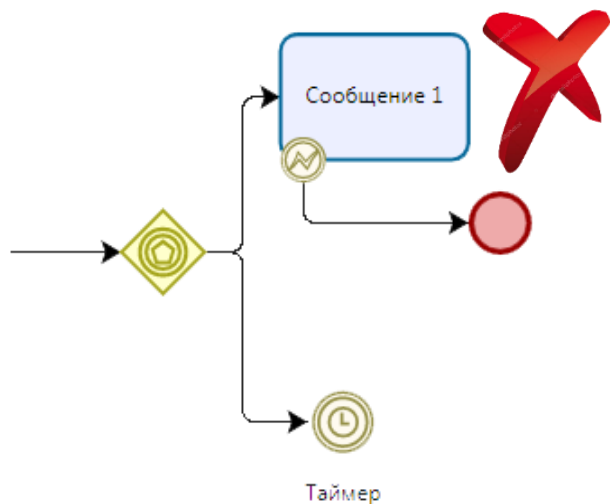
Правила для развилки по событиям



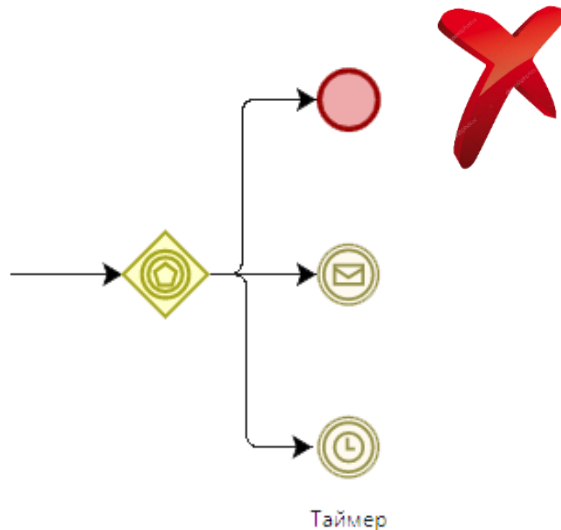
Исходящие из развилки потоки входят либо в промежуточные события, либо в задачи с типом «Received Task» (получение сообщения)

События «Получение сообщения» и задачи «Получение сообщения» не могут использоваться в одной развилке (либо задачи, либо события)

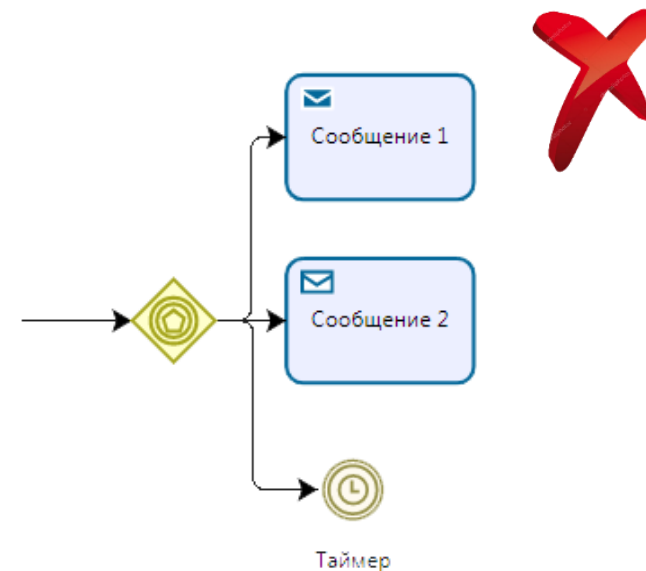
Правила для развилки по событиям



Не могут иметь
присоединенных событий



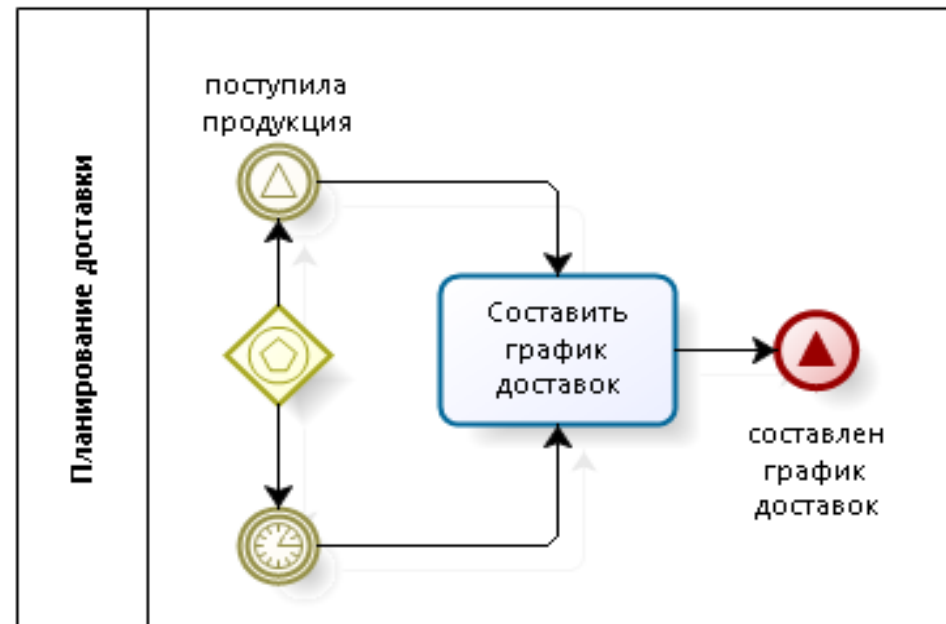
Не могут быть использованы события-инициаторы и
завершающие события!



Могут быть использованы только следующие типы событий:
сообщение-обработчик, сигнал-обработчик, таймер, триггер по
данным (conditional) и множественное событие.

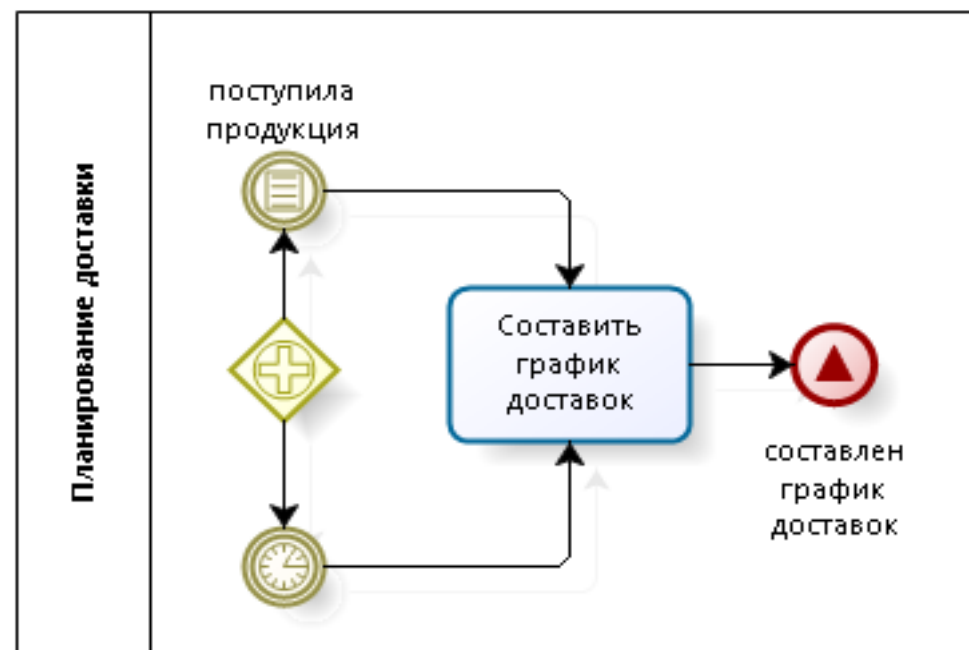
Развилка по событиям - Эксклюзивный событийный шлюз Exclusive Event- Based Gateway

- Используется для старта процесса
- Запускает новый экземпляр процесса при наступлении первого из возможных событий
 - Не может иметь входящих потоков
 - Маркер внутри ромба выглядит так же, как событие «Multiple Start Event»



Параллельная развилка по событиям Parallel Event-Based Gateway

- Используется для старта процесса
- Запускает новый экземпляр процесса при наступлении всех ожидаемых событий
 - Не может иметь входящих потоков
 - Маркер внутри ромба выглядит так же, как событие «Parallel Multiple Start Event»





Высшая
школа
бизнеса

Циклы и подпроцессы



Loop = цикл

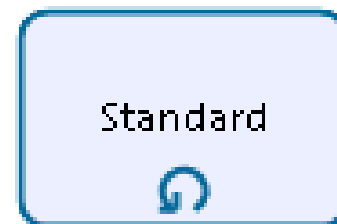
- Циклом называют действие, повторяющееся несколько раз
- Количество повторений может быть определено как:
 - заранее заданное условие, которое должно выполняться
 - условие, которое вычисляется при каждом повторении
 - для списка объектов
- Если повторяется не одно действие, а последовательность действий, то циклическим будет не один шаг, а подпроцесс



Стандартный (Standard) цикл, Стандартное повторение

- Цикл for, do-while, do-until
- Количество повторений цикла заранее не известно
 - например, автомобиль должен возвращаться на склад за товаром до тех пор, пока не закончится рабочий день; «Позвонить клиенту» (результат - дозвониться)
- Количество повторений задается счетчиком
 - например, выполнить три раза
- Вид цикла аннотируется на схеме
 - задается атрибутами активности
 - можно добавить аннотацию на схеме

– действие будет повторяться до тех пор, пока не получен нужный результат





Multi-Instance (MI) = цикл по объектам

- Цикл foreach
 - например, по строкам документа
- Число повторов цикла известно до его начала
- Выход из цикла определяется условием {None | One | All | Complex}
 - None: после обработки каждого объекта (будет создано много потоков)
 - One: после завершения обработки первого объекта
 - All: после завершения обработки всех объектов
 - Complex: задается условием

Multi-Instance
Parallel



- **Parallel = параллельный**
 - все экземпляры выполняются одновременно (напр., ознакомиться с регламентом должны все сотрудники отдела одновременно)

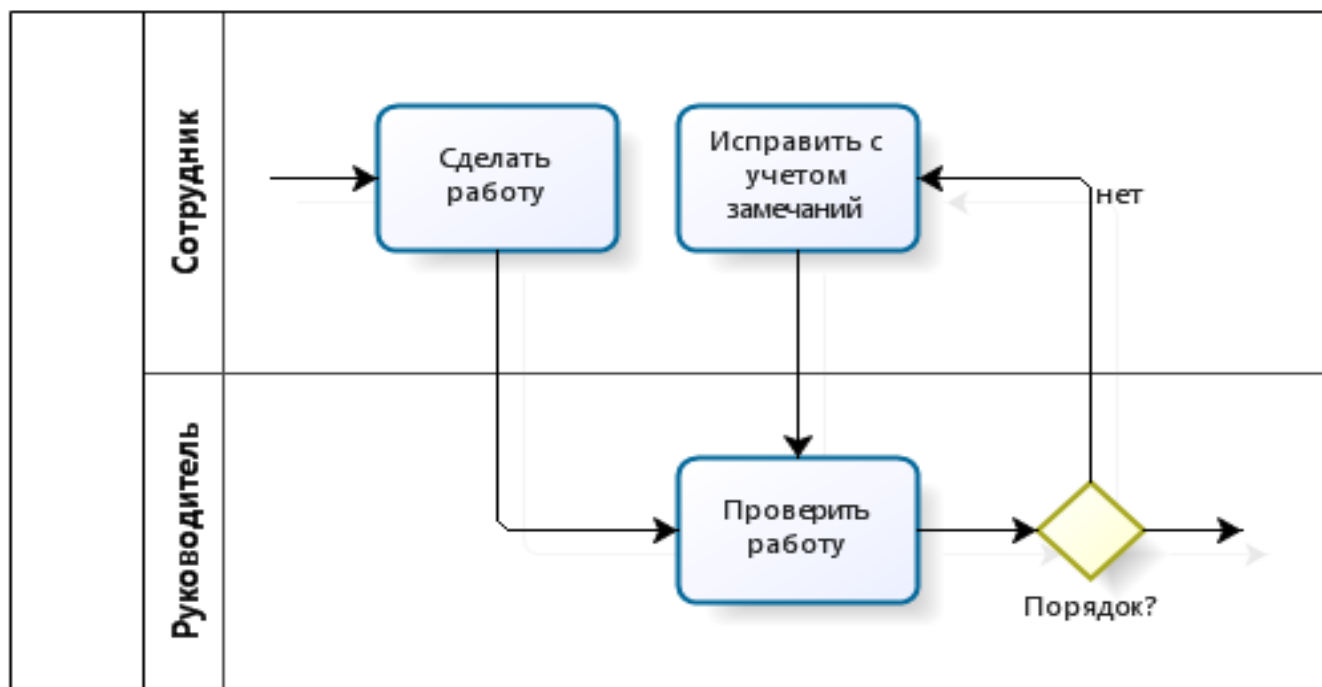
Multi-Instance
Sequential



- **Sequential = последовательный**
 - экземпляры выполняются последовательно (напр., ознакомиться с регламентом должны сначала руководители, потом менеджеры и т.д.)

Неявный цикл

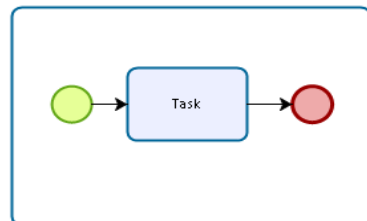
- Все варианты стандартного цикла реализуются при помощи Exclusive Gateway
 - делать работу до тех пор, пока не будет «порядок»



Subprocess = подпроцесс



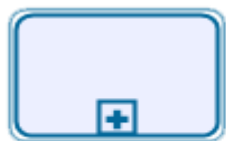
Collapsed Subprocess



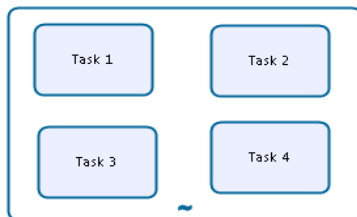
Expanded Subprocess



Multiple Subprocess



Transactional Subprocess



Ad-Hoc Subprocess

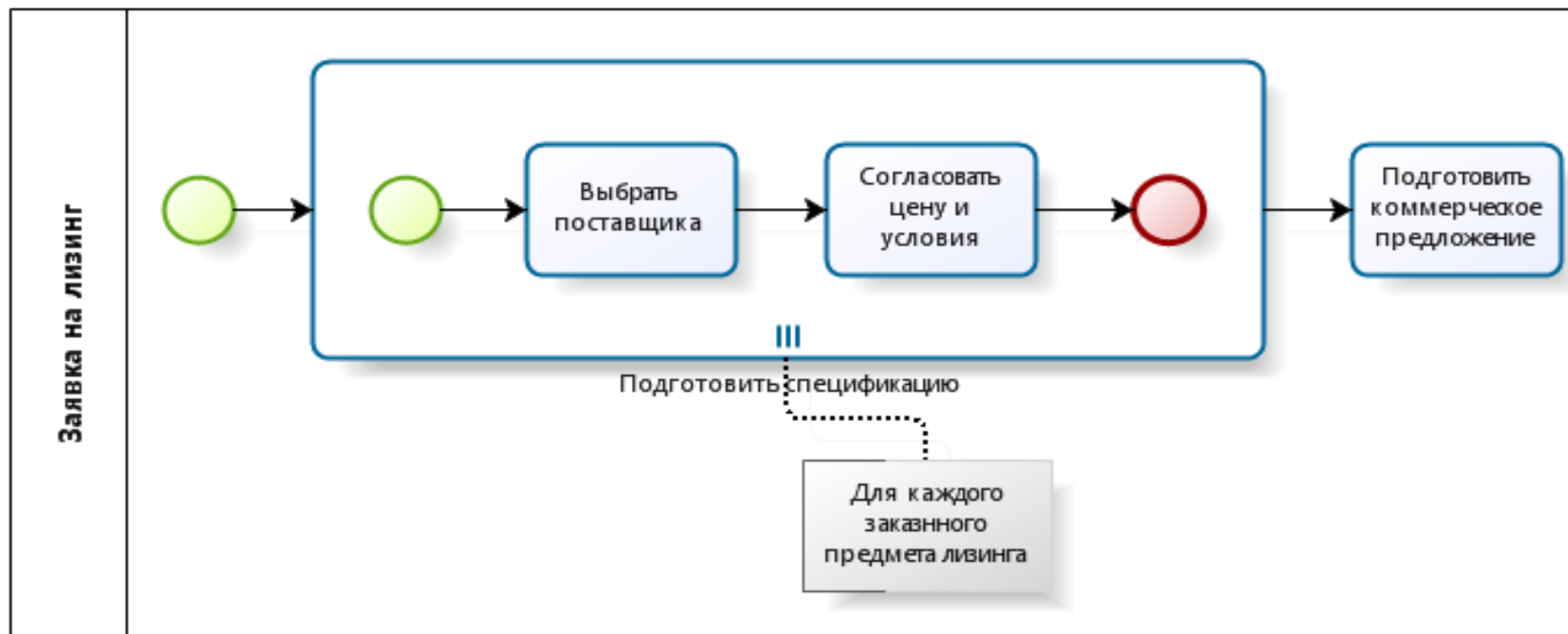


Event Subprocess

- Collapsed/Expanded = свернутый / развернутый
 - способ отображения
- Multiple = множественный
- Ad-Hoc = «для данного случая»
- Transactional = транзакционный
- Event Subprocess = обработчик событий
 - BPMN 2.0

Multi-Instance = цикл по объектам

- Может выполняться последовательно или параллельно
- Пример: документ – строка документа
 - обработка всех строк ведется параллельно (вертикальный полосы маркера)
 - обработка документа продолжается после завершения обработки всех строк (задается атрибутом подпроцесса)



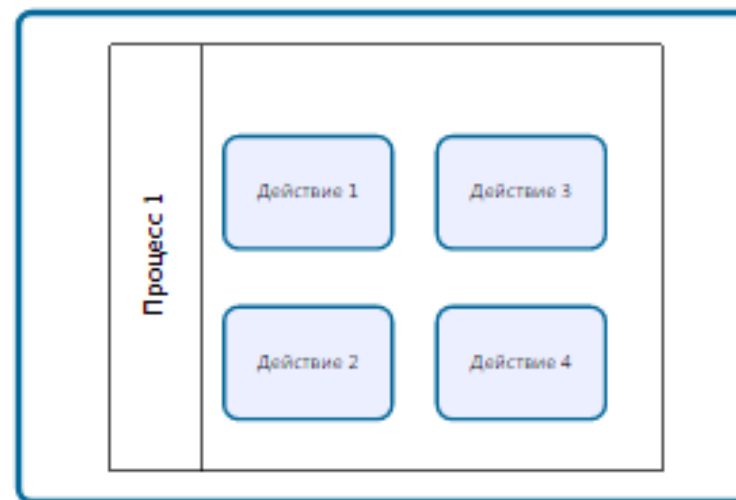
Ad-Hoc = для данного случая

- Задачи выполняются в произвольной последовательности
- Не обязательно все
- Возможно с повторами
- Условие выхода задаются атрибутом модели подпроцесса



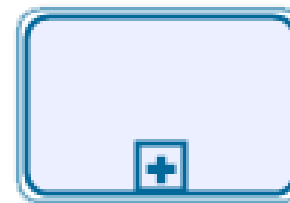
Параллельный блок

- Действия в подпроцессе **не связаны потоками управления**
- Задачи выполняются в произвольной последовательности
- Обязательно все
- Без повторов
- Условие выхода: все задачи выполнены



Transactional = транзакционный подпроцесс

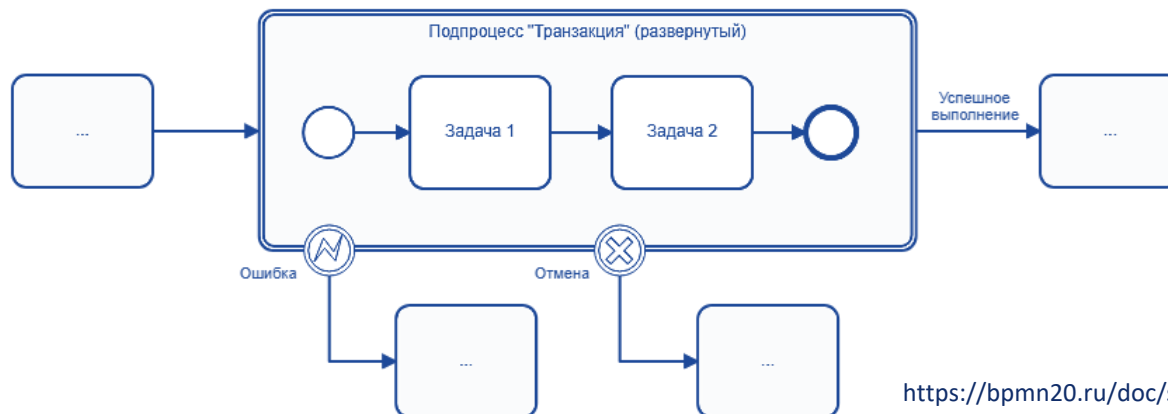
- изображается с двойной тонкой линией
- используется в тех случаях, когда при любом завершении процесса все действия должны быть завершены, либо отменены дополнительными действиями
- При обычном завершении ведет себя, как обычный Подпроцесс



Transactional Subprocess

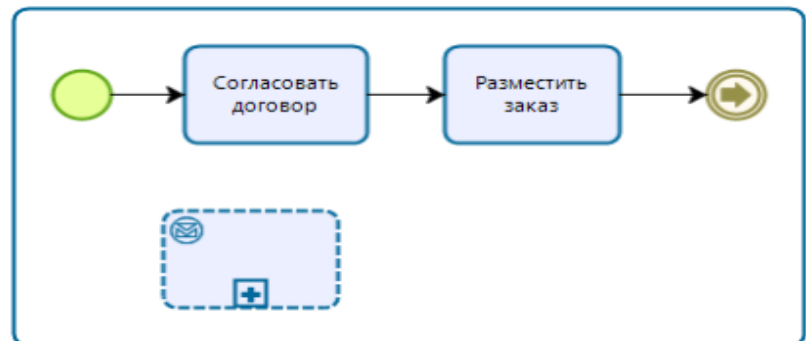
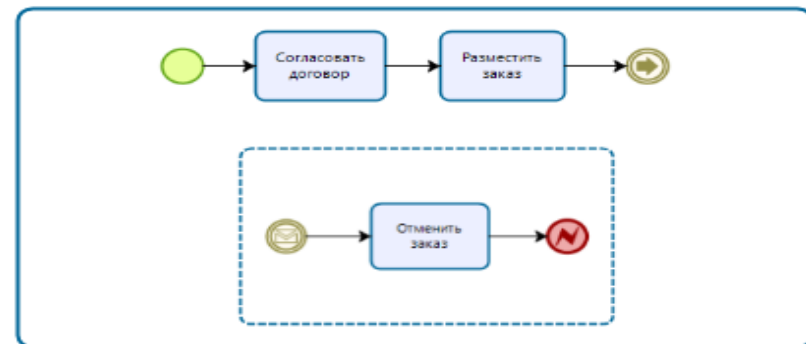
Отмена – изображается с помощью граничного (присоединенного к границе подпроцесса) события «Отмена». При возникновении события «Отмена» (Cancel) произойдет компенсация («откат назад») определенных действия транзакции, либо возврат к началу процесса.

Ошибка – изображается с помощью граничного события «Ошибка». Появление ошибки означает, что **действия Транзакции BPMN** выполняются неверно и становятся невозможными успешное завершение и отмена Транзакции. Выполнение Транзакции при этом завершается без компенсации, а поток операций родительского процесса продолжается от граничного события «Ошибка».

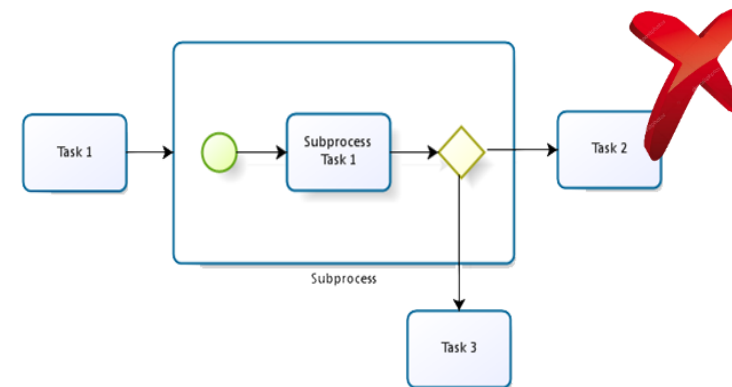
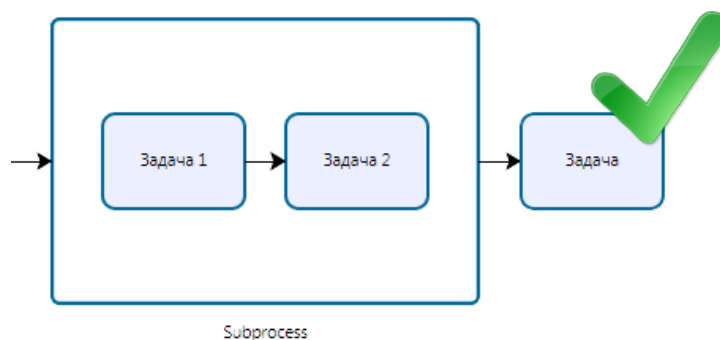
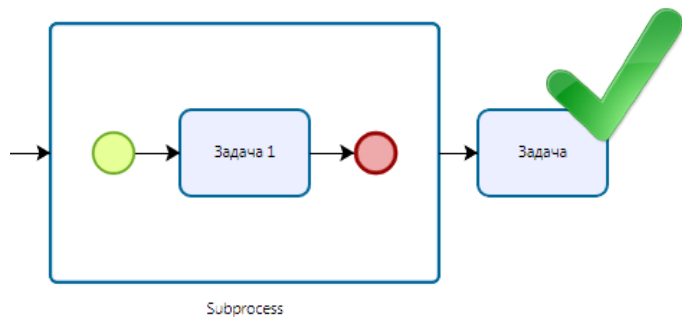


Event Subprocess = обработчик событий, событийный подпроцесс

- Не может иметь входящих и исходящих потоков
- Может быть прерывающим и непрерывающим (прерывать или не прерывать родительский процесс)
 - определяется типом стартового события прерывающее/непрерывающее
- Может иметь только одно стартовое событие
- Стартовое событие может быть только следующих типов:
 - Message (сообщение)
 - Error (ошибка)
 - Escalation (эскалация)
 - Compensation (компенсация)
 - Conditional (триггер по данным)
 - Signal (сигнал)
 - Multiple (множественное стартовое)

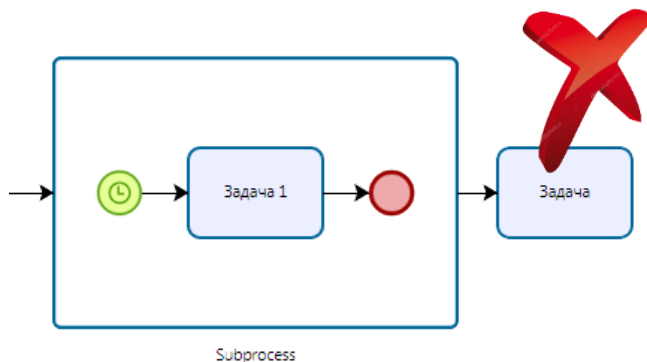


Правила для подпроцессов

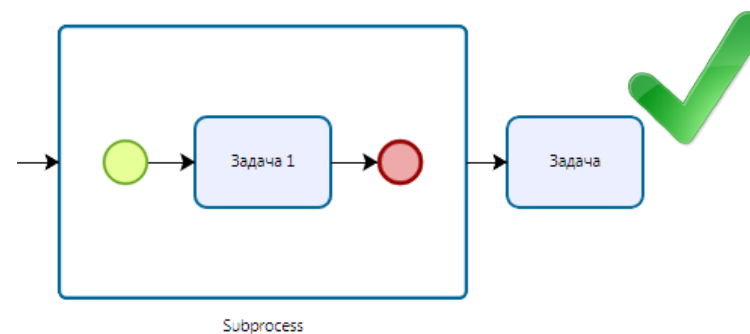


У подпроцесса есть явный или неявный (нет событий) старт и завершение

Потоки управления не могут пересекать границы подпроцесса

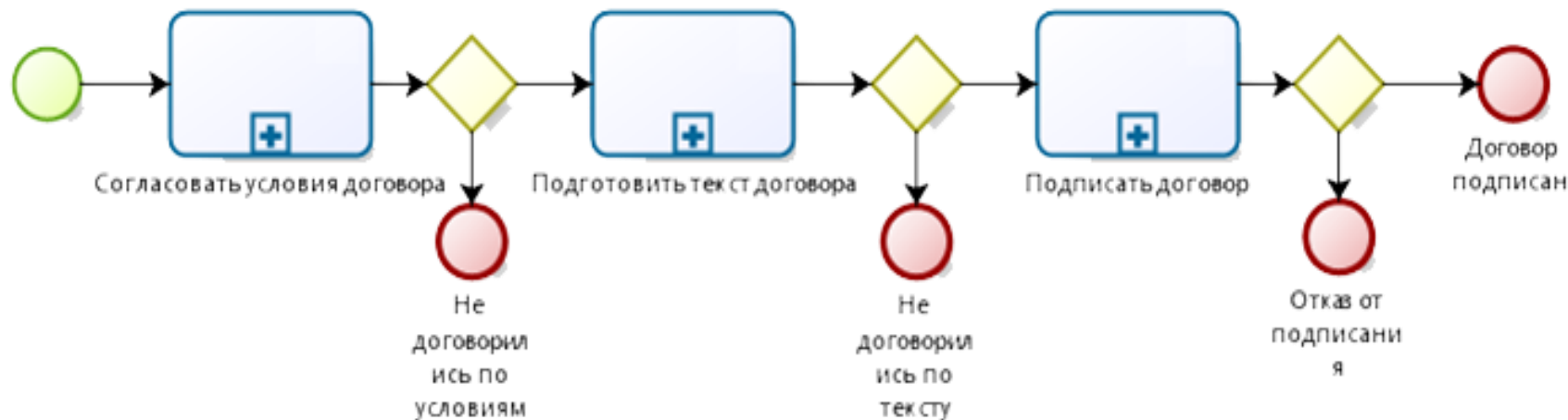


Стартовое событие процесса может быть только пустым (none start event)



Для чего нужны подпроцессы

- Декомпозиция процессов
 - Упрощает понимание и последующее изменение процесса
 - хороший стиль: максимум 7-9 задач на одном уровне
- Повторное использование одних и тех же подпроцессов в разных процессах





Высшая
школа
бизнеса

События

События BPMN 2.0	Начальные (стартовые)			Промежуточные события				Заверша- ющие
	Верхнеуровневые	В прерывающем событийном подпроцессе	В не прерывающем событийном подпроцессе	Обрабатывающие	Граничные прерывающие	Граничные непрерывающие	Генерирующие	Генерирующие
Абстрактное (Простое): нетипизированное событие, обычно показывающее начало или окончание процесса или подпроцесса								
Сообщение: отображает получение или отправку сообщений. Сообщения должны иметь получателя (участника, роль и т.д.)								
Таймер: предназначен для обозначения циклических событий, интервалов времени или конкретных моментов времени								
Эскалация: используется для передачи управления на уровень родительского бизнес-процесса относительно текущего								
Условное: ожидает наступления определенных условий. Срабатывает если условие стало истинным.								
Ссылка: используется для переноса потока управления в другое место или на другой лист диаграммы бизнес-процесса								
Ошибка: используется для генерации и обработки заранее определенного для данной пары событий типа ошибок								
Отмена: используется в подпроцессах «Транзакция» для инициирования отмены, а также для обработки отмены								
Компенсация: используется для запуска и обработки компенсации («отката» выполненных действий процесса)								
Сигнал: генерация и обработка сигналов. Сигнал в отличие от сообщения может иметь множество получателей								
Составное: обработка одного события из заданного множества или генерация определенного набора событий								
Параллельное составное: используется для обработки всего множества параллельных событий								
Останов (Терминатор): вызывает немедленное безусловное прекращение всех активных потоков бизнес-процесса								



Событие (Event)

- Может быть начальным, конечным и промежуточным
- Изображаются в виде окружности с маркером или без него.

События разделяются на 2 категории: возникающие из-за какой-то причины и инициирующие какой-то результат. И причина возникновения события, и результат, который инициирует событие, называются **триггером**.

События, обрабатывающие триггер, который привел к их возникновению, называются **обработчиками**. События, которые инициируют триггер (или некий результат), называются **инициаторами**.



1. события начала определяют условия старта процесса

В процессе стартовое событие может быть только входящим по факту («обработчик»): что-то произошло, какая-то информация получена – **кружок с тонкой границей**. Не могут иметь входящих потоков управления, только исходящий поток

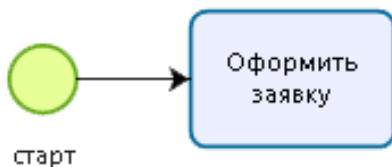


2. промежуточные события определяют развитие процесса, влияют на ход его выполнения, могут быть как входящими, так и исходящими – **кружок с границей в виде двойной линии**. Могут быть как событиями – обработчиками, так и событиями – инициаторами



3. событие окончания (завершающее), отражает условие, при котором процесс окончен – моменты, отражающие факт завершения процесса – **кружок с жирной границей**. Всегда являются событиями – инициаторами, не могут иметь исходящих потоков управления

Старт процесса (Start Event)



Обычный старт

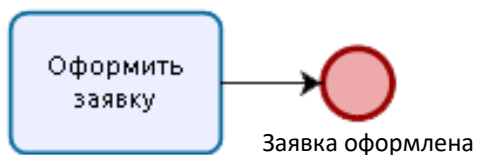


Двойной старт

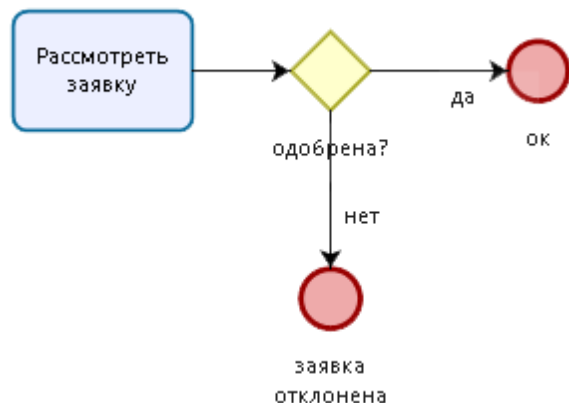
- Не обязателен, но, если есть конечное событие, то должно быть хотя бы одно стартовое событие, и наоборот
- Все объекты, у которых нет входящего потока, трактуются как стартовые и по ним при старте будут создаваться экземпляры процесса
- Может быть несколько стартовых событий
 - не должно быть несколько пустых стартовых событий в одном процессе
- Разные типы стартовых событий
 - будем рассматривать позже

Для полного понимания процесса лучше использовать точные и подписанные события старта

Завершение процесса (End Event)



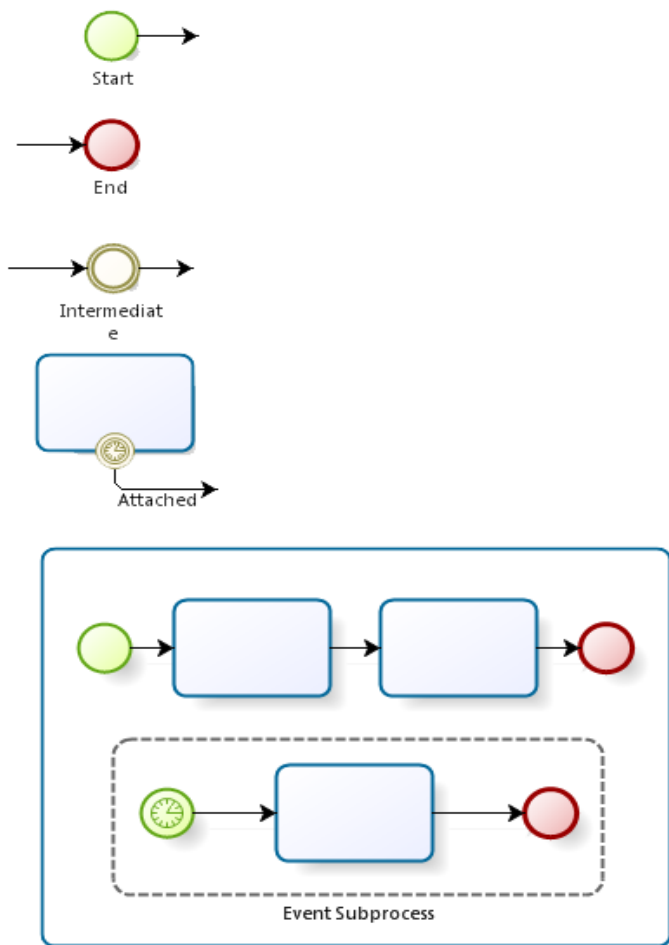
Одно завершающее событие



Несколько завершающих событий

- Не обязательно
- Если есть стартовое событие, то должно быть хотя бы одно завершающее событие
- Может быть несколько завершающих событий
 - несколько завершающих событий – рекомендуемый стиль моделирования
- Разные типы завершающих событий
- Рекомендуется именовать как бизнес-результат

Все события: по месту возникновения

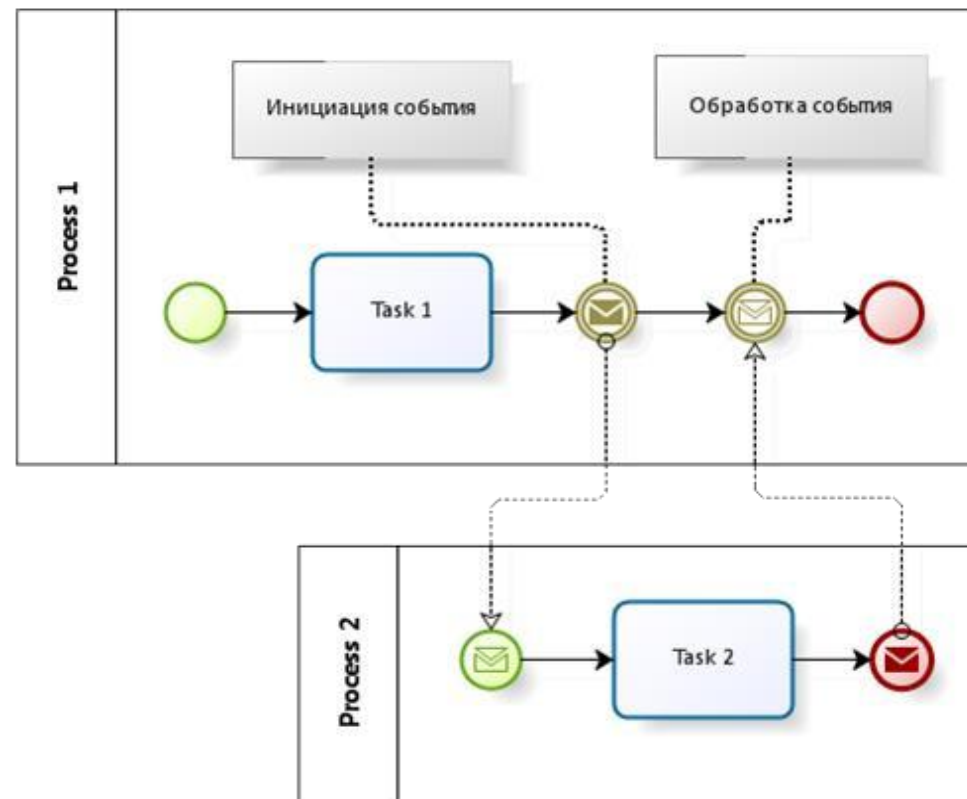


- Start = старт
 - создание экземпляра процесса
- End = завершение
 - окончание процесса
 - Intermediate = промежуточное
 - throw = инициирующее событие
 - catch = обработка события
 - **Attached = прикрепленное (к задаче)**
 - обработка события исключения
- Event Subprocess = подпроцесс обработчик
 - альтернативный вариант обработки события BPMN 2.0

Throw/Catch Event = инициация/обработка события

- Throw Event = инициация
 - изображается закрашенным маркером
 - процесс продолжается по исходящему маршруту
 - Завершение (End Event) может быть только инициатором
- Catch Event = обработка
 - изображается незакрашенным маркером
 - Процесс ждет наступления события
 - Старт (Start Event) может быть только обработчиком

- Аннотации для обучения, в модели такой текст использовать нельзя





Non-Interrupting Event = непрерывающее событие

- Обозначается окружностью с пунктирной линией
- Не прерывает основной поток процесса
- Может быть только граничным, промежуточным или стартовым в подпроцессе-обработчике событий (Event Subprocess)



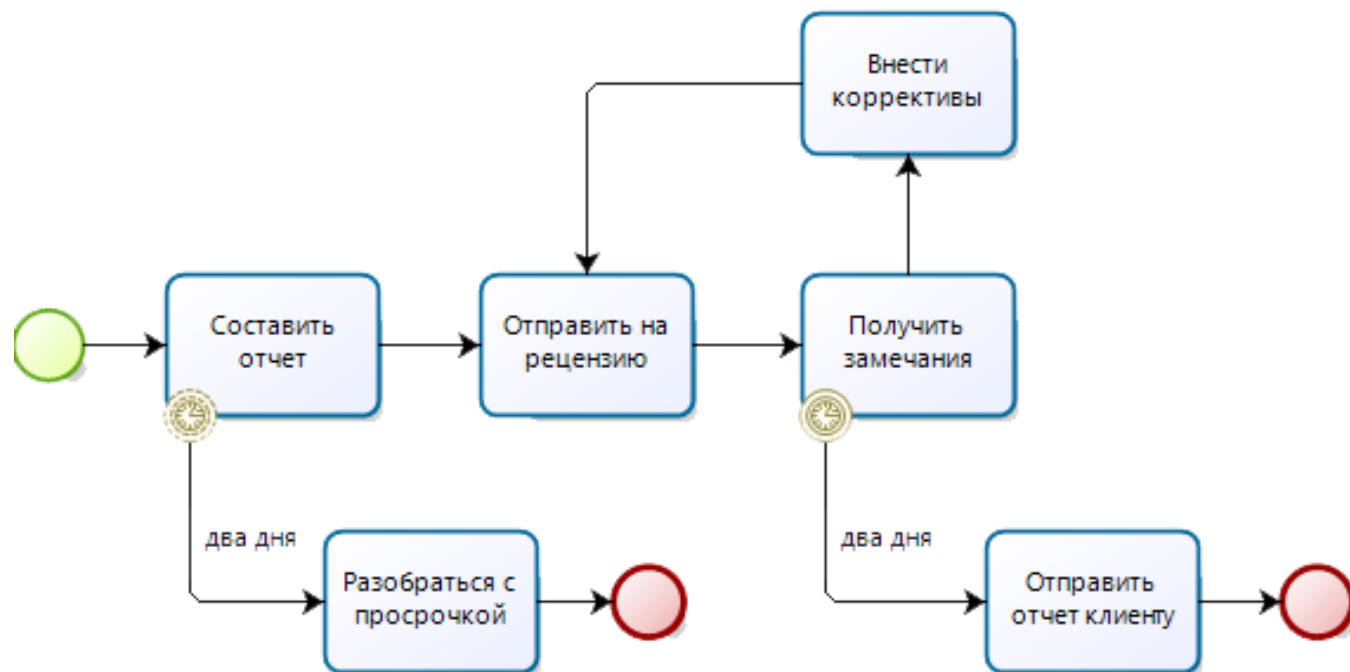
Прерывающее событие
таймер



Непрерывающее
событие таймер

Boundary Event = граничное событие/прикрепленный обработчик

- Прикрепляется к границе задачи или подпроцесса
- Показывает, что делать, если во время выполнения задачи или подпроцесса наступит некоторое событие
 - «Составить отчет» - непрерывающий таймер
 - «Получить замечание» - прерывающий таймер





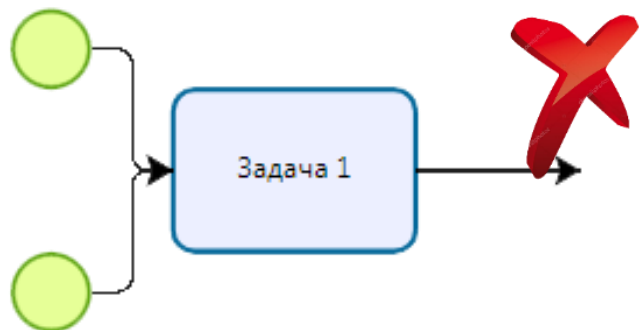
Маркеры типов событий – находятся внутри кружка-события

			Link = соединение частей диаграммы
			Timer = таймер (только обработчик)
			Conditional = бизнес-правило (только обработчик)
			Terminate = завершение всех потоков процесса
			Message = сообщение
			Signal = сигнал, широковещательное сообщение
			Error = ошибка, бизнес-исключение
		2.0	Escalation = эскалация
			Cancel = отмена с откатом транзакции
			Compensation = компенсация при откате транзакции
			Multiple = множественное событие
		2.0	Parallel Multiple = все события из множества

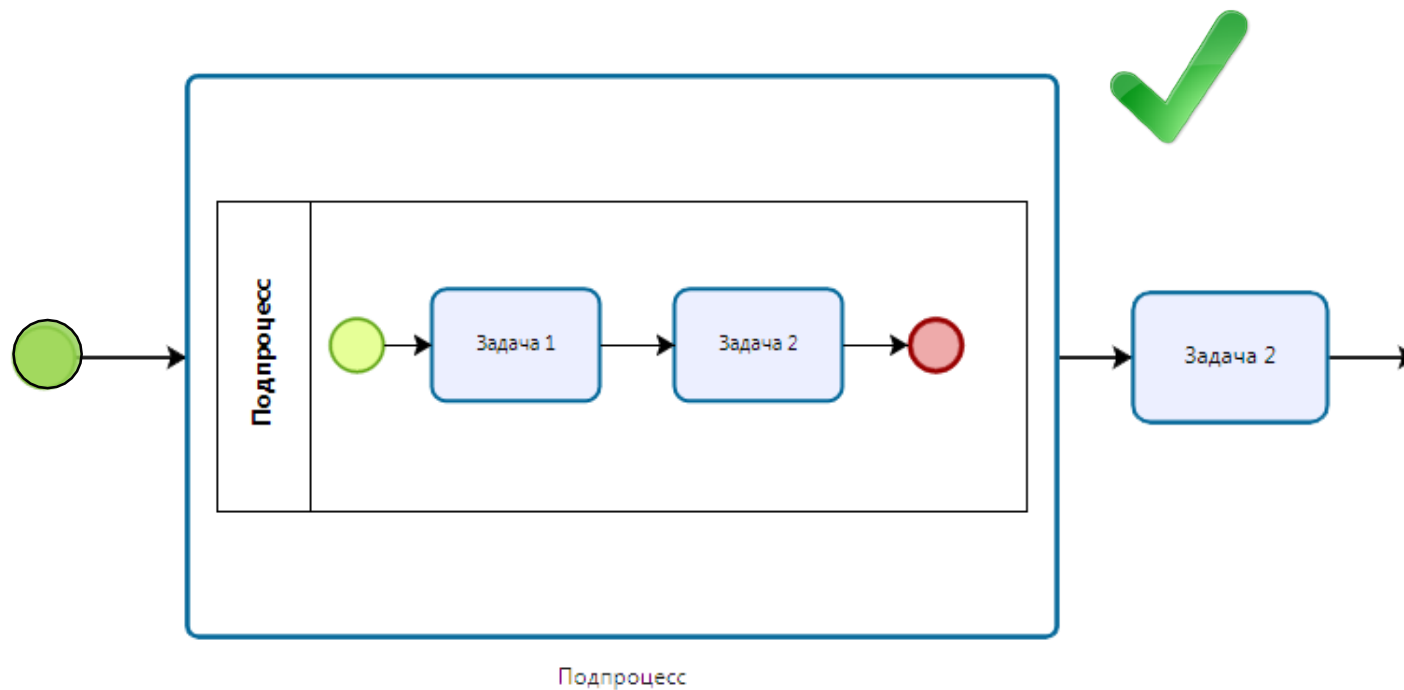
Start None Event = стартовое пустое событие



Запуск процесса вручную
пользователем



Два пустых стартовых события не допускаются



Подпроцесс стартует автоматически

Intermediate None Event = промежуточное пустое событие

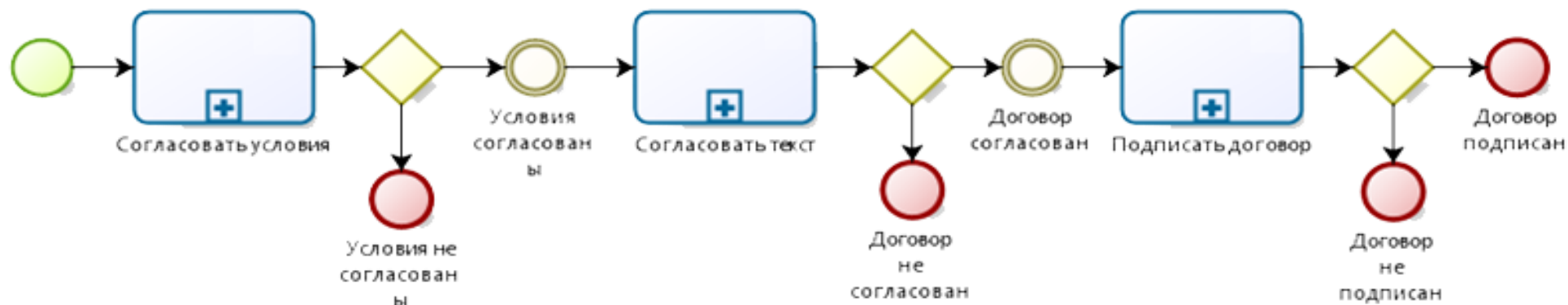


Инициатор (throw)



Обработчик (catch)

Обозначает этапы процесса, которые необходимо выделить. Часто являются лишними.



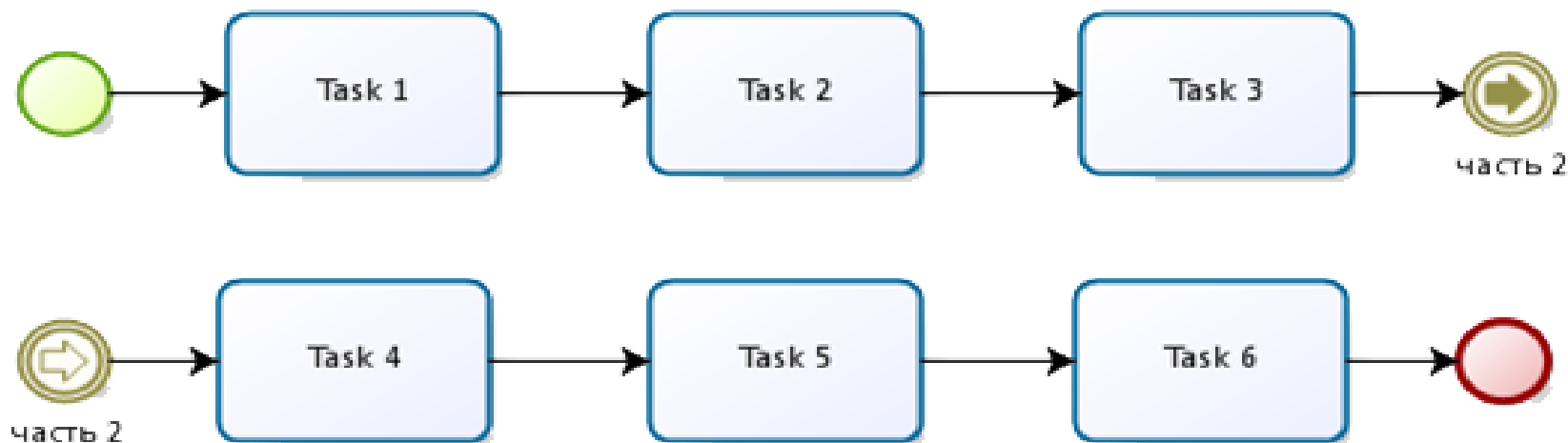
Intermediate Link Event = промежуточное событие-соединение

✓ Инициатор (throw)

✓ Обработчик (catch)

Продолжение диаграммы

- на том же или на другом листе
- связь по имени события

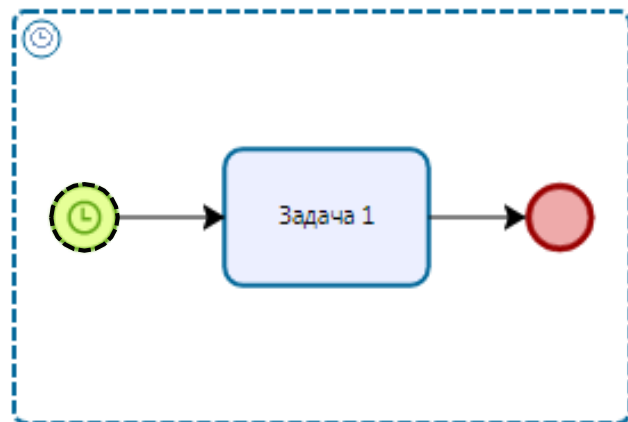


Start Timer Event = стартовое событие таймер

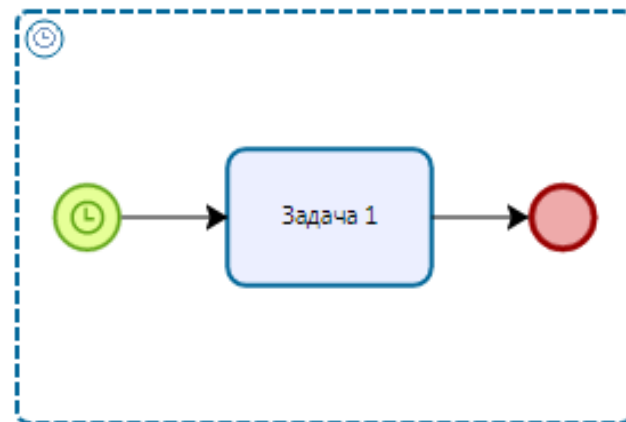
Старт подпроцесса-обработчика событий: при наступлении заданного времени



Процесс запускается по
расписанию



Непрерывающий: запускается
подпроцесс, но действие
основного процесса не
прекращается.



Прерывающий: запускается
подпроцесс, и действие
основного процесса
прекращается.

Intermediate Timer Event= промежуточное событие-таймер



Инициатор (throw)

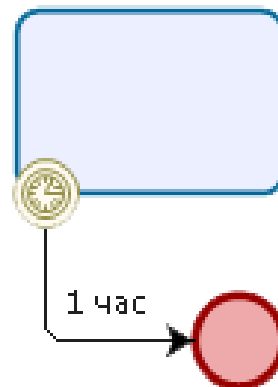
Обработчик (catch)



Процесс продолжится в указанное время.

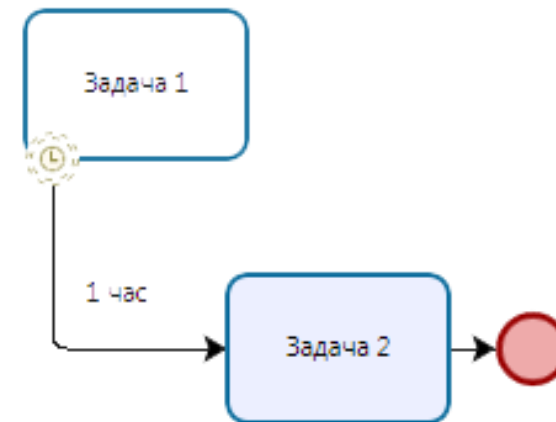
Время ожидания может определяться динамически

Граничное прерывающее



Через указанный промежуток времени сработает обработчик и задача завершится

Граничное непрерывающее



Через указанный промежуток времени сработает обработчик.

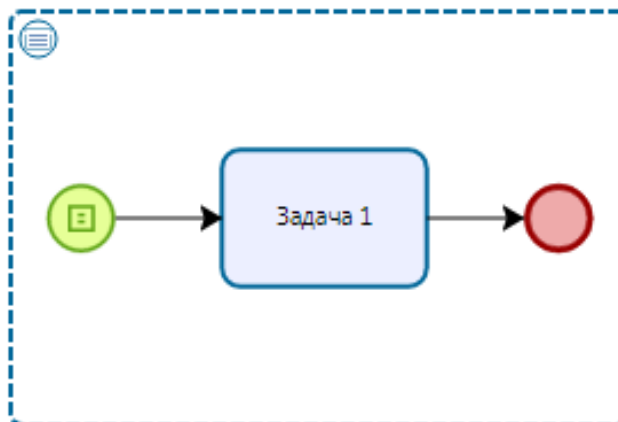
Задача 2 станет активной.
Задача 1 останется активной.

Start Conditional Event = стартовое условное событие

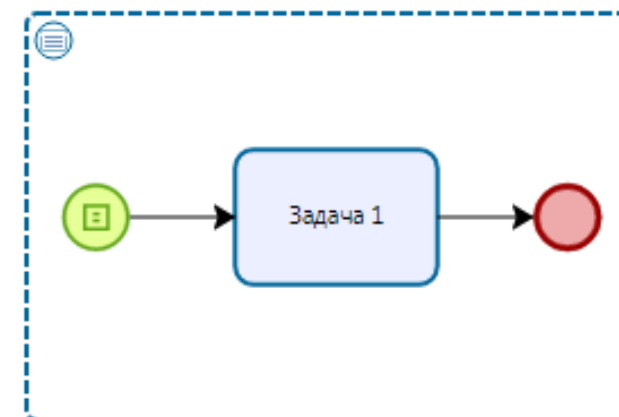


Запуск процесса по срабатыванию заданного логического условия

Старт подпроцесса-обработчика событий: заданное логическое условие принимает значение «истина» (true)



Непрерывающий: запускается подпроцесс, но действие основного процесса не прекращается.



Прерывающий: запускается подпроцесс, и действие основного процесса прекращается.

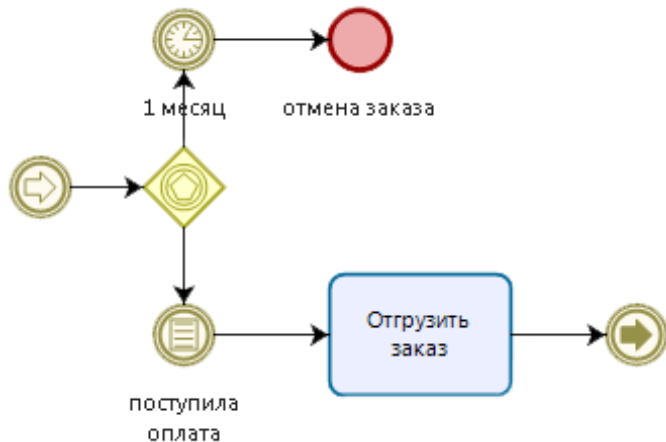
Intermediate Conditional Event = промежуточное условное событие



Инициатор (throw)

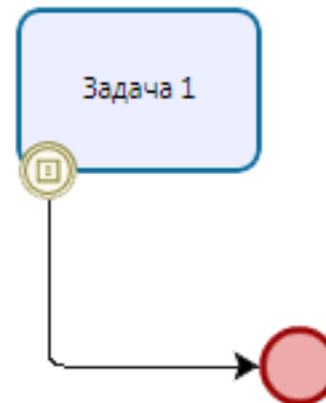


Обработчик (catch)



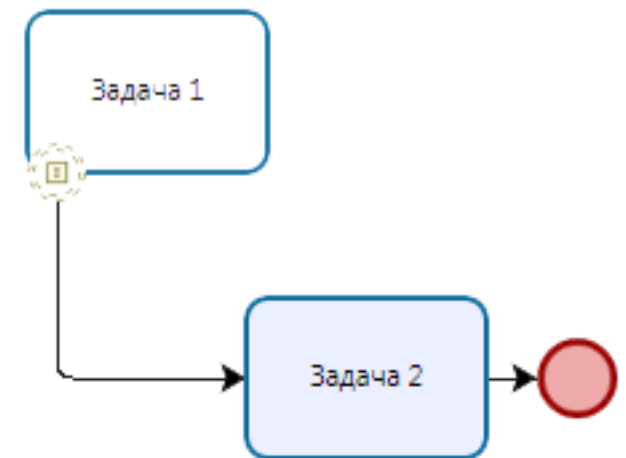
ждать перехода заданного условия из
«ложь» в «истина»

Граничное
прерывающее



При выполнении
заданного логического
условия сработает
обработчик и Задача 1
завершится

Граничное
непрерывающее



При выполнении заданного
логического условия
сработает обработчик.

Задача 2 станет активной.
Задача 1 останется активной.

Событие - Сообщение

Событие наступает при получении или отправке какого-то сообщения



Стартовое событие – «получено SMS с подтверждением оплаты»



Промежуточное событие –
«отправлено техническое задание»
(исходящее); «получено коммерческое
предложение» (входящее)

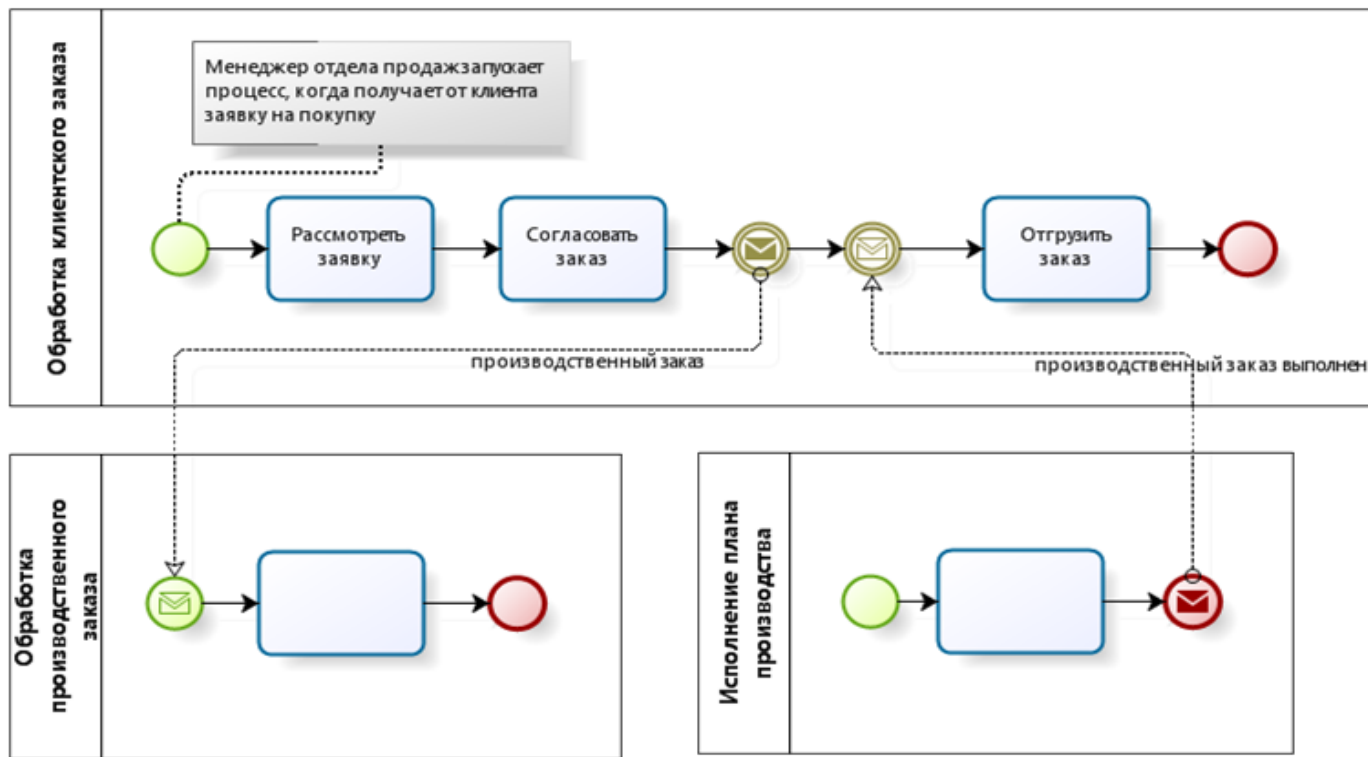


Конечное событие – «клиенту
отправлено подтверждение отгрузки
товара» (исходящее)

- Сообщения используются только для взаимодействия с другими процессами
- Сообщение-отправитель (инициатор) и сообщение-получатель (обработчик) должны находиться в разных процессах
- Инициатор отправляет только одно сообщение (одна исходящая стрелка Message Flow)
- Обработчик получает только одно сообщение (одна входящая стрелка Message Flow)

Потоки сообщений на диаграмме для автоматизации

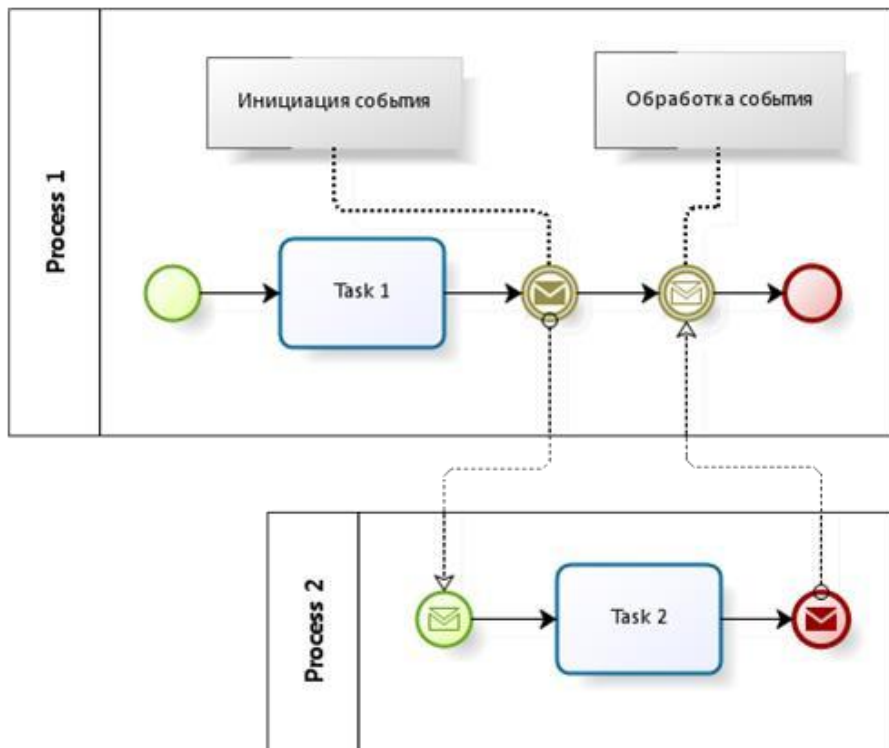
- Синхронизация потоков работ (пулов) с точным указанием точек отправки и получения сообщений



Событие-сигнал и его отличие от события-сообщения

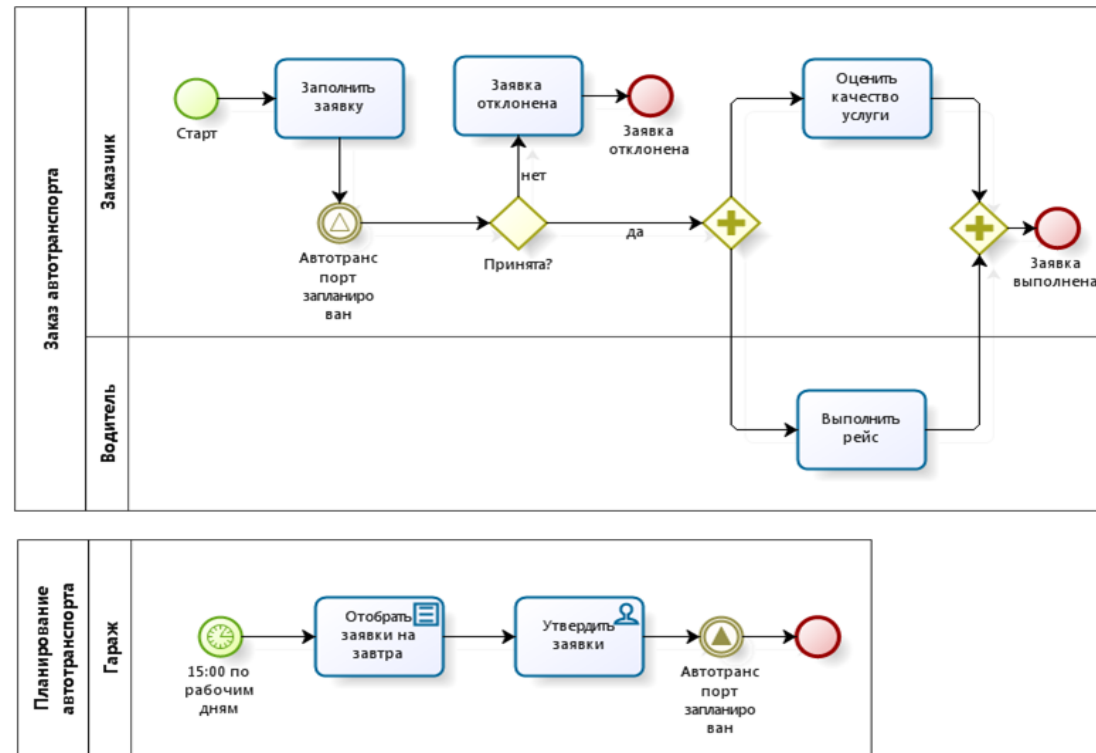
Сообщение

- получает один экземпляр процесса, идентификатор которого указан отправителем



Сигнал

- получают все экземпляры всех процессов, находящиеся на шаге ожидания данного сигнала



Intermediate Signal Event = промежуточное событие-сигнал



Инициатор (throw)



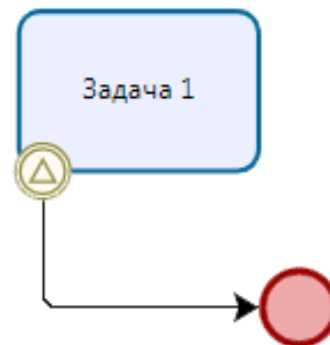
Обработчик (catch)



Инициатор: отправляет сигнал всем процессам, в которых есть событие-обработчик с аналогичным именем (может быть как стартовое так и промежуточное)

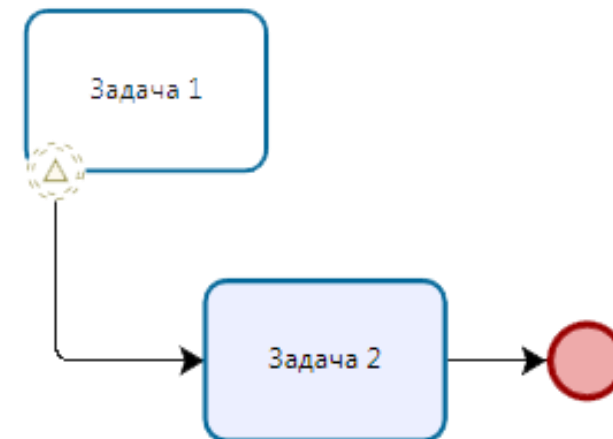
Обработчик: ожидает сигнала, посланного инициатором с аналогичным именем

Граничное прерывающее



При получении сигнала сработает обработчик и Задача 1 завершится

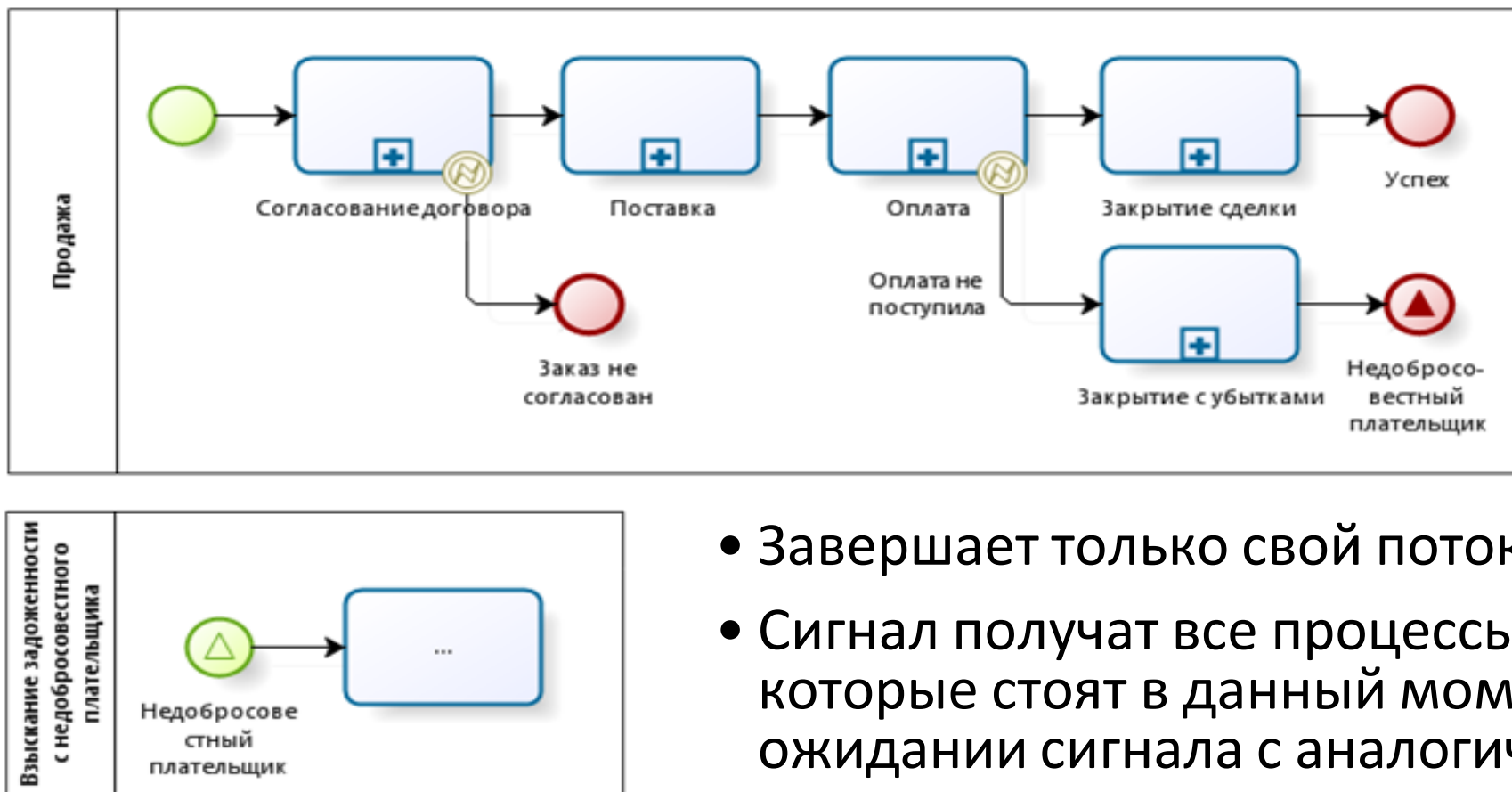
Граничное непрерывающее



При получении сигнала сработает обработчик.

Задача 2 станет активной.
Задача 1 останется активной.

End Signal Event = завершающее событие-сигнал



- Завершает только свой поток
- Сигнал получают все процессы, которые стоят в данный момент на ожидании сигнала с аналогичным именем



Стартовое событие-ошибка (редко используется)

Иницирует запуск подпроцесса в ответ на ошибку, обнаруженную в другом процессе или подпроцессе.

Это специализированное событие, *используемое для динамического запуска рабочих процессов обработки ошибок.*

Причины использования:

- ✓ **Модульная обработка ошибок:** логика обработки ошибок разделена на отдельные подпроцессы, что улучшает организацию процесса и его удобство в обслуживании.
- ✓ **Возможность повторного использования:** подпроцессы обработки ошибок можно повторно использовать в нескольких родительских процессах.
- ✓ **Целевые стратегии восстановления:** позволяют разрабатывать целевые стратегии восстановления для конкретных ошибок, повышая эффективность их устранения.

В автоматизированной системе управления цепочками поставок событие-ошибка запускает подпроцесс «Уведомление поставщика», когда пополнение запасов не происходит из-за проблем с поставщиком. Это событие запускает такие действия, как выбор альтернативного поставщика и анализ последствий.

Уведомление поставщика



Такое событие должно находиться в событийном подпроцессе события (подпроцессе по событиям)

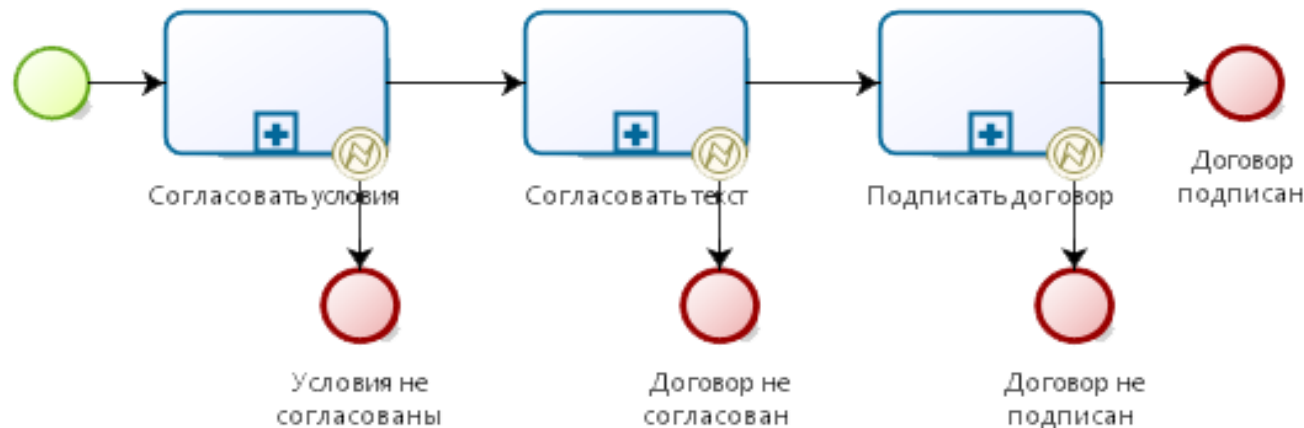


Промежуточное событие-ошибка

Привязано к действию, например к служебной задаче, и предназначено для перехвата ошибок, возникающих во время выполнения этого действия, что позволяет немедленно перейти к обработке ошибки.

Это событие, расположенное в рамках обычного процесса, указывает на то, где может возникнуть ошибка, и обычно используется для программного оповещения об исключении, тем самым передавая ошибку на обработку событию границы ошибки или событию окончания ошибки.

- ✓ упрощает эскалацию критических ошибок, которые невозможно устранить в рамках текущего процесса.
- ✓ обеспечивает надлежащее управление неразрешимыми ошибками в подпроцессах.

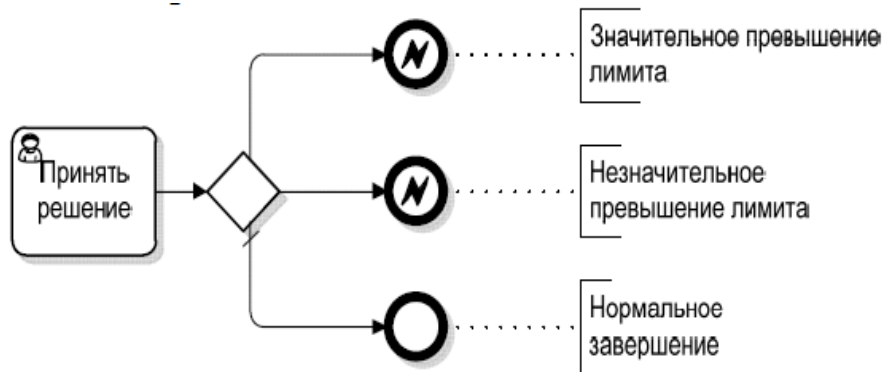




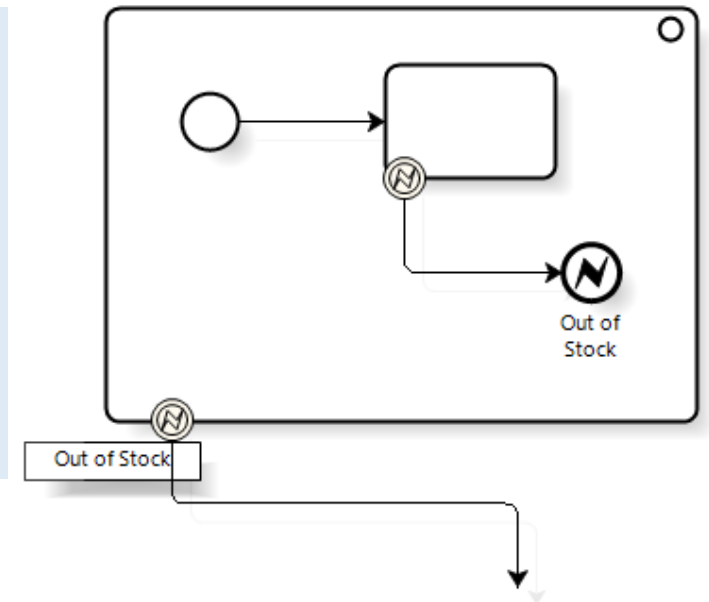
Завершающее событие-ошибка

Означает завершение процесса из-за ошибки и сигнализирует: рабочий процесс не может быть продолжен из-за возникшей проблемы. Ошибка передаётся родительскому процессу для дальнейшей обработки.

- ✓ сигнализирует о завершении текущего процесса из-за ошибки, позволяя родительским процессам или механизмам обработки ошибок более высокого уровня взять на себя управление.
- ✓ гарантирует, что ошибочные процессы не будут продолжены, поддерживая целостность и надёжность всего рабочего процесса.



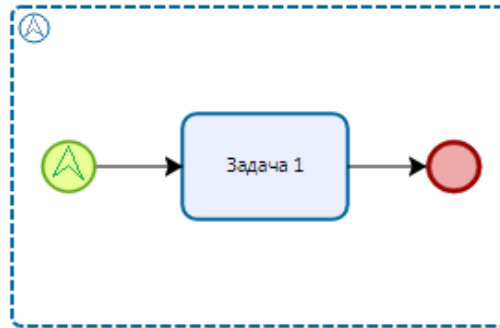
В процессе управления товарными запасами в розничной торговле событие-ошибка в рамках подпроцесса пополнения запасов указывает на то, что для критически важного товара не может быть найдено решение в данный момент. Эта ошибка распространяется на основной процесс управления товарными запасами, что приводит к временной приостановке продаж этого товара.



Событие - эскалация



Стартовое событие-эскалация – используется только в подпроцессах как *старт подпроцесса-обработчика* событий: при выполнении процесса возникло событие «эскалация», требующая дополнительных действий



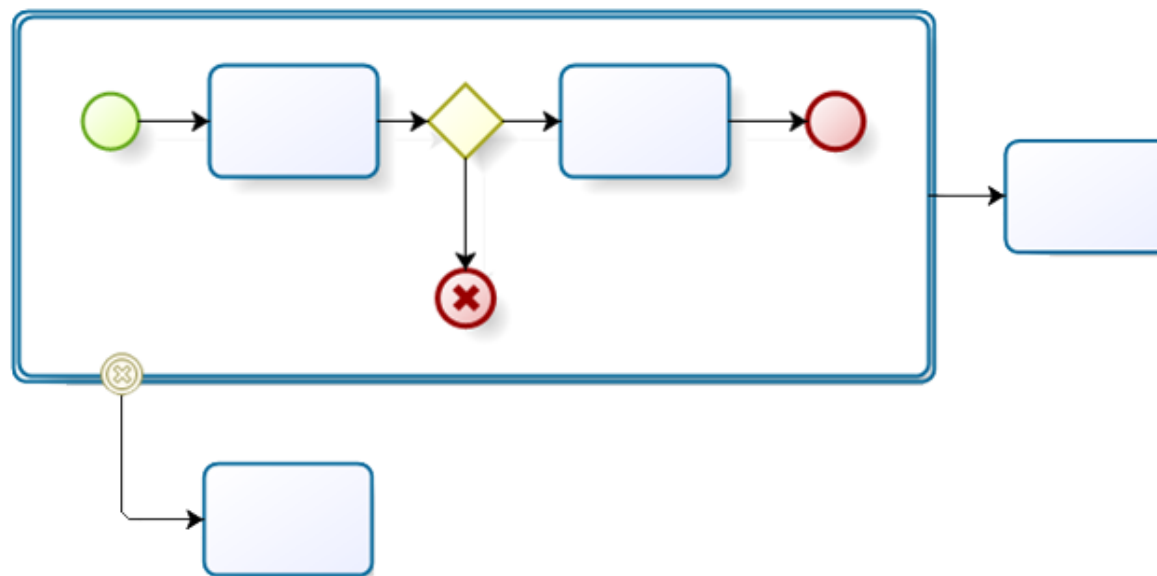
✓ **Промежуточное событие-эскалация (Intermediate Throw Event)** используется **внутри** потока процесса для **инициирования** («бросания») эскалации. Это сигнал о том, что *текущая ситуация внутри процесса требует внимания извне данного процесса.*



✓ **Завершающее событие-эскалация (End Event):** Используется для **завершения** потока процесса с генерацией эскалации. Это означает, что сам процесс не может продолжаться дальше и передает управление на более «высокий» уровень.

End Cancel Event = завершающее событие отмена

- Применимо только в транзакционном подпроцессе
 - изображен на диаграмме двойной рамкой
- Вызывает откат процесса либо компенсацию
- Прикрепленный обработчик перехватит управление, когда будут выполнены все компенсационные действия
- Прерывает все потоки процесса и все вложенные подпроцессы
- Передает управление граничному событию-обработчику
 - если нет обработчика, то завершает родительские процессы (пока не встретит обработчик либо в т.ч. процесс верхнего уровня)





Compensation Event = компенсация



В ходе процесса возникла необходимость отменить все выполненные ранее действия:

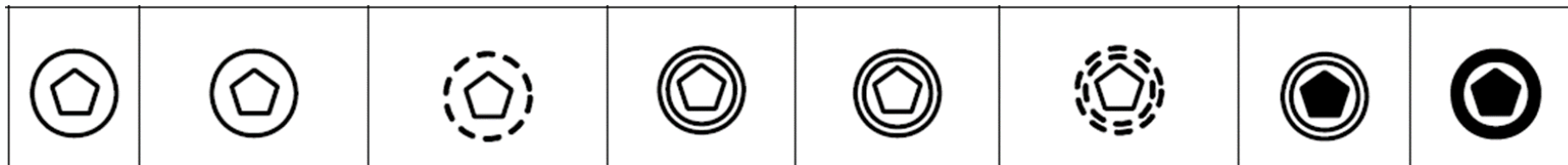
- при возврате покупки вернуть сумму, списанную со счета банковской карты
- при технических проблемах при получении денег в банкомате, отменить транзакцию и вернуть деньги на счет

Логика компенсации может оказаться сложнее:

- при отказе от заказа, если товар доставлен, его следует забрать
- если не удалось забронировать гостиницу, надо отказаться от билетов на самолет, и наоборот
- если при расчетах за товар были применены бонусы, то следует возместить не только деньги, но и бонусы
- Механизм «Компенсация» позволяет упростить откат операций



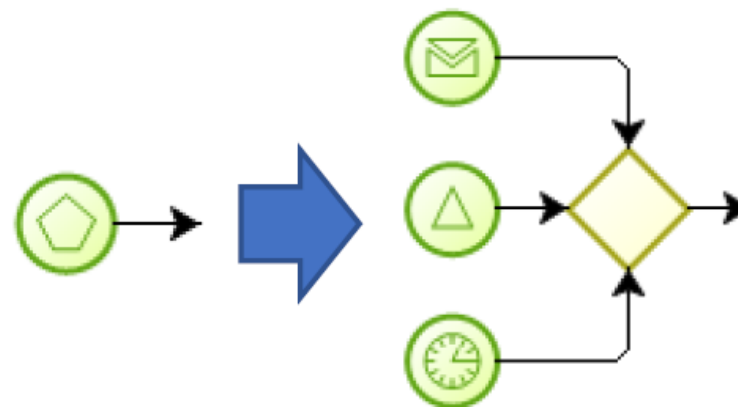
Multiple Event = множественное (составное) событие



Используется, когда возможны несколько вариантов наступления события или запуска процесса. Но для запуска процесса необходим лишь один из таких способов.

Пример: обработка заказа начнётся, если клиент оформит заказ на сайте, позвонит по телефону или лично придёт в магазин.

Не рекомендуется при подробном описании, особенно, если необходимо интегрировать модель с автоматизацией. Лучше использовать несколько отдельных типов событий:





Parallel Multiple Event = параллельное множественное событие



НЕОБХОДИМОСТЬ задействования множества способов инициирования Процесса до момента его запуска.

Аналогично множественному, но событие произойдёт, только если будут **выполнены все заданные условия**.

Пример: сделка купли-продажи недвижимого имущества вступает в силу при подписании договора покупателем, продавцом, и при государственной регистрации сделки.

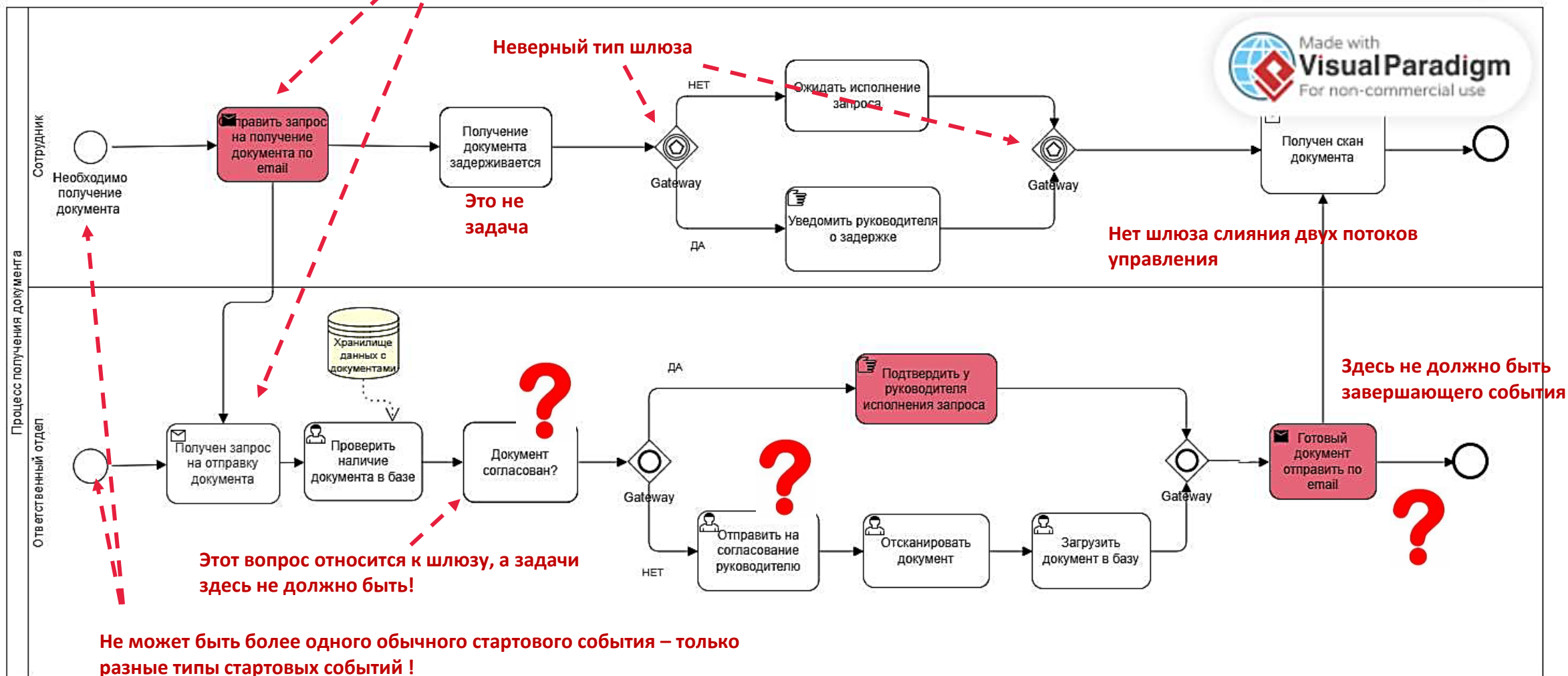
Не рекомендуется при подробном описании, особенно, если необходимо интегрировать модель с автоматизацией. Лучше использовать несколько отдельных типов событий.



Задачи-сообщения нежелательно отображать в рамках одного пула, они предназначены для межпроцессного взаимодействия (между двумя и более пулами)

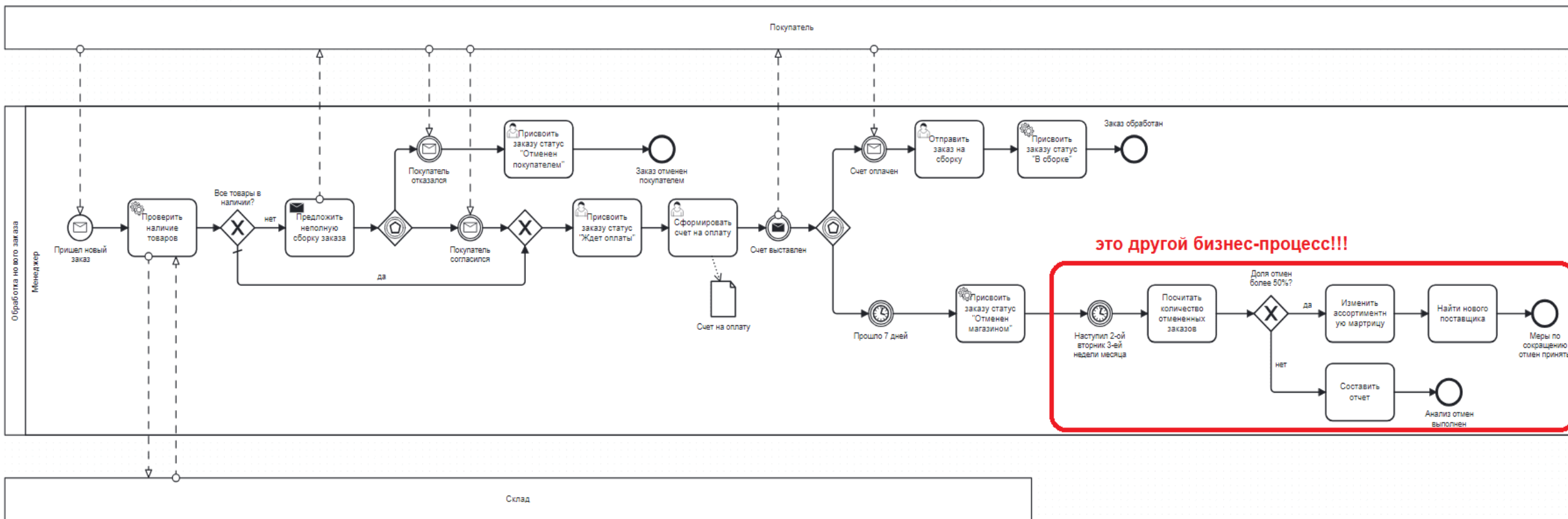


Задачи «Получен запрос....», «Получен скан....» неверно сформулированы (как события)

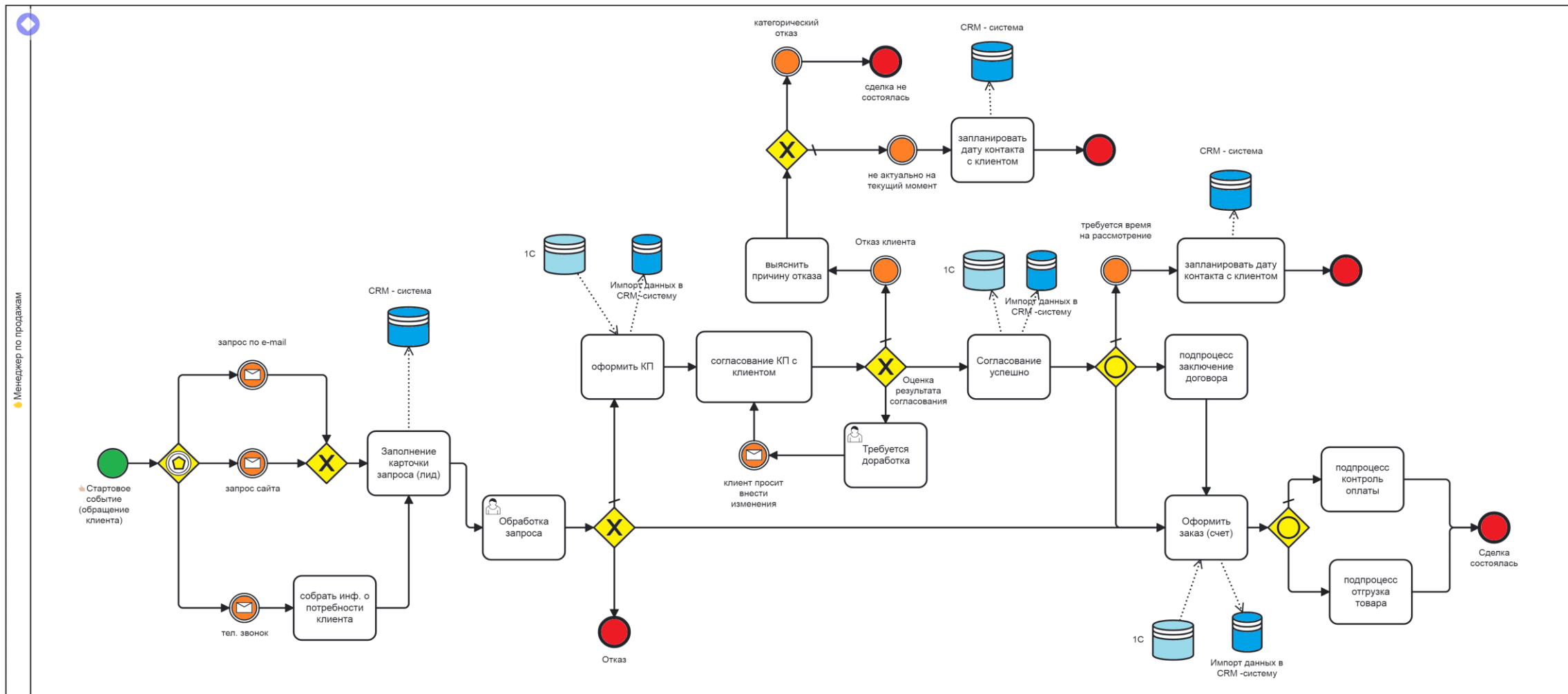




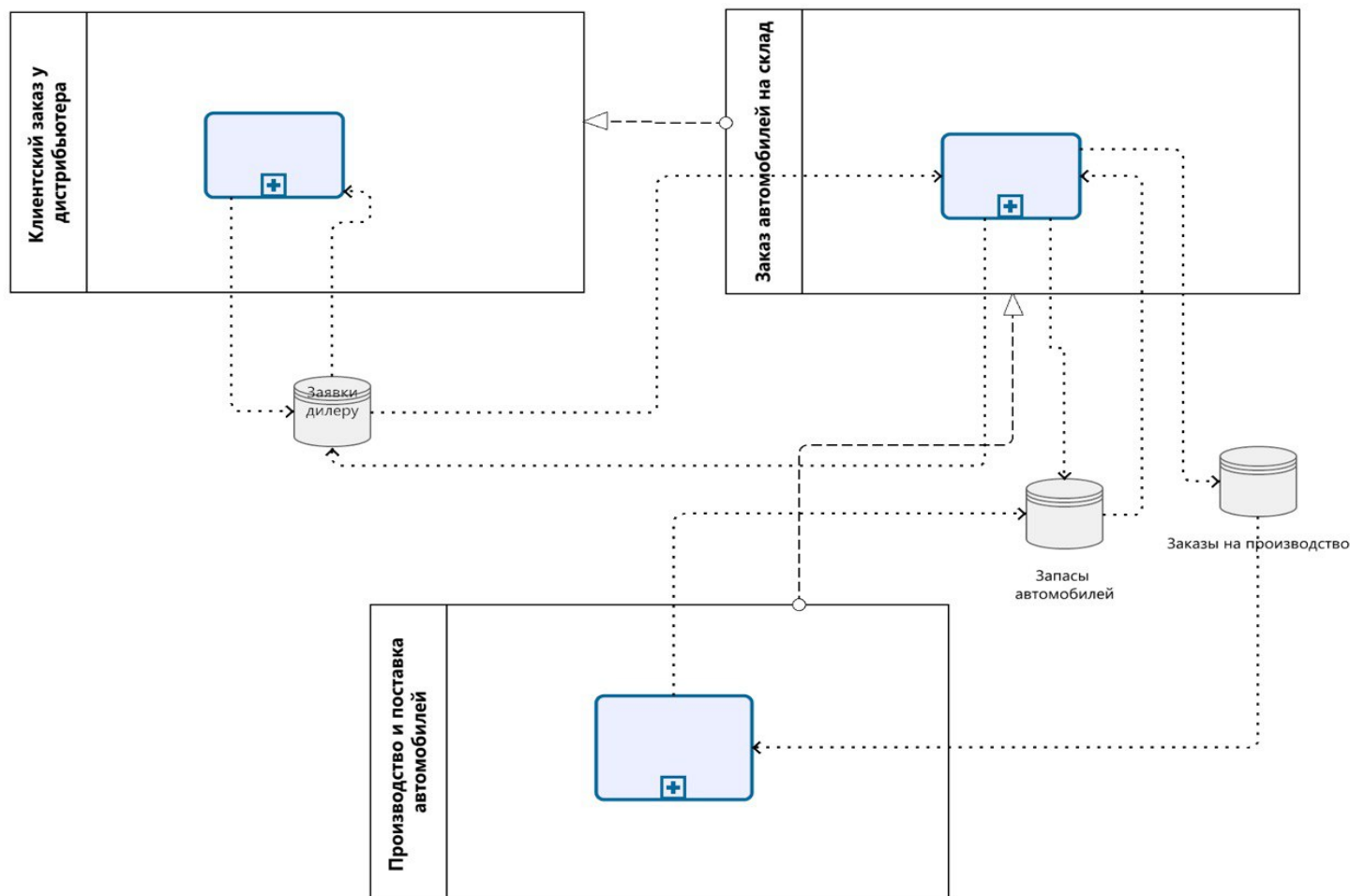
Ошибки моделирования из-за неверного определения границ процессов



Нужно было бы сделать диаграмму
взаимодействия с потоками сообщений!

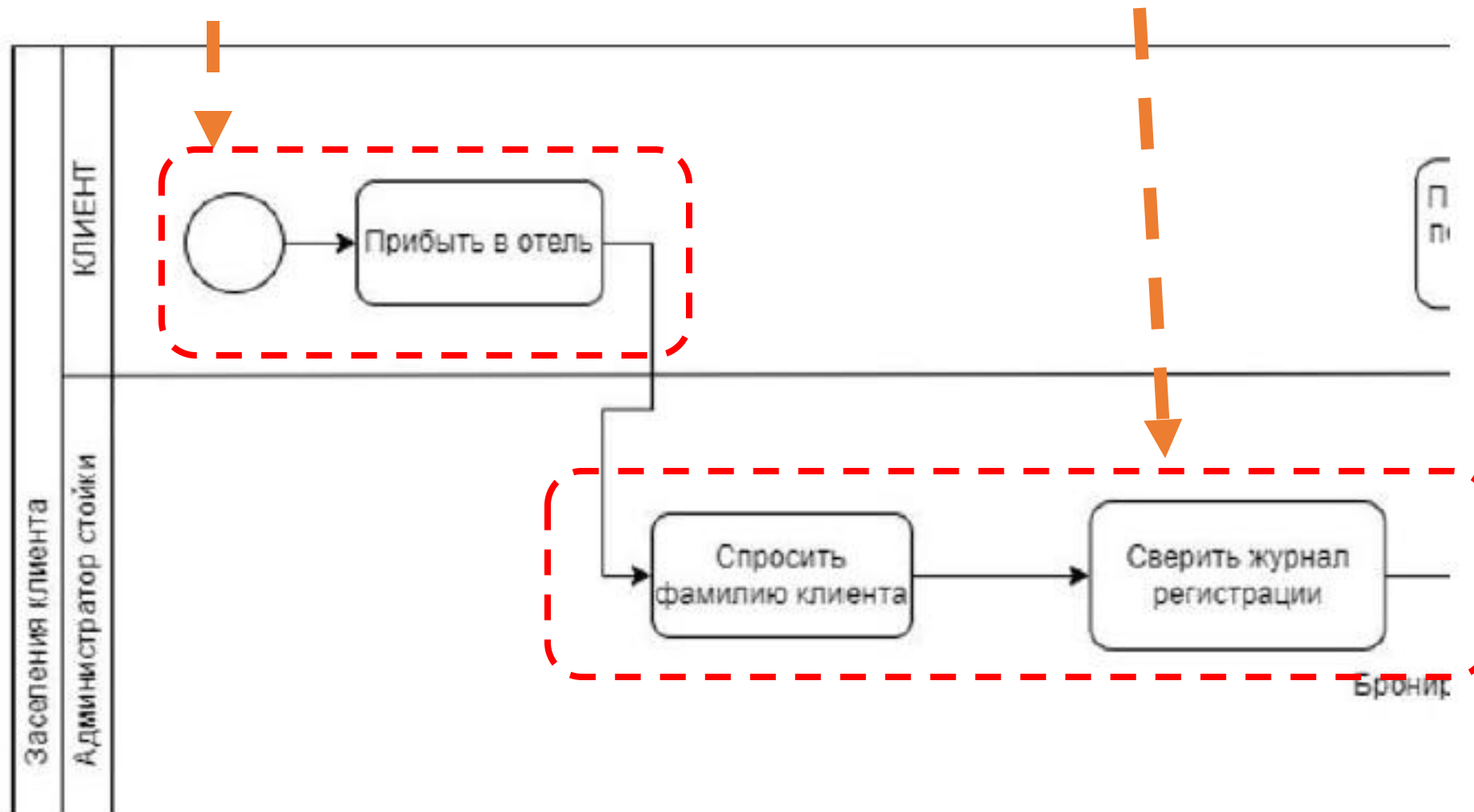


Чтобы не было путаницы и перегруженных моделей, сначала сделать верхнеуровневое межпроцессное взаимодействие BPMN, а потом уже переходить к детализации на основе приоритетов

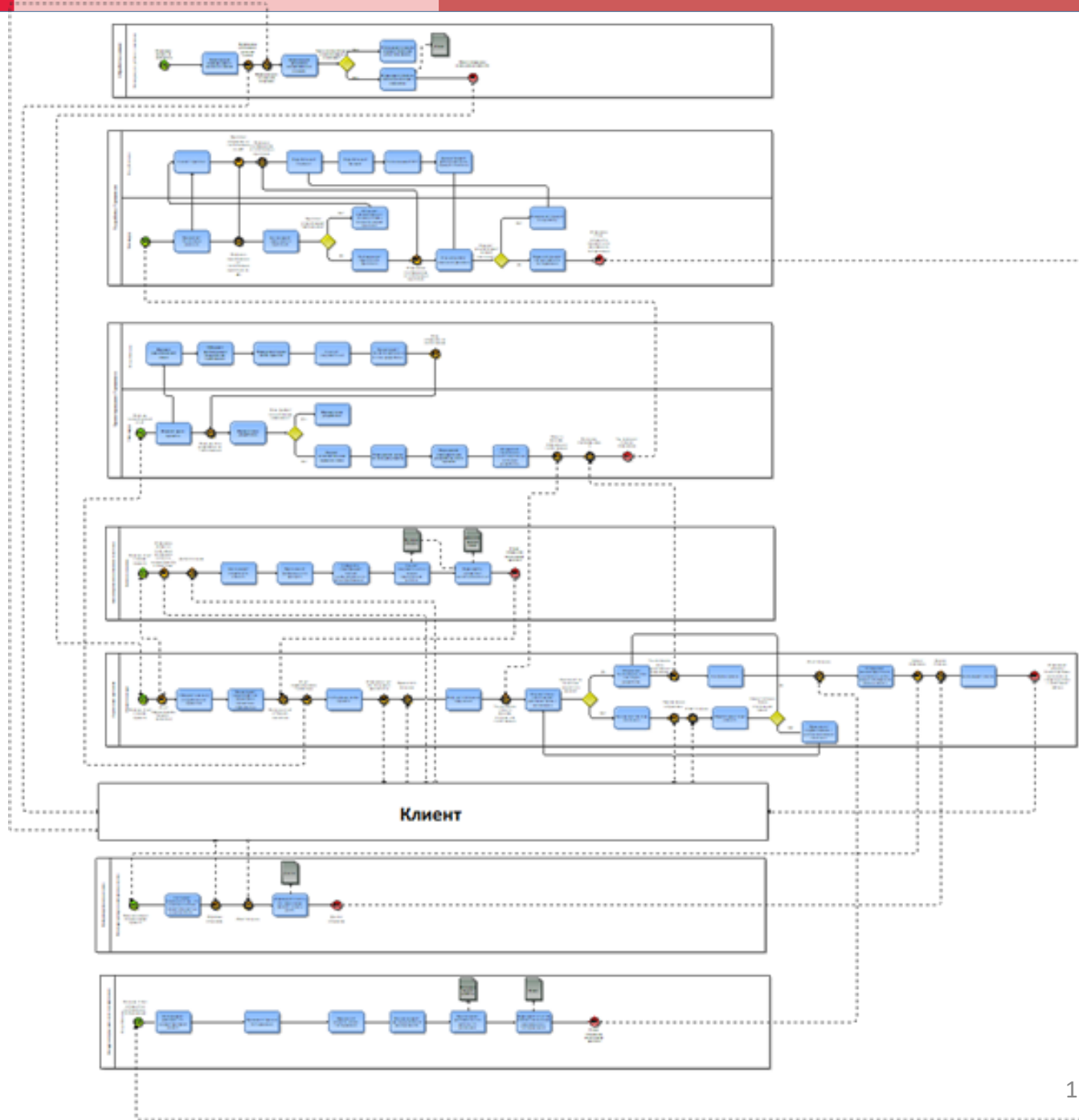


Частые ошибки

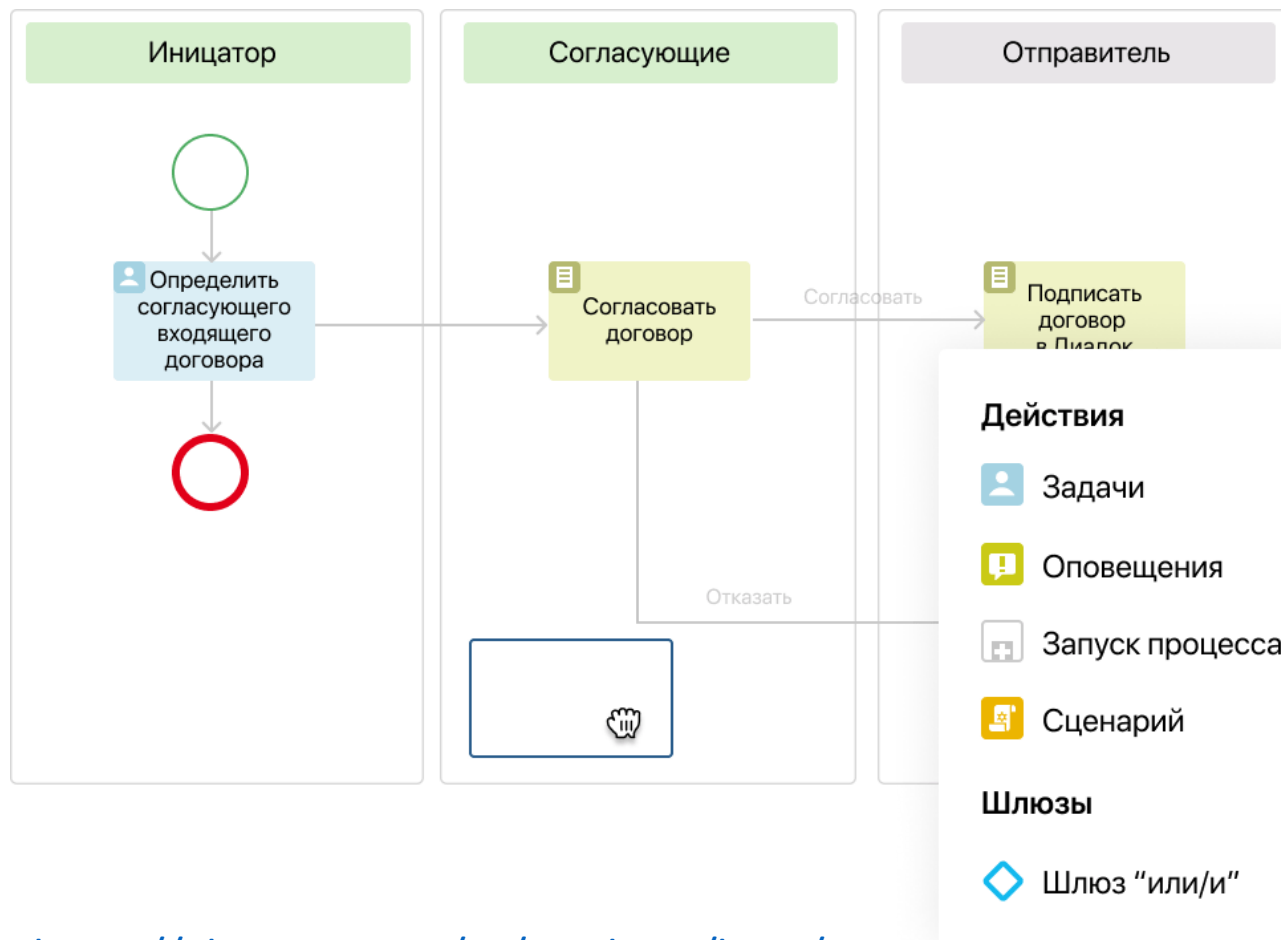
- задача как событие и наоборот, излишнее разбиение одного функционала на несколько задач



Слишком сложная модель,
нечитаема!
– нужно разделить на части



Реализация в ПО ELMA365



Наряду со стандартными элементами BPMN разработаны специальные блоки для упрощения моделирования. Например, готовые активности ECM помогают интерпретировать такие действия, как

- ✓ подписание и согласование документов, генерация по шаблону. Отправка SMS или email — своевременно информировать клиентов и сотрудников, статусы — отслеживать ход процесса. Если доступных элементов
- ✓ недостаточно, всегда можно реализовать необходимую бизнес-логику с помощью сценариев



Плюсы и минусы VRMN2.0.

- Открытый код – широкое распространение
- Подходит для описания практически любых процессов разного уровня детализации и специфики
- Большое количество ПО, материалов
- Не предназначена для описания верхнеуровневых процессов, нет связи со стратегией, с другими предметными областями
- На экспертном, аналитическом уровне использования требует тщательного обучения
- Не обновлялась более 10 лет



Выводы

- Выбор той или иной нотации часто связан с методологией моделирования, проектными задачами, корпоративными стандартами и возможностями использования
- Эволюция методологий моделирования, с учетом достижений ИТ, привела от сложных и дорогостоящих аналитических инструментов к тренду Low-code и упрощенных нотаций, вовлекающих различные роли как в описание, так и в совершенствование процессов
- Возникшие на разных этапах развития ИТ и соответствующих методологий, нотации детализированного описания процессов являются фундаментом не только для прикладного, но и научно-исследовательского применения, поскольку развивают логику и системное мышление



Высшая
школа
бизнеса